

Table of Contents

PART 3 — CLUSTERING	6
What is Clustering? — ক্লাস্টারিং কী?	7
Chapters in this Part	8
এই তিনটি অ্যালগরিদম কীভাবে আলাদা এবং কখন কোনটি ব্যবহার করবেন?	8
এই Part কীভাবে পরবর্তী Dimensionality Reduction অধ্যায়ের জন্য প্রস্তুত করে	9
Chapter 14 — K-Means Clustering	10
1. Introduction	10
K-Means Clustering	10
2. Why This Algorithm Was Developed	11
3. Real-life Motivation	12
4. Intuition	12
5. Working Principle	12
6. Mathematical Backend	13
7. Formula	15
8. Formula Derivation	16
9. Meaning of Every Symbol	17
10. Step-by-Step Formula Explanation	17
11. Numerical Example	18
12. Visualization	22
Visualization — ASCII Diagram	22
.....	22
13. Hyperparameters	23
14. Scikit-learn Import	27
15. Python Example	28
From-Scratch বাস্তবায়ন (NumPy ব্যবহার করে)	28
Scikit-learn ব্যবহার করে বাস্তবায়ন	29
16. Prediction / Clustering Process	30
17. Advantages	31
18. Disadvantages	31
19. Applications	33
20. Comparison with Previous Algorithm	33
21. Common Mistakes	33

22. Interview Questions.....	34
23. Summary	35
24. Complete Mathematical Implementation (Full Manual Walkthrough)	36
Dataset	37
Initialization	37
Core Formula	37
Distance Calculation এবং Iteration 1	37
Iteration 2 (Round 2)	38
Stopping Condition.....	39
Training Progress Table	39
Final Cluster Assignment.....	39
Prediction (নতুন Sample-এর জন্য).....	39
Interpretation.....	39
Chapter 15 — Agglomerative Clustering.....	40
1. Introduction.....	40
Agglomerative Clustering	40
2. Why This Algorithm Was Developed.....	41
3. Real-life Motivation.....	41
4. Intuition.....	41
উদাহরণ (Intuition).....	41
5. Working Principle.....	42
মূল গাঠনিক প্রক্রিয়া (How it Works) — ধাপে ধাপে.....	42
ধাপ ১ — প্রাথমিক অবস্থা.....	42
ধাপ ২ — দূরত্ব গণনা.....	42
ধাপ ৩ — সবচেয়ে কাছের জোড়া একত্রিত করা	42
ধাপ ৪ — Distance Matrix আপডেট ও পুনরাবৃত্তি	42
ধাপ ৫ — ফলাফল: Dendrogram	42
6. Mathematical Backend	42
Linkage Criteria — কীভাবে দুই Cluster-এর দূরত্ব মাপা হয়?.....	42
7. Formula.....	43
গাণিতিক ভিত্তি — Ward's Method	43
8. Formula Derivation.....	43
9. Meaning of Every Symbol.....	44

10. Step-by-Step Formula Explanation.....	44
11. Numerical Example	44
সংখ্যাসূচক উদাহরণ — সম্পূর্ণ ধাপে ধাপে প্রক্রিয়া (1D ডেটা, Single Linkage)	44
12. Visualization	45
Visualization — ASCII Dendrogram.....	45
13. Hyperparameters	45
গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা.....	45
n_clusters — Default: 2	45
linkage — Default: 'ward'	46
distance_threshold — Default: None	46
গুরুত্বপূর্ণ অ্যাট্রিবিউট.....	46
14. Scikit-learn Import	47
15. Python Example	47
From-Scratch বাস্তবায়ন (Ward Linkage, NumPy)	47
Scikit-learn ব্যবহার করে বাস্তবায়ন.....	48
16. Prediction / Clustering Process.....	48
17. Advantages	48
18. Disadvantages.....	49
19. Applications	49
কোন কোন সেক্টরে ব্যবহার হয়?	49
20. Comparison with Previous Algorithm.....	50
K-Means বনাম Agglomerative Clustering.....	50
21. Common Mistakes.....	50
22. Interview Questions.....	51
23. Summary	51
Quick Revision.....	51
24. Complete Mathematical Implementation (Full Manual Walkthrough).....	52
Dataset.....	52
Initialization	52
Core Formula — Ward's Method.....	52
Iteration 1 — প্রাথমিক Distance Matrix	52
Iteration 2 — Distance Matrix Update.....	53
Iteration 3 — Distance Matrix Update.....	53

Iteration 4 — চূড়ান্ত Merge.....	54
Merge Progress Table	54
Final Cluster Assignment (<i>n_clusters</i> = 2 ধরলে)	54
Interpretation.....	54
Chapter 16 — DBSCAN Clustering.....	55
1. Introduction.....	55
2. Why This Algorithm Was Developed.....	56
3. Real-life Motivation.....	57
4. Intuition.....	58
5. Working Principle.....	58
6. Mathematical Backend	59
7. Formula.....	63
8. Formula Derivation.....	64
9. Meaning of Every Symbol.....	65
10. Step-by-Step Formula Explanation.....	65
11. Numerical Example	67
12. Visualization	68
Visualization — ASCII Diagram.....	68
13. Hyperparameters	68
14. Scikit-learn Import.....	73
15. Python Example	73
From-Scratch বাস্তবায়ন (NumPy ব্যবহার করে)	73
Scikit-learn ব্যবহার করে বাস্তবায়ন.....	74
16. Prediction / Clustering Process.....	74
17. Advantages.....	75
18. Disadvantages.....	76
19. Applications.....	76
20. Comparison with Previous Algorithm.....	76
K-Means বনাম DBSCAN	76
21. Common Mistakes.....	77
22. Interview Questions.....	78
23. Summary	79
Hierarchical DBSCAN (HDBSCAN).....	81

24. Complete Mathematical Implementation (Full Manual Walkthrough)	81
Dataset (সম্প্রসারিত)	81
Distance Calculation — A5 -এর জন্য	81
Cluster Assignment (আপডেট করা)	82
Progress Table (সম্পূর্ণ, ৫টি পয়েন্ট)	82
Interpretation.....	82

PART 3 — CLUSTERING

From the Machine Learning Handbook — A Professionally Organized Guide

Chapters in this Part

- Chapter 14 — K-Means Clustering
- Chapter 15 — Agglomerative Clustering
- Chapter 16 — DBSCAN Clustering

What is Clustering? — ক্লাস্টারিং কী?

Clustering হলো একটি Unsupervised Learning টেকনিক, যেখানে একটি ডেটাসেটের মধ্যে থাকা ডেটা পয়েন্টগুলোকে তাদের পারস্পরিক সাদৃশ্য (Similarity) বা দূরত্বের (Distance) ভিত্তিতে স্বয়ংক্রিয়ভাবে একাধিক গ্রুপ বা “Cluster”-এ ভাগ করা হয়। একই Cluster-এর অন্তর্ভুক্ত ডেটা পয়েন্টগুলো একে অপরের সাথে যতটা সম্ভব সাদৃশ্যপূর্ণ থাকে, আর ভিন্ন Cluster-এর ডেটা পয়েন্টগুলোর মধ্যে যতটা সম্ভব অসাদৃশ্য (Dissimilarity) থাকে।

Clustering বনাম Classification — মূল পার্থক্য কোথায়?

Classification এবং Clustering দুটোই ডেটাকে গ্রুপে ভাগ করে বলে অনেক সময় বিভ্রান্তি তৈরি হয়, কিন্তু তাদের মধ্যে একটি মৌলিক পার্থক্য রয়েছে।

- Classification হলো একটি **Supervised Learning** সমস্যা — মডেলকে ট্রেনিং করানোর সময় প্রতিটি ডেটার সঠিক Label বা Class আগে থেকেই দেওয়া থাকে (যেমন: Spam/Not Spam)। মডেল সেই Label থেকে শিখে নতুন ডেটার Label ভবিষ্যদ্বাণী করে।
- Clustering হলো একটি **Unsupervised Learning** সমস্যা — কোনো Label একেবারেই দেওয়া থাকে না। মডেলকে নিজে থেকেই ডেটার অন্তর্নিহিত গঠন (Hidden Structure) আবিষ্কার করে বলতে হয় কোন ডেটাগুলো একসাথে থাকা উচিত।

কেন Clustering-এ Label প্রয়োজন হয় না?

বাস্তব জগতের অধিকাংশ ডেটার কোনো পূর্বনির্ধারিত Label থাকে না, এবং প্রতিটি ডেটাকে ম্যানুয়ালি Label করা প্রায়শই সময়সাপেক্ষ, ব্যয়বহুল, এমনকি অসম্ভবও হতে পারে। Clustering অ্যালগরিদম শুধুমাত্র ডেটা পয়েন্টগুলোর জ্যামিতিক অবস্থান, দূরত্ব বা ঘনত্ব বিশ্লেষণ করেই সম্পূর্ণ স্বাধীনভাবে গ্রুপ তৈরি করতে পারে, ফলে Label তৈরির প্রয়োজনীয়তা সম্পূর্ণভাবে দূর হয়ে যায়।

Clustering কোথায় ব্যবহৃত হয়? — বাস্তব জীবনের প্রয়োগ

- **Customer Segmentation:** গ্রাহকদের ক্রয়-আচরণ, ব্যয়ের ধরন ও আগ্রহের ভিত্তিতে স্বয়ংক্রিয়ভাবে বিভিন্ন গ্রাহক-শ্রেণিতে ভাগ করে টার্গেটেড মার্কেটিং করা।
- **Image Segmentation:** একটি ছবির সমজাতীয় Pixel বা Color Region-গুলোকে একত্রিত করে বস্তু শনাক্তকরণ বা মেডিক্যাল ইমেজিং-এ টিউমার শনাক্ত করা।
- **Anomaly ও Fraud Detection:** স্বাভাবিক লেনদেনের ঘনত্বপূর্ণ Cluster থেকে দূরে থাকা অস্বাভাবিক লেনদেনকে (Outlier) শনাক্ত করা।
- **Document ও Topic Clustering:** সংবাদ প্রতিবেদন বা গবেষণাপত্রকে বিষয়ভিত্তিক গ্রুপে ভাগ করা।
- **Genomics ও Bioinformatics:** জিন এক্সপ্রেশন ডেটা বিশ্লেষণ করে জীবের বিবর্তনীয় সম্পর্ক নির্ণয় করা।

- **Recommendation Systems:** সমরুচির ব্যবহারকারীদের একত্রিত করে পণ্য বা কনটেন্ট সুপারিশ করা।

Chapters in this Part

- **Chapter 14 — K-Means Clustering:** একটি Centroid-ভিত্তিক, Iterative Partitional Clustering অ্যালগরিদম, যা পূর্বনির্ধারিত সংখ্যক (K) গোলাকার Cluster তৈরি করে।
- **Chapter 15 — Agglomerative Clustering:** একটি Bottom-Up Hierarchical Clustering পদ্ধতি, যা ধাপে ধাপে নিকটতম Cluster-জোড়া একত্রিত করে একটি Dendrogram তৈরি করে।
- **Chapter 16 — DBSCAN Clustering:** একটি Density-ভিত্তিক Clustering অ্যালগরিদম, যা Cluster সংখ্যা আগে থেকে না জেনেই যেকোনো আকৃতির Cluster খুঁজে বের করতে পারে এবং Noise স্বয়ংক্রিয়ভাবে সনাক্ত করে।

এই তিনটি অ্যালগরিদম কীভাবে আলাদা এবং কখন কোনটি ব্যবহার করবেন?

তিনটি অ্যালগরিদমই Unsupervised হলেও তাদের কার্যপ্রণালীর দর্শন সম্পূর্ণ ভিন্ন।

- **K-Means** দূরত্ব-ভিত্তিক গড় (Mean) ব্যবহার করে Cluster তৈরি করে এবং Cluster সংখ্যা (K) আগে থেকে নির্দিষ্ট করে দিতে হয়। এটি দ্রুত ও Scalable, তবে শুধুমাত্র গোলাকার (Convex) আকৃতির Cluster ভালোভাবে শনাক্ত করতে পারে এবং Outlier-এর প্রতি সংবেদনশীল।
- **Agglomerative Clustering** কোনো পূর্বনির্ধারিত Cluster সংখ্যা ছাড়াই ধাপে ধাপে সব ডেটাকে একত্রিত করে একটি সম্পূর্ণ Hierarchy (Dendrogram) তৈরি করে, যা থেকে পরবর্তীতে যেকোনো সংখ্যক Cluster বেছে নেওয়া যায়। এটি সম্পূর্ণ Deterministic, তবে বড় ডেটাসেটে ধীরগতির।
- **DBSCAN** ঘনত্ব (Density) পরিমাপ করে Cluster তৈরি করে, ফলে এটি যেকোনো আকৃতির (Arbitrary-shaped) Cluster শনাক্ত করতে পারে এবং Noise বা Outlier-কে স্বয়ংক্রিয়ভাবে আলাদা করে ফেলে। তবে এর ফলাফল ϵ ও min_samples হাইপারপ্যারামিটারের সঠিক নির্বাচনের ওপর অত্যন্ত নির্ভরশীল।

সাধারণ নিয়ম হিসেবে — Cluster-এর সংখ্যা জানা থাকলে ও গতি প্রয়োজন হলে **K-Means**; সম্পূর্ণ Hierarchy বা Dendrogram-ভিত্তিক বিশ্লেষণ প্রয়োজন হলে **Agglomerative**; আর Cluster-এর আকৃতি অনিয়মিত এবং Noise আলাদা করা জরুরি হলে **DBSCAN** ব্যবহার করা হয়।

এই Part কীভাবে পরবর্তী Dimensionality Reduction অধ্যায়ের জন্য প্রস্তুত করে

Clustering অ্যালগরিদমগুলো সরাসরি Feature Space-এর ওপর কাজ করে, এবং যখন Feature-এর সংখ্যা (Dimension) অনেক বেশি হয়, তখন দূরত্ব-ভিত্তিক গণনা (Euclidean Distance) তার কার্যকারিতা হারাতে শুরু করে — একে বলা হয় Curse of Dimensionality। এই Part-এ আলোচিত K-Means, Agglomerative এবং DBSCAN তিনটি অ্যালগরিদমই এই সমস্যায় ভোগে। পরবর্তী Part-এ আলোচিত Principal Component Analysis (PCA) এবং t-SNE-এর মতো Dimensionality Reduction টেকনিকগুলো উচ্চ-মাত্রিক ডেটাকে নিম্ন-মাত্রিক স্পেসে রূপান্তরিত করে, যা Clustering অ্যালগরিদমগুলোর কার্যকারিতা ও Visualization-কে আরও উন্নত করতে সাহায্য করে। এই কারণেই বাস্তব প্রজেক্টে প্রায়শই প্রথমে Dimensionality Reduction এবং তারপর Clustering প্রয়োগ করা হয়।

Chapter 14 — K-Means Clustering

1. Introduction

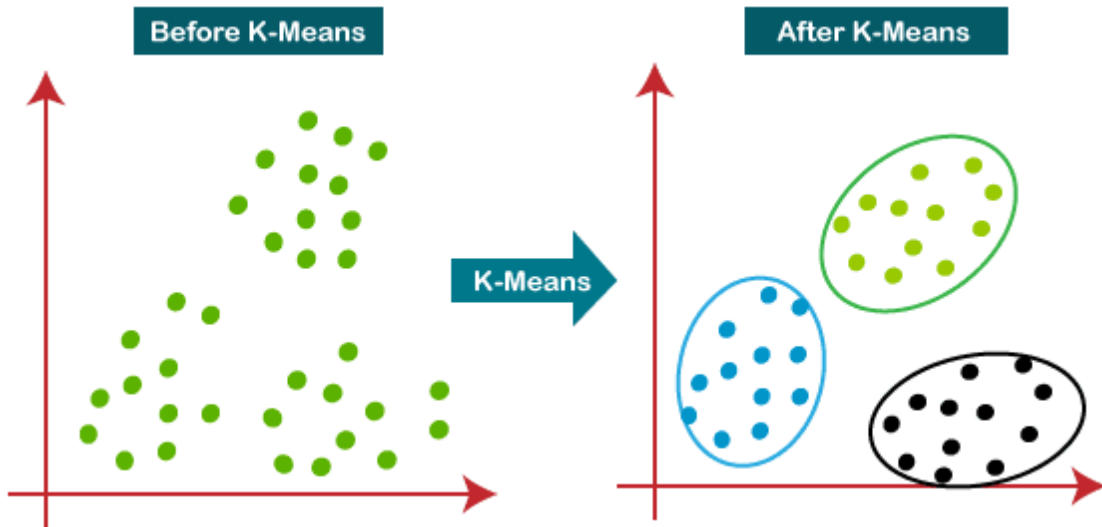
K-Means Clustering

K-Means হলো একটি অত্যন্ত জনপ্রিয়, নন-প্যারামেট্রিক (Non-parametric) এবং সেন্ট্রয়েড-ভিত্তিক (Centroid-based) হার্ড ক্লাস্টারিং (Hard Clustering) অ্যালগরিদম।

সহজ ভাষায়, এটি এমন একটি গাণিতিক পদ্ধতি যা কোনো পূর্বনির্ধারিত লেবেলবিহীন (Unlabeled) ডেটাসেটের ডেটা পয়েন্টগুলোর পারস্পরিক জ্যামিতিক দূরত্ব পরিমাপ করে তাদের স্বয়ংক্রিয়ভাবে K সংখ্যক পৃথক ক্লাস্টার বা গ্রুপে বিভক্ত করে।

এই অ্যালগরিদমে দুটি গুরুত্বপূর্ণ উপাদান রয়েছে।

- **K (Number of Clusters):** এটি একটি User-defined Hyperparameter, যা নির্ধারণ করে ডেটাসেটকে মোট কতটি ক্লাস্টারে ভাগ করা হবে।



• **Mean (Centroid):** প্রতিটি ক্লাস্টারের অন্তর্ভুক্ত সকল ডেটা পয়েন্টের গাণিতিক গড় (Arithmetic Mean), যাকে Centroid বলা হয়। এই Centroid-ই ক্লাস্টারের কেন্দ্রবিন্দু হিসেবে কাজ করে।

Classification of K-Means

মেশিন লার্নিংয়ের গাঠনিক বিন্যাস অনুযায়ী K-Means-কে নিম্নোক্ত শ্রেণিতে বিভক্ত করা যায়।

- এটি একটি **Unsupervised Learning Model**, কারণ এটি কোনো Target Label বা শিক্ষক ছাড়াই ডেটার অন্তর্নিহিত গঠন (Hidden Structure) আবিষ্কার করে।

- এটি একটি **Instance-based Model**, কারণ এটি কোনো গ্লোবাল গাণিতিক সমীকরণ শেখে না; বরং ডেটা পয়েন্টগুলোর অবস্থান ও দূরত্বের ভিত্তিতে ক্লাস্টার তৈরি করে।
- এটি একটি **Hard Clustering Model**, কারণ প্রতিটি ডেটা পয়েন্ট শুধুমাত্র একটি ক্লাস্টারের সদস্য হতে পারে। একই ডেটা পয়েন্ট একাধিক ক্লাস্টারের অন্তর্ভুক্ত হতে পারে না। এই বৈশিষ্ট্য K-Means-কে Soft Clustering অ্যালগরিদম (যেমন Gaussian Mixture Model) থেকে আলাদা করে।

2. Why This Algorithm Was Developed

Problem Solved by K-Means

K-Means মূলত ক্লাস্টারিং (Clustering) সমস্যার সমাধান করে।

জ্যামিতিক দৃষ্টিকোণ থেকে, এটি Feature Space-কে একাধিক অঞ্চলে (Regions) ভাগ করে, যাতে একই ক্লাস্টারের ডেটা পয়েন্টগুলো একে অপরের যতটা সম্ভব কাছাকাছি থাকে এবং ভিন্ন ক্লাস্টারের ডেটা পয়েন্টগুলোর মধ্যকার দূরত্ব যতটা সম্ভব বেশি হয়।

বাস্তব জীবনে K-Means নিম্নলিখিত সমস্যাগুলোর সমাধান করতে পারে।

- গ্রাহকের কেনাকাটার ধরণ, ব্যয়ের পরিমাণ এবং আচরণের ভিত্তিতে স্বয়ংক্রিয়ভাবে বিভিন্ন Customer Segment তৈরি করা।
- ডিজিটাল ছবির সমজাতীয় Pixel বা Color Value-গুলোকে একত্রিত করে Image Segmentation সম্পন্ন করা।

Importance of K-Means Clustering

বর্তমানে Linear Regression, Logistic Regression কিংবা Decision Tree-এর মতো শক্তিশালী Supervised Learning Model থাকা সত্ত্বেও K-Means-এর গুরুত্ব অত্যন্ত বেশি।

এর প্রধান কারণগুলো হলো:

- **লেবেলবিহীন ডেটা বিশ্লেষণ (Handling Unlabeled Data):** বাস্তব বিশ্বের অধিকাংশ ডেটার কোনো Label থাকে না। প্রতিটি ডেটাকে ম্যানুয়ালি Label করা সময়সাপেক্ষ এবং ব্যয়বহুল। K-Means কোনো Label ছাড়াই অর্থপূর্ণ গ্রুপ তৈরি করতে পারে।
- **উচ্চ কম্পিউটেশনাল দক্ষতা (High Computational Efficiency):** Hierarchical Clustering বা HDBSCAN-এর মতো অনেক ক্লাস্টারিং অ্যালগরিদমের তুলনায় K-Means অনেক দ্রুত কাজ করে এবং বৃহৎ ডেটাসেটও অল্প সময়ে প্রক্রিয়াকরণ করতে পারে।
- **সহজ ব্যাখ্যাযোগ্যতা (Interpretability):** প্রতিটি ক্লাস্টারের একটি নির্দিষ্ট Centroid থাকায় সহজেই বোঝানো যায় কেন নির্দিষ্ট ডেটা পয়েন্টগুলো একই ক্লাস্টারে রয়েছে। ফলে Business Analysis এবং Decision Making সহজ হয়।

- **Feature Engineering ও Data Preprocessing:** অনেক ক্ষেত্রে Supervised Learning Model ট্রেনিংয়ের আগে K-Means ব্যবহার করে Cluster Distance-এর মতো নতুন Feature তৈরি করা হয়, যা পরবর্তীতে মডেলের কার্যক্ষমতা এবং Accuracy বৃদ্ধি করতে সাহায্য করে।

3. Real-life Motivation

কল্পনা করা যাক, একটি ই-কমার্স কোম্পানির কাছে লক্ষ লক্ষ গ্রাহকের কেনাকাটার ডেটা আছে, কিন্তু কোনো গ্রাহককে আগে থেকে “High Spender” বা “Budget Buyer” বলে Label করা নেই। মানুষের পক্ষে হাতে করে প্রতিটি গ্রাহককে বিশ্লেষণ করে শ্রেণিবদ্ধ করা অসম্ভব। ঠিক এই বাস্তব সমস্যা থেকেই K-Means-এর প্রয়োজনীয়তা তৈরি হয় — এটি স্বয়ংক্রিয়ভাবে গ্রাহকদের কেনাকাটার প্যাটার্নের ভিত্তিতে স্বাভাবিক গ্রুপে ভাগ করে দেয়, যাতে কোম্পানি প্রতিটি গ্রুপের জন্য আলাদা মার্কেটিং কৌশল নিতে পারে। একইভাবে, একটি হাসপাতাল রোগীদের উপসর্গের ধরন অনুযায়ী দ্রুত প্রাথমিক শ্রেণিবিভাগ করতে, বা একটি Logistics কোম্পানি ডেলিভারি পয়েন্টগুলোকে ভৌগোলিকভাবে কাছাকাছি অঞ্চলে ভাগ করে রুট অপটিমাইজ করতে K-Means ব্যবহার করতে পারে।

4. Intuition

K-Means-এর মূল স্বত্তা (Intuition) বোঝার জন্য একটি সহজ উদাহরণ ভাবা যাক — ধরা যাক একটি ফলের ঝুড়িতে বিভিন্ন আকারের আম ও লেবু মিশানো আছে, এবং কাউকে বলা হলো চোখ বন্ধ করে শুধু আকার ও ওজন অনুভব করে ফলগুলোকে দুটি স্তুপে ভাগ করতে। স্বাভাবিকভাবেই সে কাছাকাছি আকারের ফলগুলোকে একই স্তুপে রাখবে এবং প্রতিটি স্তুপের একটি “কেন্দ্রীয় অনুভূতি” (গড় আকার) তৈরি হবে। K-Means অ্যালগরিদমও ঠিক এভাবেই কাজ করে — এটি প্রথমে আন্দাজে কিছু কেন্দ্রবিন্দু (Centroid) বসায়, তারপর প্রতিটি ডেটা পয়েন্টকে সবচেয়ে কাছের কেন্দ্রবিন্দুর সাথে যুক্ত করে, এবং প্রতিটি গ্রুপের নতুন “কেন্দ্রীয় অনুভূতি” (গড়) হিসাব করে কেন্দ্রবিন্দু সরিয়ে নেয়। এই প্রক্রিয়াটি বারবার পুনরাবৃত্তি হতে থাকে যতক্ষণ না কেন্দ্রবিন্দুগুলো আর নড়াচড়া করে — অর্থাৎ, প্রতিটি ফল তার সবচেয়ে স্বাভাবিক দলে পৌঁছে যায়।

5. Working Principle

ব্যাকগ্রাউন্ড ওয়ার্কফ্লো এবং অ্যালগরিদম

K-Means অ্যালগরিদমটি মূলত একটি ইটারেটিভ (Iterative) পদ্ধতি অনুসরণ করে গ্লোবাল মিনিমাম বা সর্বনিম্ন কস্টের দিকে এগিয়ে যায়। এর মূল ধাপগুলো নিচে দেওয়া হলো:

Input:

- Dataset X
- Number of Clusters K

Output:

- Final Clusters $(S_1, S_2, \dots, S_K)$

- Final Centroids $(\mu_1, \mu_2, \dots, \mu_K)$

Step 1: Centroid Initialization

ডেটাসেট থেকে র‍্যান্ডমভাবে (Randomly) K সংখ্যক ডেটা পয়েন্ট নির্বাচন করে সেগুলোকে প্রাথমিক সেন্ট্রয়েড হিসেবে নির্ধারণ করা হয়।

$$\mu_1, \mu_2, \dots, \mu_K$$

Step 2: Iterative Optimization

নিচের ধাপগুলো বারবার পুনরাবৃত্তি করা হয়, যতক্ষণ না অ্যালগরিদমটি স্থির (Converge) হয়।

A. Cluster Assignment (Expectation Step)

প্রতিটি ডেটা পয়েন্ট x_i -এর জন্য সকল Centroid-এর সাথে Euclidean Distance গণনা করা হয়।

এরপর x_i -কে যে Centroid-এর দূরত্ব সবচেয়ে কম, সেই Cluster-এ অন্তর্ভুক্ত করা হয়।

$$S_j = \{x_i : \|x_i - \mu_j\|^2 \leq \|x_i - \mu_l\|^2, \forall l\}$$

B. Centroid Update (Maximization Step)

প্রতিটি Cluster-এর অন্তর্ভুক্ত সকল ডেটা পয়েন্টের গাণিতিক গড় (Arithmetic Mean) নির্ণয় করে নতুন Centroid গণনা করা হয়।

$$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

C. Convergence Check

নতুন Centroid এবং পূর্ববর্তী Centroid-এর মান তুলনা করা হয়।

যদি সকল Centroid-এর অবস্থান অপরিবর্তিত থাকে (অর্থাৎ নতুন ও পুরোনো Centroid একই হয়), তাহলে Iteration বন্ধ করা হয়। অন্যথায়, পুনরায় Cluster Assignment ধাপ থেকে একই প্রক্রিয়া শুরু হয়।

6. Mathematical Backend

Mathematical Foundation of K-Means Clustering

Feature Representation

K-Means অ্যালগরিদমে প্রতিটি ডেটা একটি Feature Vector হিসেবে প্রকাশ করা হয়। যদি কোনো ডেটাসেটে D টি Feature থাকে, তাহলে একটি ডেটা পয়েন্টকে লেখা যায়,

$$x_i = [x_{i1}, x_{i2}, x_{i3}, \dots, x_{iD}]$$

যেখানে,

- $x_i = i$ -তম Data Point
- D = মোট Feature বা Dimension
- N = মোট Data Point-এর সংখ্যা

অর্থাৎ সম্পূর্ণ Dataset-কে প্রকাশ করা যায়,

$$X = \{x_1, x_2, \dots, x_N\}$$

Number of Clusters

K-Means-এ শুরুতেই একটি Hyperparameter নির্বাচন করতে হয়, যাকে K বলা হয়।

$$K = \text{Number of Clusters}$$

এটি নির্ধারণ করে Dataset-কে মোট কতটি Cluster-এ ভাগ করা হবে।

Centroid of a Cluster

প্রতিটি Cluster-এর একটি কেন্দ্রবিন্দু থাকে, যাকে **Centroid** বলা হয়।

Centroid হলো সেই Cluster-এর অন্তর্ভুক্ত সব Data Point-এর গাণিতিক গড় (Arithmetic Mean)।

$$c_j = \frac{1}{|A_j|} \sum_{x_i \in A_j} x_i$$

যেখানে,

- $c_j = j$ -তম Cluster-এর Centroid
- $A_j = j$ -তম Cluster
- $|A_j| =$ ঐ Cluster-এর মোট Data Point সংখ্যা

সহজভাবে,

$$\text{Centroid} = \frac{\text{Cluster-এর সব Data Point-এর যোগফল}}{\text{Cluster-এর মোট Data Point সংখ্যা}}$$

এই সমীকরণ প্রতিটি Iteration শেষে নতুন Centroid নির্ণয়ের জন্য ব্যবহৃত হয়।

Euclidean Distance

K-Means সাধারণত দুটি Data Point-এর মধ্যকার দূরত্ব নির্ণয়ের জন্য **Euclidean Distance** ব্যবহার করে।

$$d(x_i, c_j) = \sqrt{\sum_{k=1}^D (x_{ik} - c_{jk})^2}$$

যেখানে,

- x_i = একটি Data Point
- c_j = একটি Centroid
- D = মোট Feature

দ্বিমাত্রিক (2D) ক্ষেত্রে সমীকরণটি হয়,

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

এই Distance ব্যবহার করে প্রতিটি Data Point-কে সবচেয়ে নিকটবর্তী Centroid-এর Cluster-এ Assign করা হয়।

7. Formula

Objective Function (Within-Cluster Sum of Squares)

K-Means-এর মূল লক্ষ্য হলো প্রতিটি Cluster-এর ভেতরে থাকা Data Point-গুলোর Centroid থেকে মোট Squared Distance যতটা সম্ভব কমানো।

এই Objective Function-কে Within-Cluster Sum of Squares (WCSS) অথবা Sum of Squared Errors (SSE) বলা হয়।

$$J = \sum_{j=1}^K \sum_{x_i \in A_j} \|x_i - c_j\|^2$$

এখানে,

- J = Total Cost Function
- K = মোট Cluster সংখ্যা
- A_j = j -তম Cluster
- c_j = j -তম Cluster-এর Centroid

সহজভাবে,

$$\text{Total Cost} = \sum (\text{Point থেকে নিজের Centroid পর্যন্ত দূরত্ব})^2$$

K-Means Algorithm-এর প্রধান উদ্দেশ্য হলো,

min/

অর্থাৎ Total Cost যতটা সম্ভব কমিয়ে আনা।

8. Formula Derivation

Why Does the Cost Always Decrease?

K-Means অ্যালগরিদমের Cost Function প্রতিটি Iteration-এ কখনো বৃদ্ধি পায় না; বরং এটি হয় কমে অথবা অপরিবর্তিত থাকে। এর কারণ হলো অ্যালগরিদমটি দুটি ধারাবাহিক অপটিমাইজেশন ধাপ অনুসরণ করে।

প্রথম ধাপে (Assignment Step), প্রতিটি Data Point-এর সঙ্গে সবগুলো Centroid-এর দূরত্ব গণনা করা হয় এবং পয়েন্টটিকে সবচেয়ে নিকটবর্তী Centroid-এর Cluster-এ স্থানান্তর করা হয়। যেহেতু প্রতিটি Data Point সর্বদা সর্বনিম্ন দূরত্বের Cluster-এ Assign করা হয়, তাই এই ধাপে মোট Squared Distance বা Cost কখনো বৃদ্ধি পায় না।

দ্বিতীয় ধাপে (Update Step), প্রতিটি Cluster-এর অন্তর্ভুক্ত সকল Data Point-এর গাণিতিক গড় (Arithmetic Mean) নির্ণয় করে নতুন Centroid গণনা করা হয়। গাণিতিকভাবে প্রমাণিত যে কোনো Cluster-এর Mean সেই অবস্থান, যেখানে ঐ Cluster-এর সকল Data Point থেকে Squared Distance-এর যোগফল সর্বনিম্ন হয়। ফলে নতুন Centroid নির্ধারণের পর মোট Cost আরও কমে যায় অথবা পূর্বের মতোই থাকে।

এই দুটি ধাপ পর্যায়ক্রমে পুনরাবৃত্তি হতে থাকে। প্রতিটি Iteration-এ Cost Function হয় কমে, নতুবা অপরিবর্তিত থাকে। যেহেতু Cost Function-এর একটি নিম্নসীমা (Lower Bound) রয়েছে এবং এটি কখনো ঋণাত্মক হতে পারে না, তাই একসময় অ্যালগরিদমটি এমন একটি অবস্থায় পৌঁছে যায় যেখানে আর কোনো পরিবর্তন ঘটে না। এই অবস্থাকেই **Convergence** বলা হয়।

Update Step

প্রতিটি Cluster-এর নতুন Mean নির্ণয় করে নতুন Centroid গণনা করা হয়।

এর ফলে নতুন Centroid Cluster-এর প্রকৃত কেন্দ্রের আরও কাছাকাছি চলে আসে এবং মোট Squared Distance আরও কমে যায়।

এই দুটি ধাপ পুনরাবৃত্তি হতে থাকে যতক্ষণ না আর কোনো পরিবর্তন ঘটে।

ফলে প্রতিটি Iteration-এ,

$$J_{\text{new}} \leq J_{\text{old}}$$

অর্থাৎ Cost Function হয় কমে, নতুবা অপরিবর্তিত থাকে। একসময় Algorithm একটি স্থিতিশীল অবস্থায় (Convergence) পৌঁছে যায়।

9. Meaning of Every Symbol

পূর্ববর্তী সেকশনগুলোতে ব্যবহৃত প্রতিটি গাণিতিক প্রতীকের অর্থ একত্রে নিচে সংক্ষিপ্তভাবে দেওয়া হলো, যাতে সূত্রগুলো পড়ার সময় দ্রুত রেফারেন্স হিসেবে ব্যবহার করা যায়:

প্রতীক	অর্থ
N	ডেটাসেটের মোট Data Point-এর সংখ্যা
D	প্রতিটি Data Point-এর Feature বা Dimension-এর সংখ্যা
K	পূর্বনির্ধারিত Cluster-এর সংখ্যা
x_i	i -তম Data Point (একটি D -মাত্রিক ভেক্টর)
μ_j	j -তম Cluster-এর Centroid
S_j	j -তম Cluster-এর অন্তর্ভুক্ত সব Data Point-এর সেট
$\ x_i - \mu_j\ ^2$	x_i এবং μ_j -এর মধ্যে Squared Euclidean Distance
J (বা WCSS)	Objective Function — মোট Within-Cluster Sum of Squares
t	Iteration-এর ক্রম নির্দেশক

10. Step-by-Step Formula Explanation

Determining the Optimal Number of Clusters

K-Means Algorithm চালানোর আগে উপযুক্ত K নির্বাচন করা অত্যন্ত গুরুত্বপূর্ণ।

বাস্তবে Optimal K নির্ধারণের জন্য বিভিন্ন পদ্ধতি ব্যবহৃত হয়, যেমন,

- Elbow Method
- Silhouette Score
- Gap Statistic
- Prediction Strength

এর মধ্যে **Elbow Method** এবং **Silhouette Score** সবচেয়ে বেশি ব্যবহৃত ও পাঠ্যবইয়ে আলোচিত পদ্ধতি।

K-Means অ্যালগরিদমের মূল উদ্দেশ্য হলো প্রতিটি ডেটা পয়েন্টকে তার নিকটবর্তী সেন্ট্রয়েডের (Centroid/Cluster Mean) সাথে যুক্ত করে ক্লাস্টারের ভেতরের বিশৃঙ্খলা বা ভুলের পরিমাণ সর্বনিম্ন করা। গাণিতিক ভাষায় একে Within-Cluster Sum of Squares (WCSS) বা Inertia বলা হয়।

ধরা যাক, আমাদের কাছে n সংখ্যক স্যাম্পল সংবলিত একটি ডেটাসেট $D = \{x_1, x_2, \dots, x_n\}$ আছে। আমরা এই ডেটাকে K সংখ্যক ক্লাস্টারে $S = \{S_1, S_2, \dots, S_K\}$ ভাগ করতে চাই। এর কস্ট ফাংশন (J) সমীকরণটি হলো:

$$J = \sum_{j=1}^K \sum_{x_i \in S_j} \|x_i - \mu_j\|^2$$

যেখানে:

- K : মোট ক্লাস্টারের সংখ্যা (User-defined Hyperparameter)।
- S_j : j -তম ক্লাস্টারের অন্তর্ভুক্ত ডেটা পয়েন্টগুলোর সেট।
- μ_j : j -তম ক্লাস্টারের সেন্ট্রয়েড বা গড় মান (Mean Vector)।
- $\|x_i - \mu_j\|^2$: ডেটা পয়েন্ট x_i এবং সেন্ট্রয়েড μ_j -এর মধ্যকার ইউক্লিডিয়ান দূরত্বের বর্গ (Squared Euclidean Distance)।

11. Numerical Example

ধাপ ১: ডেটাসেট ও প্যারামিটার নির্ধারণ

ধরা যাক, আমাদের কাছে ২-ডাইমেনশনাল ($d = 2$) একটি ছোট ডেটাসেট আছে যেখানে $n = 4$ টি ডেটা পয়েন্ট রয়েছে:

- $A_1 = (1,1)$
- $A_2 = (2,1)$
- $A_3 = (4,3)$
- $A_4 = (5,4)$

আমরা চাই ডেটাগুলোকে $K = 2$ টি ক্লাস্টারে ভাগ করতে।

ধাপ ২: র্যান্ডম সেন্ট্রয়েড নির্বাচন (Initialization)

আমরা আন্দাজে ডেটাসেট থেকে প্রথম দুটি পয়েন্টকে প্রাথমিক সেন্ট্রয়েড হিসেবে ধরে নিলাম:

- সেন্ট্রয়েড ১ (μ_1) = (1,1)
- সেন্ট্রয়েড ২ (μ_2) = (2,1)

ইটারেশন ১ (Iteration 1)

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

আমরা প্রতিটি ডেটা পয়েন্ট $x_i = (x_{i1}, x_{i2})$ থেকে সেন্ট্রয়েড $\mu_j = (\mu_{j1}, \mu_{j2})$ এর ইউক্লিডিয়ান দূরত্বের বর্গ হিসাব করব। সূত্র: $\|x_i - \mu_j\|^2 = (x_{i1} - \mu_{j1})^2 + (x_{i2} - \mu_{j2})^2$

- পয়েন্ট $A_1(1,1)$ এর জন্য:

- μ_1 থেকে দূরত্ব $= (1 - 1)^2 + (1 - 1)^2 = 0$
- μ_2 থেকে দূরত্ব $= (1 - 2)^2 + (1 - 1)^2 = 1 + 0 = 1$
- সিদ্ধান্ত: যেহেতু $0 < 1$, তাই $A_1 \rightarrow \text{Cluster 1}$
- পয়েন্ট $A_2(2,1)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= (2 - 1)^2 + (1 - 1)^2 = 1 + 0 = 1$
 - μ_2 থেকে দূরত্ব $= (2 - 2)^2 + (1 - 1)^2 = 0$
 - সিদ্ধান্ত: যেহেতু $0 < 1$, তাই $A_2 \rightarrow \text{Cluster 2}$
- পয়েন্ট $A_3(4,3)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= (4 - 1)^2 + (3 - 1)^2 = 9 + 4 = 13$
 - μ_2 থেকে দূরত্ব $= (4 - 2)^2 + (3 - 1)^2 = 4 + 4 = 8$
 - সিদ্ধান্ত: যেহেতু $8 < 13$, তাই $A_3 \rightarrow \text{Cluster 2}$
- পয়েন্ট $A_4(5,4)$ এর জন্য:
 - μ_1 থেকে দূরত্ব $= (5 - 1)^2 + (4 - 1)^2 = 16 + 9 = 25$
 - μ_2 থেকে দূরত্ব $= (5 - 2)^2 + (4 - 1)^2 = 9 + 9 = 18$
 - সিদ্ধান্ত: যেহেতু $18 < 25$, তাই $A_4 \rightarrow \text{Cluster 2}$

ইটারেশন ১ শেষে নতুন ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1\}$
- $S_2 = \{A_2, A_3, A_4\}$

অংশ B: বর্তমান কস্ট বা WCSS হিসাব

$$J = \sum \|x_i - \mu_1\|^2 + \sum \|x_i - \mu_2\|^2$$

$$J = [0] + [0 + 8 + 18] = 26$$

অংশ C: নতুন সেন্ট্রয়েড গণনা (Centroid Update)

- নতুন সেন্ট্রয়েড μ_1 :

$$\mu_1 = \left(\frac{1}{1}, \frac{1}{1}\right) = (1,1)$$

- নতুন সেন্ট্রয়েড μ_2 :

$$\mu_2 = \left(\frac{2+4+5}{3}, \frac{1+3+4}{3} \right) = \left(\frac{11}{3}, \frac{8}{3} \right) \approx (3.67, 2.67)$$

ইটারেশন ২ (Iteration 2)

যেহেতু সেন্ট্রয়েডের মান পরিবর্তিত হয়েছে, তাই আমরা নতুন সেন্ট্রয়েড $\mu_1 = (1,1)$ এবং $\mu_2 = (3.67, 2.67)$ ব্যবহার করে আবার দূরত্ব হিসাব করব।

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

- পয়েন্ট $A_1(1,1)$ এর জন্য:

- μ_1 থেকে দূরত্ব = 0
- μ_2 থেকে দূরত্ব = $(1 - 3.67)^2 + (1 - 2.67)^2 = (-2.67)^2 + (-1.67)^2 \approx 7.13 + 2.79 = 9.92$
- সিদ্ধান্ত: $A_1 \rightarrow \text{Cluster 1}$

- পয়েন্ট $A_2(2,1)$ এর জন্য:

- μ_1 থেকে দূরত্ব = $(2 - 1)^2 + (1 - 1)^2 = 1$
- μ_2 থেকে দূরত্ব = $(2 - 3.67)^2 + (1 - 2.67)^2 = (-1.67)^2 + (-1.67)^2 \approx 2.79 + 2.79 = 5.58$
- সিদ্ধান্ত: যেহেতু $1 < 5.58$, তাই A_2 ক্লাস্টার পরিবর্তন করে ১ নম্বর ক্লাস্টারে চলে গেল। ($A_2 \rightarrow \text{Cluster 1}$)

- পয়েন্ট $A_3(4,3)$ এর জন্য:

- μ_1 থেকে দূরত্ব = $(4 - 1)^2 + (3 - 1)^2 = 13$
- μ_2 থেকে দূরত্ব = $(4 - 3.67)^2 + (3 - 2.67)^2 = (0.33)^2 + (0.33)^2 \approx 0.11 + 0.11 = 0.22$
- সিদ্ধান্ত: $A_3 \rightarrow \text{Cluster 2}$

- পয়েন্ট $A_4(5,4)$ এর জন্য:

- μ_1 থেকে দূরত্ব = 25
- μ_2 থেকে দূরত্ব = $(5 - 3.67)^2 + (4 - 2.67)^2 = (1.33)^2 + (1.33)^2 \approx 1.77 + 1.77 = 3.54$
- সিদ্ধান্ত: $A_4 \rightarrow \text{Cluster 2}$

ইটারেশন ২ শেষে নতুন ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1, A_2\}$
- $S_2 = \{A_3, A_4\}$

অংশ B: বর্তমান কস্ট বা WCSS হিসাব

$$J = [0 + 1] + [0.22 + 3.54] = 4.76$$

(দেখা যাচ্ছে কস্ট ২৬ থেকে কমে ৪.৭৬ এ নেমে এসেছে, অর্থাৎ মডেলটি সঠিক দিকে যাচ্ছে)

অংশ C: নতুন সেন্ট্রয়েড গণনা (Centroid Update)

- নতুন সেন্ট্রয়েড μ_1 :

$$\mu_1 = \left(\frac{1+2}{2}, \frac{1+1}{2} \right) = (1.5, 1)$$

- নতুন সেন্ট্রয়েড μ_2 :

$$\mu_2 = \left(\frac{4+5}{2}, \frac{3+4}{2} \right) = (4.5, 3.5)$$

ইটারেশন ৩ (Iteration 3)

এখন নতুন আপডেটেড সেন্ট্রয়েড হলো $\mu_1 = (1.5, 1.0)$ এবং $\mu_2 = (4.5, 3.5)$ ।

অংশ A: দূরত্ব গণনা ও ক্লাস্টার অ্যাসাইনমেন্ট

- পয়েন্ট $A_1(1,1)$: μ_1 থেকে দূরত্ব = 0.25 | μ_2 থেকে দূরত্ব = 18.5 $\rightarrow A_1 \rightarrow S_1$
- পয়েন্ট $A_2(2,1)$: μ_1 থেকে দূরত্ব = 0.25 | μ_2 থেকে দূরত্ব = 12.5 $\rightarrow A_2 \rightarrow S_1$
- পয়েন্ট $A_3(4,3)$: μ_1 থেকে দূরত্ব = 10.25 | μ_2 থেকে দূরত্ব = 0.5 $\rightarrow A_3 \rightarrow S_2$
- পয়েন্ট $A_4(5,4)$: μ_1 থেকে দূরত্ব = 21.25 | μ_2 থেকে দূরত্ব = 0.5 $\rightarrow A_4 \rightarrow S_2$

ইটারেশন ৩ শেষে ক্লাস্টার গ্রুপ:

- $S_1 = \{A_1, A_2\}$
- $S_2 = \{A_3, A_4\}$

Convergence Reached (স্থিরতা অর্জন): খেয়াল করে দেখুন, ইটারেশন ৩-এর ক্লাস্টার অ্যাসাইনমেন্ট এবং ইটারেশন ২-এর ক্লাস্টার অ্যাসাইনমেন্ট হুবহু একই এসেছে। এর মানে হলো ডেটা পয়েন্টগুলো আর তাদের দল পরিবর্তন করছে না। সেন্ট্রয়েড পুনর্গণনা করলেও মান একই থাকবে। অতএব, মডেলটি গ্লোবাল মিনিমাম বা সর্বনিম্ন কস্ট পয়েন্ট অর্জন করেছে এবং ট্রেনিং এখানেই সমাপ্ত।

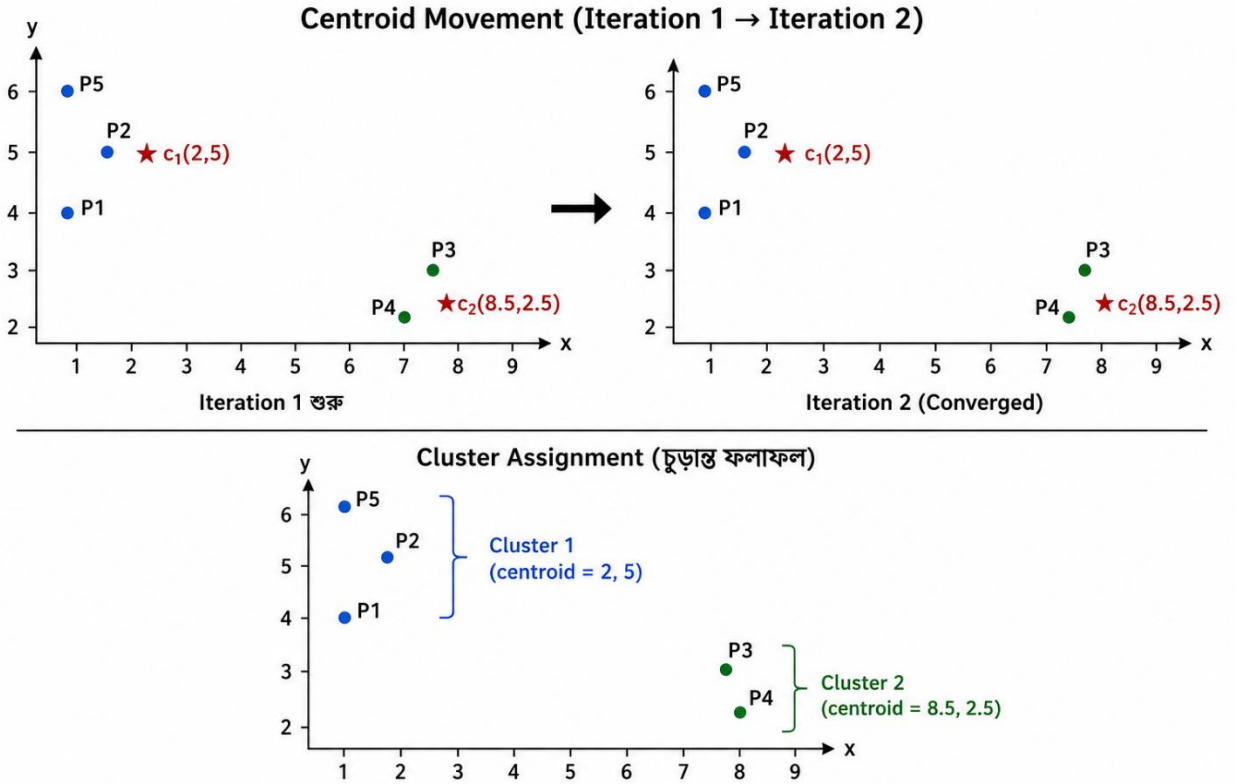
চূড়ান্ত ফলাফল টেবিল (Final Model State)

ক্লাস্টার আইডি (Cluster)	অন্তর্ভুক্ত ডেটা পয়েন্টসমূহ	চূড়ান্ত সেন্ট্রয়েড (Centroid)	ক্লাস্টার কস্ট (WCSS)
Cluster 1	$A_1(1,1), A_2(2,1)$	$\mu_1 = (1.5, 1.0)$	$0.25 + 0.25 = 0.50$
Cluster 2	$A_3(4,3), A_4(5,4)$	$\mu_2 = (4.5, 3.5)$	$0.50 + 0.50 = 1.00$

সর্বনিম্ন চূড়ান্ত কস্ট (Minimum Cost) $J = 0.50 + 1.00 = 1.50$

12. Visualization

Visualization — ASCII Diagram



13. Hyperparameters

Parameters:

১. মোস্ট ইম্পোর্টেন্ট প্যারামিটারসমূহ (The Heavy Hitters)

মডেলের ক্লাস্টারিং পারফরম্যান্স, গতি এবং সেন্ট্রয়েডের সঠিক অবস্থান নির্ধারণে এই প্যারামিটারগুলো সবচেয়ে গুরুত্বপূর্ণ ভূমিকা পালন করে।

`n_clusters`

- **Default:** 8
- **প্যারামিটারটি কী কাজ করে:** এটি মূলত ডেটাসেটকে মোট কতটি ক্লাস্টারে ভাগ করা হবে এবং কতটি সেন্ট্রয়েডের স্থানাঙ্ক (Coordinates) তৈরি করা হবে তা নির্ধারণ করে।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো ধনাত্মক পূর্ণসংখ্যা (Integer) গ্রহণ করে। বাস্তব প্রজেক্টে ব্যবসার লক্ষ্য বা ডেটার ধরনের ওপর ভিত্তি করে এটি সেট করা হয়। যদি সঠিক ক্লাস্টার সংখ্যা জানা না থাকে, তবে **Elbow Method** বা **Silhouette Analysis**-এর মতো গাণিতিক পদ্ধতি ব্যবহার করে সবচেয়ে নিখুঁত K বা ক্লাস্টার সংখ্যা খুঁজে বের করে এখানে ইনপুট দিতে হয়।

`init`

- **Default:** 'k-means++'
- **প্যারামিটারটি কী কাজ করে:** অ্যালগরিদমটি শুরু করার সময় প্রাথমিক সেন্ট্রয়েডের অবস্থান কীভাবে নির্বাচন করা হবে, এটি তা নিয়ন্ত্রণ করে। ডিফল্ট 'k-means++' পদ্ধতিটি ডেটা পয়েন্টগুলোর সামগ্রিক জড়তার (Inertia) ওপর ভিত্তি করে একটি প্রায়োগিক সম্ভাব্যতা বন্টন (Empirical probability distribution) ব্যবহার করে সেন্ট্রয়েডের বীজ (Seeds) বপন করে। এটি সাধারণ পদ্ধতির চেয়ে অনেক দ্রুত মডেলকে কনভার্জ (স্থির) করতে সাহায্য করে। Scikit-Learn-এর এই ইমপ্লিমেন্টেশনটি প্রতিটি ধাপে বেশ কয়েকটি ট্রায়াল চালিয়ে সেরা সেন্ট্রয়েডটি বেছে নেয় (Greedy K-Means++)।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:**
 - 'random': এটি ডেটাসেট থেকে সম্পূর্ণ র্যান্ডমলি (আন্দাজে) K সংখ্যক সারি বা পর্যবেক্ষণ বেছে নিয়ে প্রাথমিক সেন্ট্রয়েড নির্ধারণ করে। এটি মূলত শিক্ষানবিস বা তাত্ত্বিক পরীক্ষার ক্ষেত্রে ব্যবহার করা হয়, তবে এটি ব্যবহার করলে মডেলটি প্রায়শই লোকাল মিনিমামে (Local Minima) আটকে যায়।
 - array-like (ম্যাট্রিক্স): যদি আপনার আগে থেকেই সেন্ট্রয়েডের অবস্থান জানা থাকে, তবে তার স্থানাঙ্ক সরাসরি (`n_clusters`, `n_features`) শেপের একটি অ্যারে আকারে পাস করতে পারেন।

- callable (ফাংশন): কাস্টম কোনো শুরুকরণ কৌশল (Custom Initialization Strategy) তৈরি করতে চাইলে এখানে একটি পাইথন ফাংশন পাস করা যায়।

n_init

- **Default:** 'auto'
- **প্যারামিটারটি কী কাজ করে:** K-Means অ্যালগরিদমটি ভিন্ন ভিন্ন প্রাথমিক সেন্ট্রয়েড বীজ নিয়ে মোট কতবার নতুন করে রান (Run) করবে, এটি তা নির্ধারণ করে। একাধিকবার রান করার পর, যে রানটি সবচেয়ে সর্বনিম্ন জড়তা (Inertia/WCSS) প্রদান করে, মডেলটি সেটিকেই চূড়ান্ত ফলাফল হিসেবে গ্রহণ করে। এটি মডেলকে লোকাল মিনিমা থেকে রক্ষা করতে অত্যন্ত সাহায্য করে।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো পূর্ণসংখ্যা (যেমন: 5, 10, 20) গ্রহণ করতে পারে।
 - যখন এটি 'auto' থাকে, তখন এটি init-এর মানের ওপর ভিত্তি করে স্বয়ংক্রিয়ভাবে রানের সংখ্যা নির্ধারণ করে। যদি init='random' বা callable ফাংশন ব্যবহার করা হয়, তবে এটি **১০ বার** রান করে। আর যদি init='k-means++' বা কাস্টম অ্যারে দেওয়া হয়, তবে এটি মাত্র **১ বার** রান করে (কারণ k-means++ প্রথমবারেই চমৎকার সেন্ট্রয়েড খুঁজে পায়)।
 - উচ্চ-মাত্রার স্পার্স ডেটার (Sparse High-Dimensional Data) ক্ষেত্রে লোকাল মিনিমা এড়াতে এখানে ম্যানুয়ালি বড় সংখ্যা (যেমন: 15 বা 20) ব্যবহার করা উচিত।

২. মিডিয়াম ইম্পর্টেন্ট প্যারামিটারসমূহ (Optimization & Execution)

মডেলের কনভার্জেন্সের গতিবেগ, কম্পিউটেশনাল সময় এবং মেমোরি অপ্টিমাইজেশনের জন্য এই প্যারামিটারগুলো ব্যবহার করা হয়।

max_iter

- **Default:** 300
- **প্যারামিটারটি কী কাজ করে:** এটি একটি একক রানের (Single Run) জন্য K-Means অ্যালগরিদমটি সর্বোচ্চ কতবার ইটারেশন বা চক্র (Centroid Update -> Assign Cluster) চালাবে তা নির্ধারণ করে দেয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** যেকোনো ধনাত্মক পূর্ণসংখ্যা পাস করা যায়। ডেটাসেট যদি অনেক বিশাল হয় এবং ৩০০ ইটারেশন হওয়ার আগেই যদি মডেল স্থির হয়ে যায়, তবে কোনো সমস্যা নেই। তবে ডেটা যদি খুব জটিল হয় এবং ৩০০ ইটারেশনের মধ্যেও সেন্ট্রয়েডগুলো স্থির হতে না পারে, তবে এই মান বাড়িয়ে 500 বা 1000 করা প্রয়োজন হতে পারে (যদিও বাস্তব ক্ষেত্রে ৩০০ ইটারেশনই যথেষ্ট)।

tol

- **Default:** 1e-4 (0.0001)
- **প্যারামিটারটি কী কাজ করে:** এটি হলো কনভার্জেন্স থ্রেশহোল্ড বা সহনশীলতার মাত্রা (Convergence Tolerance)। পরপর দুটি ইটারেশনে সেন্ট্রয়েডগুলোর অবস্থানের পার্থক্যের Frobenius Norm যদি এই tol-এর চেয়ে কম হয়, তবে অ্যালগরিদম ধরে নেয় যে মডেলটি পুরোপুরি কনভার্স বা স্থির হয়ে গেছে এবং সর্বোচ্চ ইটারেশন (max_iter) পূর্ণ হওয়ার আগেই ট্রেনিং বন্ধ করে দেয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এটি যেকোনো ফ্লোট (Float) সংখ্যা গ্রহণ করে। যদি ক্লাস্টারিং অত্যন্ত নিখুঁত এবং সুনির্দিষ্ট করতে হয়, তবে এই মান কমিয়ে 1e-5 বা 1e-6 করা যায় (এতে ট্রেনিং সময় একটু বেশি লাগবে)। আবার দ্রুত খসড়া ফলাফলের জন্য এটি বাড়িয়ে 1e-3 করা যেতে পারে।

algorithm

- **Default:** 'lloyd'
- **প্যারামিটারটি কী কাজ করে:** ক্লাস্টারিং গাণিতিক সমস্যাটি সমাধান করার জন্য ব্যাকএন্ডে কোন ক্লাসিক্যাল বা পরিবর্তিত অ্যালগরিদম মেকানিজম ব্যবহার করা হবে তা এটি নিয়ন্ত্রণ করে। ডিফল্ট 'lloyd' হলো চিরাচরিত Expectation-Maximization (EM) স্টাইল অ্যালগরিদম।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:**
 - 'elkan': এটি মূলত জ্যামিতিক ত্রিভুজ অসমতা (Triangle Inequality) ব্যবহার করে কাজ করে। যেসব ডেটাসেটে ক্লাস্টারগুলো খুব সুনির্দিষ্ট এবং পরিষ্কারভাবে আলাদা করা থাকে, সেখানে দূরত্ব গণনার অপ্ৰয়োজনীয় ধাপগুলো এড়িয়ে এটি লয়েডের চেয়ে অনেক দ্রুত কাজ করতে পারে। তবে এটি মেমোরির দিক থেকে বেশ ব্যয়বহুল, কারণ ব্যাকএন্ডে এটি (n_samples, n_clusters) শেপের একটি অতিরিক্ত অ্যারে মেমোরিতে ধরে রাখে। ডেটা সাইজ মাঝারি এবং ক্লাস্টার সুনির্দিষ্ট হলে এটি ব্যবহার করা দারুণ কার্যকর।

৩. লেস ইম্পর্টেন্ট প্যারামিটারসমূহ (System & Housekeeping)

এই প্যারামিটারগুলো মডেলের ক্লাস্টারিং পারফরম্যান্স বা একুরেসিতে কোনো পরিবর্তন আনে না, বরং এগুলো কোডের পুনরুৎপাদনযোগ্যতা (Reproducibility) এবং সিস্টেম রিসোর্স নিয়ন্ত্রণে ব্যবহৃত হয়।

random_state

- **Default:** None
- **প্যারামিটারটি কী কাজ করে:** সেন্ট্রয়েড শুরুকরণের (Centroid Initialization) সময় যে সুডোরাণ্ডম নম্বর জেনারেটর কাজ করে, এটি তার বীজ (Seed) নির্ধারণ করে।

- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** এখানে যেকোনো নির্দিষ্ট পূর্ণসংখ্যা (যেমন: 0, 42) ব্যবহার করা উচিত। অ্যাসাইনমেন্ট, একাডেমিক রিসার্চ বা 실험 ট্র্যাকিংয়ের সময় কোডটি যতবারই রান করা হোক না কেন, প্রতিবার যেন ছবছ একই প্রাথমিক সেন্ট্রয়েড এবং একই ক্লাস্টার ফলাফল পাওয়া যায়, তা নিশ্চিত করতে এটি লক করা বাধ্যতামূলক।

copy_x

- **Default:** True
- **প্যারামিটারটি কী কাজ করে:** দূরত্বের সংখ্যাসূচক নির্ভুলতা (Numerical Accuracy) বৃদ্ধির জন্য K-Means ব্যাকএন্ডে ডেটাকে প্রথমে সেন্ট্রালাইজ (Center) করে নেয়। এটি True থাকলে মূল ইনপুট ডেটাসেটটি অপরিবর্তিত থাকে এবং একটি কপি নিয়ে কাজ করা হয়।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** False দিলে মেমোরি সাশ্রয় হয় কারণ এটি সরাসরি মূল মেমোরির ডেটার ওপর অপারেশন চালায় এবং কাজ শেষে ডেটার গড় পুনরায় যোগ করে দেয়। তবে এর ফলে খুব সামান্য সংখ্যাসূচক মানের পার্থক্য (Small Numerical Differences) তৈরি হতে পারে। ডেটার সাইজ যদি র‍্যামের তুলনায় অত্যন্ত বিশাল হয়, কেবল তখনই মেমোরি বাঁচাতে এটি False করা যেতে পারে।

verbose

- **Default:** 0
- **প্যারামিটারটি কী কাজ করে:** এটি ভার্সিটি মোড নিয়ন্ত্রণ করে, অর্থাৎ মডেল ট্রেনিং চলাকালে স্ক্রিনে ভেতরের বিস্তারিত লগ বা অগ্রগতি দেখাবে কি না।
- **অন্যান্য অপশনসমূহ এবং কখন ব্যবহার করবেন:** যেকোনো ধনাত্মক পূর্ণসংখ্যা দেওয়া যায়। verbose=1 বা তার বেশি দিলে প্রতি ইটারেশনে জড়তা বা কস্টের মান কত কমছে তা কনসোলে দেখা যায়, যা মূলত দীর্ঘ সময় ধরে চলা বড় মডেলগুলোর ডিবাগিংয়ে সাহায্য করে।

KMeans() Attributes

cluster_centers_

Default: Created after fit()

প্রতিটি Cluster-এর Final Centroid (Center Point)-এর Coordinates সংরক্ষণ করে। Training শেষে এই Centroid-গুলোর অবস্থানই প্রতিটি Cluster-এর কেন্দ্র নির্দেশ করে।

labels_

Default: Created after fit()

Training Data-এর প্রতিটি Sample কোন Cluster-এ Assigned হয়েছে, সেই Cluster Label সংরক্ষণ করে। Label সাধারণত 0 থেকে n_clusters - 1 পর্যন্ত হয়।

inertia_

Default: Created after fit()

Model-এর Final Within-Cluster Sum of Squares (WCSS) সংরক্ষণ করে। এটি প্রতিটি Data Point থেকে তার Assigned Centroid পর্যন্ত Squared Distance-এর যোগফল। ছোট inertia_ সাধারণত ভালো Clustering নির্দেশ করে এবং Elbow Method-এ এটি ব্যবহৃত হয়।

n_iter_

Default: Created after fit()

Model Converge হতে মোট কতটি Iteration লেগেছে তা সংরক্ষণ করে। যদি আগে Converge হয়ে যায়, তাহলে এটি max_iter-এর চেয়ে কম হতে পারে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_steps_

Default: Created after fit() (*Only when using BisectingKMeans, not standard KMeans*)

Standard KMeans-এ এই Attribute থাকে না। BisectingKMeans ব্যবহার করলে Clustering সম্পন্ন করতে মোট কতটি Splitting Step হয়েছে তা সংরক্ষণ করে।

14. Scikit-learn Import

নিচে K-Means-এর জন্য প্রয়োজনীয় মূল Import স্টেটমেন্টগুলো দেখানো হলো; সম্পূর্ণ কোড পরবর্তী সেকশনে (১৫) রয়েছে।

```
import numpy as np
from sklearn.cluster import KMeans
```

15. Python Example

From-Scratch বাস্তবায়ন (NumPy ব্যবহার করে)

নিচে K-Means অ্যালগরিদমটি কোনো Library ছাড়া, শুধুমাত্র NumPy দিয়ে ধাপে ধাপে বাস্তবায়ন করা হলো, যাতে ব্যাকএন্ডে ঠিক কী ঘটছে তা সম্পূর্ণভাবে বোঝা যায়।

```
import numpy as np

def euclidean_sq(a, b):
    """দুটি পয়েন্টের মধ্যে Squared Euclidean Distance গণনা করে"""
    return np.sum((a - b) ** 2)

def kmeans_from_scratch(X, k, initial_centroids, max_iter=100):
    centroids = np.array(initial_centroids, dtype=float)
    for iteration in range(max_iter):
        # ধাপ ১: প্রতিটি পয়েন্টকে নিকটতম Centroid-এ Assign করা
        labels = []
        for point in X:
            distances = [euclidean_sq(point, c) for c in centroids]
            labels.append(int(np.argmin(distances)))
        labels = np.array(labels)

        # ধাপ ২: নতুন Centroid গণনা করা (প্রতিটি Cluster-এর গড়)
        new_centroids = np.array([
            X[labels == j].mean(axis=0) for j in range(k)
        ])

        # ধাপ ৩: Convergence যাচাই করা
        if np.allclose(new_centroids, centroids):
            print(f"Converged at iteration {iteration + 1}")
            break
        centroids = new_centroids

        # ধাপ ৪: চূড়ান্ত SSE (Sum of Squared Errors) গণনা
        sse = sum(euclidean_sq(X[i], centroids[labels[i]]) for i in range(len(X)))

    return labels, centroids, sse

# ব্যবহার
X = np.array([[2, 4], [3, 5], [8, 3], [9, 2], [1, 6]])
initial = [[2, 4], [8, 3]] # P1 এবং P3 কে প্রাথমিক Centroid ধরা হলো
labels, centroids, sse = kmeans_from_scratch(X, k=2, initial_centroids=initial)
print("Labels:", labels)
print("Final Centroids:", centroids)
print("Final SSE:", sse)
```

Scikit-learn ব্যবহার করে বাস্তবায়ন

```
from sklearn.cluster import KMeans
import numpy as np
```

```
X = np.array([[2, 4], [3, 5], [8, 3], [9, 2], [1, 6]])
```

```
# n_init=1 এবং init নিজে নির্দিষ্ট করে দেওয়া হলো, যাতে ফলাফল  
# হাতে-করা ম্যানুয়াল গণনার সাথে ছবছ মেলে
```

```
model = KMeans(  
    n_clusters=2,  
    init=np.array([[2, 4], [8, 3]]),  
    n_init=1,  
    max_iter=300,  
    random_state=42  
)  
model.fit(X)
```

```
print("Cluster Labels:", model.labels_)  
print("Centroids:", model.cluster_centers_)  
print("SSE (Inertia):", model.inertia_)
```

```
# নতুন একটি Sample-এর Cluster ভবিষ্যদ্বাণী করা
```

```
new_sample = np.array([[7, 3]])  
print("Predicted Cluster:", model.predict(new_sample))
```

16. Prediction / Clustering Process

KMeans() Methods

fit(X)

Training Data ব্যবহার করে K-Means Model Train করে। এটি Initial Centroid নির্বাচন করে, Iteratively Cluster Assign করে এবং Final Centroid নির্ধারণ করে।

predict(X)

Train করা Model ব্যবহার করে নতুন Data কোন Cluster-এর সবচেয়ে কাছাকাছি তা নির্ধারণ করে সেই Cluster Label Return করে।

fit_predict(X)

এক ধাপে Model Train করে এবং প্রতিটি Sample-এর Cluster Label Return করে। এটি fit() এবং predict() একসাথে করার সমতুল্য।

transform(X)

প্রতিটি Data Point থেকে প্রতিটি Cluster Centroid পর্যন্ত Distance গণনা করে Return করে। এটি Cluster Label নয়, Distance Matrix প্রদান করে।

fit_transform(X)

প্রথমে Model Train করে, তারপর প্রতিটি Sample থেকে প্রতিটি Cluster Centroid পর্যন্ত Distance Matrix Return করে। Return হওয়া Matrix-এর Shape (n_samples, n_clusters) হয়।

score(X)

Model-এর Negative Inertia (Negative WCSS) Return করে। Scikit-learn-এবড় Score ভালো ধরা হয়, তাই এটি -inertia_ Return করে।

get_params(deep=True)

Model-এর বর্তমান সব Parameter Dictionary আকারে Return করে। Model Inspect করা বা GridSearchCV-তে ব্যবহার করার জন্য এটি উপকারী।

set_params(params)

এক বা একাধিক Parameter-এর মান পরিবর্তন করে। Parameter পরিবর্তনের পরে Model-কে পুনরায় fit() করতে হয়।

get_metadata_routing()

নতুন একটি Sample-এর Cluster ভবিষ্যদ্বাণী কীভাবে ম্যানুয়ালি গণনা করা হয় তার সম্পূর্ণ ধাপে ধাপে উদাহরণ এই অধ্যায়ের Section 24 (Prediction অংশ)-এ দেখানো হয়েছে।

17. Advantages

- **সহজ ও দ্রুত (Simple and Fast):** অ্যালগরিদমটি বোঝা ও বাস্তবায়ন করা তুলনামূলক সহজ, এবং বড় ডেটাসেটেও দ্রুত ফলাফল দেয়।
- **Scalability:** ডেটার পরিমাণ বৃদ্ধি পেলেও Time Complexity প্রায় Linear ($O(N \cdot K \cdot D \cdot T)$) থাকে বলে এটি Big Data-তেও কার্যকর।
- **সহজ ব্যাখ্যাযোগ্যতা:** প্রতিটি Cluster-এর একটি সুনির্দিষ্ট Centroid থাকায় ফলাফল ব্যাখ্যা করা এবং Business Decision নেওয়া সহজ হয়।
- **সহজ Convergence:** প্রতিটি Iteration-এ Objective Function কখনো বৃদ্ধি পায় না, ফলে অ্যালগরিদমটি নিশ্চিতভাবে একটি স্থিতিশীল সমাধানে পৌঁছায়।

18. Disadvantages

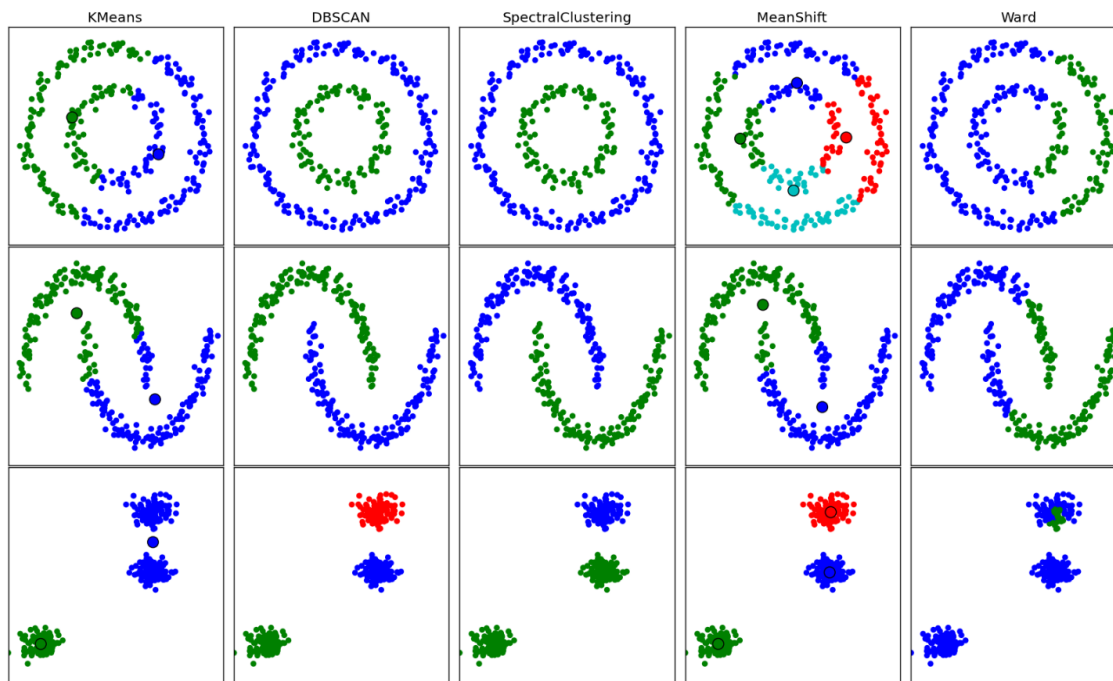
Limitations of K-Means Clustering

১. আউটলায়ারের প্রতি চরম সংবেদনশীলতা (Extreme Sensitivity to Outliers)

- **সমস্যার প্রকৃতি:** K-Means অ্যালগরিদম প্রতিটি ক্লাস্টারের নতুন কেন্দ্রবিন্দু নির্ধারণের জন্য গাণিতিক গড় বা Arithmetic Mean ব্যবহার করে।
- **প্রভাব:** ডেটাসেটে যদি কোনো চরম Outlier বা ব্যতিক্রমী বড়/ছোট ডেটা পয়েন্ট থাকে, তবে তা Mean-কে নিজের দিকে মারাত্মকভাবে টেনে নিয়ে যায়। ফলে পুরো ক্লাস্টারের আসল জ্যামিতিক বাউন্ডারি বা আকৃতি বিচ্যুত (Distorted) হয়ে পড়ে।

২. K-এর মান আগে থেকে নির্ধারণ করতে হয় (Pre-defined Number of Clusters)

- **সমস্যার প্রকৃতি:** K-Means চালানোর একদম শুরুতে ইউজারকে নিজের থেকে n_clusters বা K-এর মান নির্দিষ্ট করে দিতে হয়।
- **প্রভাব:** বাস্তব জীবনের অজানা ডেটাসেট দেখে মানুষের পক্ষে আগে থেকে জানা সম্ভব নয় যে ভেতরে কয়টি আসল গ্রুপ লুকিয়ে আছে। যদিও Elbow Method বা Silhouette Score ব্যবহার করে এটি অনুমান করা যায়, তবে তা কম্পিউটেশনালি সময়সাপেক্ষ এবং সবসময় নিখুঁত সমাধান দেয় না।



classification - What is the meaning of spherical dataset? - Data Science Stack Exchange

৩ . শুধুমাত্র বৃত্তাকার বা গোলকধাঁধার মতো ক্লাস্টারে সীমাবদ্ধ (Assumes Spherical Clusters)

- **সমস্যার প্রকৃতি:** K-Means সম্পূর্ণভাবে Euclidean Distance-এর ওপর ভিত্তি করে ডেটা পয়েন্টগুলোকে নিকটবর্তী সেন্ট্রয়েডে অ্যাসাইন করে।
- **প্রভাব:** এটি ধরে নেয় যে সব ক্লাস্টারের আকৃতি বৃত্তাকার বা গোলকাকার (Spherical) এবং তাদের ঘনত্ব (Density) সমান। উপাত্তের বিন্যাস যদি জটিল, লম্বাটে, বাঁকা (Elongated/Non-convex patterns) কিংবা একে অপরের ভেতরে জটিলভাবে শিকলের মতো ঢুকে থাকে, তবে K-Means সেখানে সম্পূর্ণ ব্যর্থ হয়।

৪ . লোকাল মিনিমারে আটকে যাওয়ার ঝুঁকি (Prone to Local Minima)

- **সমস্যার প্রকৃতি:** অ্যালগরিদমটি শুরুতে প্রাথমিক সেন্ট্রয়েডগুলো সম্পূর্ণ আন্দাজে বা র্যান্ডমভাবে বসানোর মাধ্যমে (Centroid Initialization) কাজ শুরু করে।
- **প্রভাব:** প্রাথমিক সেন্ট্রয়েড সিলেকশন যদি ভালো না হয়, তবে মডেলটি গ্লোবাল মিনিমাম (Global Minimum) বা সর্বোত্তম বৈশ্বিক কস্ট পয়েন্টে পৌঁছাতে না পেরে মাঝপথেই লোকাল মিনিমারে (Local Minima) আটকে যায় এবং ভুল ক্লাস্টারিং করে। যদিও Scikit-Learn ডিফল্টভাবে k-means++ এবং n_init ব্যবহার করে এই সমস্যা কমানোর চেষ্টা করে।

৫ . ফিচার স্কেলিং বাধ্যতামূলক (High Dependency on Feature Scaling)

- **সমস্যার প্রকৃতি:** K-Means একটি পিওর দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম।

- **প্রভাব:** যদি ডেটাসেটের ফিচারগুলোর স্কেল অসমান হয় (যেমন: Salary কলামের মান 0 থেকে 1,000,000 এবং Age কলামের মান 0 থেকে 100), তবে বড় স্কেলের ফিচারটি পুরো জ্যামিতিক দূরত্বকে ডমিনেন্ট (Dominant) করে ফেলবে। ফলে ছোট ফিচারের গুরুত্ব সম্পূর্ণ হারিয়ে যায়। তাই StandardScaler প্রয়োগ না করলে এর আউটপুট অর্থহীন হয়ে পড়ে।

৬. বৃহৎ ডাইমেনশনে কার্যকারিতা হ্রাস (Curse of Dimensionality)

- **সমস্যার প্রকৃতি:** উপাত্ত ক্ষেত্রে ফিচারের সংখ্যা বা ডাইমেনশন (D) যখন বহুগুণ বেড়ে যায়, তখন উচ্চ-মাত্রিক স্পেসে প্রতিটি পয়েন্টের মধ্যবর্তী ইউক্লিডিয়ান দূরত্ব প্রায় সমান হতে শুরু করে।
- **প্রভাব:** ডাইমেনশন অত্যধিক বেশি হলে K-Means আর "সবচেয়ে কাছের সেন্ট্রয়েড" সঠিকভাবে আলাদা করতে পারে না। এতে ক্লাস্টারিং এক্যুরেসি মারাত্মকভাবে কমে যায় এবং কম্পিউটেশনাল মেমোরি কস্ট বহুগুণ বৃদ্ধি পায়।

19. Applications

- গ্রাহকদের ক্রয়-আচরণের ভিত্তিতে Customer Segmentation ও টার্গেটেড মার্কেটিং।
- ডিজিটাল ছবির Color Quantization ও Image Segmentation।
- Document বা সংবাদ প্রতিবেদনের Topic-ভিত্তিক গ্রুপিং।
- Data Preprocessing-এ Supervised Model-এর জন্য নতুন Feature (Cluster Distance) তৈরি করা।
- Anomaly Detection-এ প্রাথমিক ধাপ হিসেবে স্বাভাবিক ডেটার Cluster চিহ্নিত করা।

20. Comparison with Previous Algorithm

K-Means হলো এই Part-এ আলোচিত **প্রথম Clustering অ্যালগরিদম**, তাই এখানে তুলনা করার মতো কোনো পূর্ববর্তী Clustering অ্যালগরিদম নেই। তবে পরবর্তী অধ্যায়গুলোতে (Agglomerative Clustering ও DBSCAN) আলোচনার পর, এই Part-এর শেষে সংযোজিত বিস্তারিত তুলনামূলক টেবিলগুলোতে K-Means-কে Agglomerative Clustering এবং DBSCAN-এর সাথে Training Process, Distance Measure, Cluster আকৃতি, Noise Handling, Scalability ও Computational Cost-এর নিরিখে সরাসরি তুলনা করা হয়েছে।

21. Common Mistakes

- **Feature Scaling উপেক্ষা করা:** StandardScaler বা MinMaxScaler ছাড়া সরাসরি K-Means চালালে বড় স্কেলের Feature ফলাফলকে ডমিনেন্ট করে ফেলে।
- **K-এর মান যাচাই ছাড়াই বসিয়ে দেওয়া:** Elbow Method বা Silhouette Score দিয়ে যাচাই না করে আন্দাজে K নির্ধারণ করলে ভুল Cluster গঠন হয়।
- **n_init=1 ব্যবহার করা:** এতে Random Initialization-এর কারণে মডেল Local Minima-তে আটকে যাওয়ার ঝুঁকি অনেক বেড়ে যায়; Default n_init="auto" বা একাধিক Run ব্যবহার করা উচিত।
- **অ-গোলাকার ডেটাতে K-Means প্রয়োগ করা:** ডেটার আকৃতি জটিল বা Non-convex হলে K-Means ব্যর্থ হবে; এক্ষেত্রে DBSCAN বা Agglomerative বেশি উপযোগী।

- **Outlier আগে থেকে পরিক্ষার না করা:** যেহেতু Mean অত্যন্ত Outlier-sensitive, তাই ট্রেনিং-এর আগে চরম Outlier চিহ্নিত করে বাদ দেওয়া বা পৃথকভাবে চিহ্নিত করা উচিত।

22. Interview Questions

প্রশ্ন ১: K-Means কীভাবে আউটলায়ার (Outliers) হ্যান্ডেল করে এবং কস্ট ফাংশনের ওপর এর প্রভাব কী?

- **উত্তর:** K-Means অ্যালগরিদম আউটলায়ার বা ব্যতিক্রমী ডেটা পয়েন্টের প্রতি অত্যন্ত সংবেদনশীল (Sensitive), কারণ এটি সেন্ট্রয়েড আপডেট করার জন্য গাণিতিক গড় (Mean) ব্যবহার করে।
- **প্রভাব:** যেহেতু কস্ট ফাংশনটি প্রতিটি পয়েন্ট থেকে সেন্ট্রয়েডের ইউক্লিডিয়ান দূরত্বের বর্গকে সর্বনিম্ন করার চেষ্টা করে ($J = \sum \sum \|x_i - \mu_j\|^2$), তাই একটিমাত্র বিশাল দূরত্বের আউটলায়ার পয়েন্ট পুরো কস্ট ফাংশনকে ডমিনেন্ট বা প্রভাবিত করতে পারে। এর ফলে সেন্ট্রয়েডটি আক্ষরিক অর্থেই আউটলায়ার পয়েন্টটির দিকে মারাত্মকভাবে সরে যায় এবং ক্লাস্টারের আসল জ্যামিতিক সীমানা নষ্ট হয়।

প্রশ্ন ২: Scikit-Learn-এর KMeans-এ n_init প্যারামিটারটির উদ্দেশ্য কী এবং এটি কীভাবে লোকাল মিনিমা রোধ করে?

- **উত্তর:** বাস্তব ক্ষেত্রে K-Means অ্যালগরিদমটি অত্যন্ত দ্রুত কাজ করলেও এটি প্রাথমিক সেন্ট্রয়েডগুলো শুরুতে ঠিক কোথায় বসানো হচ্ছে তার ওপর ভিত্তি করে প্রায়ই লোকাল মিনিমাতে (Local Minima) আটকে যায়।
- **মেকানিজম:** n_init প্যারামিটারটি নির্ধারণ করে যে অ্যালগরিদমটি সম্পূর্ণ ভিন্ন ভিন্ন র্যান্ডম সেন্ট্রয়েড বীজ (Centroid seeds) নিয়ে মোট কতবার নতুন করে রান করবে। Scikit-Learn এই ধারাবাহিক রানগুলো সম্পন্ন করার পর যে রানের আউটপুট সবচেয়ে সর্বনিম্ন জড়তা বা কস্ট (inertia_) প্রদান করে, সেটিকেই চূড়ান্ত ক্লাস্টার মডেল হিসেবে বেছে নেয়। এর ফলে মডেলটির গ্লোবাল মিনিমাম (Global Minimum) বা সবচেয়ে সঠিক সমাধান খুঁজে পাওয়ার সম্ভাবনা বহুগুণ বেড়ে যায়।

প্রশ্ন ৩: K-Means ক্লাস্টারিং চালানোর আগে ফিচার স্কেলিং (Feature Scaling) করা কেন বাধ্যতামূলক?

- **উত্তর:** K-Means একটি দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম, যা ক্লাস্টারের সীমানা নির্ধারণের সময় প্রতিটি ফিচার বা ডাইমেনশনের দূরত্বকে সমান গুরুত্ব দেয়।
- **মেকানিজম:** যদি ডেটাসেটের ফিচারগুলোর স্কেল বা রেঞ্জ অসমান হয় (যেমন: Salary কলামের মান ০ থেকে ১,০০০,০০০ এবং Age কলামের মান ০ থেকে ১০০), তবে দূরত্ব গণনার পুরো প্রক্রিয়াটি সম্পূর্ণভাবে বড় স্কেলের ফিচারটি (Salary) দ্বারা নিয়ন্ত্রিত হবে এবং ছোট স্কেলের ফিচারের (Age) গুরুত্ব হারিয়ে যাবে। StandardScaler-এর মতো উপাদান ব্যবহার করে ফিচার স্কেলিং করলে সমস্ত ফিচারের ভ্যারিয়েন্স একই স্কেলে চলে আসে, যা প্রতিটি ফিচারকে ক্লাস্টার গঠনে সমানভাবে অবদান রাখার সুযোগ করে দেয়।

প্রশ্ন ৪: KMeans-এর ক্ষেত্রে লয়েড (Lloyd) এবং এলকান (Elkan) অ্যালগরিদমের মধ্যে মূল পার্থক্য কী?

- **উত্তর:** লয়েড এবং এলকান উভয় অ্যালগরিদমই ব্যাকঅ্যান্ডে একই K-Means গাণিতিক সমস্যার সমাধান করে, তবে তাদের কাজ করার অভ্যন্তরীণ কৌশল ভিন্ন।
- **লয়েড অ্যালগরিদম (Lloyd's):** এটি হলো চিরাচরিত Expectation-Maximization স্টাইলের অ্যালগরিদম, যা প্রতি ইটারেশনে প্রতিটি ডেটা পয়েন্ট থেকে প্রতিটি সেন্ট্রয়েডের দূরত্ব ম্যানুয়ালি গণনা করে।
- **এলকান অ্যালগরিদম (Elkan's):** এটি মূলত জ্যামিতিক ত্রিভুজ অসমতা (Triangle Inequality) ব্যবহার করে কাজ করে। যেসব ডেটাসেটে ক্লাস্টারগুলো খুব সুনির্দিষ্টভাবে আলাদা থাকে, সেখানে এটি দূরত্বের একটি উচ্চ ও নিম্ন সীমাদ্র্যাক করে অপ্রয়োজনীয় দূরত্ব গণনা এড়িয়ে চলে। ফলে এটি লয়েডের চেয়ে অনেক দ্রুত কাজ করতে পারে, তবে ব্যাকঅ্যান্ডে একটি অতিরিক্ত দূরত্ব অ্যারে মেমোরিতে ধরে রাখার কারণে এটি অত্যন্ত র‍্যাম বা মেমোরি-সংবেদনশীল (Memory intensive)।

প্রশ্ন ৫: মডেলটি সম্পূর্ণ স্থির (Converge) হওয়ার আগেই যদি অ্যালগরিদমটি বন্ধ হয়ে যায়, তবে সেন্ট্রয়েড ও ক্লাস্টার অ্যাসাইনমেন্টের কী ঘটে?

- **উত্তর:** যদি KMeans অ্যালগরিদমটি পুরোপুরি স্থির বা কনভার্জ হওয়ার আগেই নির্ধারিত সর্বোচ্চ ইটারেশন সীমা (max_iter) স্পর্শ করে বা সহনশীলতার মাত্রা (tol) অতিক্রমের কারণে বন্ধ হয়ে যায়, তবে এর অভ্যন্তরীণ গাণিতিক অবস্থা সামঞ্জস্যহীন (Inconsistent) হয়ে পড়ে।
- **প্রভাব:** এমন পরিস্থিতিতে মেমোরিতে থাকা সেন্ট্রয়েডের স্থানাঙ্কগুলো (cluster_centers_) ওই ক্লাস্টারের অন্তর্ভুক্ত ডেটা পয়েন্টগুলোর প্রকৃত গাণিতিক গড়ের সাথে মিলবে না। তবে প্রডাকশনে কাজ সচল রাখার জন্য, এস্টিমেটরটি একদম শেষ ইটারেশনের পর পুনরায় ডেটা পয়েন্টগুলোর লেবেল রি-অ্যাসাইন করে দেয়, যাতে চূড়ান্ত labels_ অ্যাট্রিবিউটটি মূল ডেটার ওপর চালানো .predict() মেথডের ফলাফলের সাথে পুরোপুরি সামঞ্জস্যপূর্ণ থাকে।

23. Summary

K-Means ক্লাস্টারিংয়ের চূড়ান্ত সারসংক্ষেপ

পেশাদারী ইন্টারভিউ এবং পরীক্ষার কুইক রিভিশনের সুবিধার্থে পুরো অধ্যায়ের মূল গাণিতিক ও কার্ঠামোগত বিষয়গুলো নিচে সংক্ষেপে টেবিল এবং কী-পয়েন্টের মাধ্যমে উপস্থাপন করা হলো:

১. কুইক রিভিশন ম্যাট্রিক্স (Quick Revision Matrix)

ক্যাটাগরি	মূল আলোচ্য বিষয় ও গাণিতিক রূপ
-----------	--------------------------------

মূল সমীকরণ (Core Formula)	$J = \sum_{j=1}^K \sum_{x_i \in S_j} \ x_i - \mu_j\ ^2$ (Within-Cluster Sum of Squares)
অ্যালগরিদমের ধরন	Unsupervised, Iterative Partitional Clustering, Eager Learner
জটিলতা (Complexity)	গড় সময় জটিলতা: $O(K \cdot n \cdot T)$; যেখানে T = Iterations
শুরুকরণ কৌশল (Init)	k-means++ (দ্রুত কনভার্জেন্স ও লোকাল মিনিমা এড়ানোর জন্য ডিফল্ট কৌশল)
স্থিরতার শর্ত (Convergence)	সর্বোচ্চ ইটারেশন সীমা (max_iter=300) অথবা সেন্ট্রয়েডের নড়াচড়া থ্রেশহোল্ডের নিচে নামা (tol=1e-4)
মূল মেথডসমূহ (API)	.fit(), .predict(), .transform(), .fit_predict(), .fit_transform()
প্রধান অ্যট্রিবিউটস	cluster_centers_ (সেন্ট্রয়েড), labels_ (ক্লাস্টার ইনডেক্স), inertia_ (WCSS কস্ট)

২. মূল জাস্টিফিকেশন ও টেকনিক্যাল ইনসাইটস (Core Structural Insights)

- ফিচার স্কেলিং বাধ্যতামূলক:** K-Means যেহেতু জ্যামিতিক ইউক্লিডিয়ান দূরত্বের ওপর ভিত্তি করে দল গঠন করে, তাই অসমান স্কেলের ফিচার থাকলে বড় স্কেলের ফিচারটি পুরো দূরত্বকে ডমিনেট করে ফেলে। এই কারণে মডেল ফিট করার পূর্বে StandardScaler ব্যবহার করা অপরিহার্য।
- লোকাল মিনিমা ও রিস্টার্ট লজিক:** অ্যালগরিদমটি শুরুতে র্যান্ডম সেন্ট্রয়েড নির্বাচনের ওপর অত্যন্ত নির্ভরশীল হওয়ায় প্রায়ই লোকাল মিনিমাতে আটকে যায়। এই সমস্যা দূর করতে n_init='auto' প্যারামিটার ব্যবহার করে ভিন্ন ভিন্ন সেন্ট্রয়েড বীজ নিয়ে ব্যাকঅ্যান্ডে একাধিকবার লুপ চালানো হয় এবং সর্বনিম্ন ইনোশিয়ার রানটি বেছে নেওয়া হয়।
- এলবো মেথড (Elbow Method):** ক্লাস্টারের সংখ্যা বা K -এর মান আগে থেকে জানা না থাকলে, প্রতিবার K -এর মান বাড়িয়ে ইনোশিয়ার বা WCSS-এর ড্রপ রেট প্লট করা হয়। গ্রাফের রেখাটি যে বিন্দুতে এসে ছুট করে বাটির কনুইয়ের মতো (Elbow) বেঁকে যায় এবং কস্ট কমানোর গতি ধীর হয়ে যায়, সেটিকেই আদর্শ ক্লাস্টার সংখ্যা হিসেবে বিবেচনা করা হয়।
- পাইপলাইন ইন্টিগ্রেশন:** প্রডাকশন কোডে ডেটা লিক (Data Leakage) সম্পূর্ণ বন্ধ করতে এবং ফিচার স্কেলিং ও মডেলিং মেকানিজমকে লিক-প্রুফ রাখতে সর্বদা ColumnTransformer এবং Pipeline ক্রমানুসারে সাজিয়ে GridSearchCV-এর ভেতর দিয়ে রান করা উচিত।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

এই সেকশনে K-Means অ্যালগরিদমের সম্পূর্ণ গাণিতিক প্রক্রিয়া একটি নতুন, ভিন্ন ছোট ডেটাসেটের ওপর শুরু থেকে শেষ পর্যন্ত, কোনো ধাপ বাদ না দিয়ে ম্যানুয়ালি দেখানো হলো।

Dataset

আমাদের কাছে $D = 2$ ডাইমেনশনের একটি ডেটাসেট আছে, যেখানে $N = 5$ টি Data Point রয়েছে:

- $P_1 = (2,4)$
- $P_2 = (3,5)$
- $P_3 = (8,3)$
- $P_4 = (9,2)$
- $P_5 = (1,6)$

আমরা $K = 2$ নির্ধারণ করলাম।

Initialization

প্রাথমিক Centroid হিসেবে ডেটাসেট থেকে সরাসরি দুটি পয়েন্ট বেছে নেওয়া হলো (Forgy Initialization পদ্ধতি):

- $\mu_1^{(0)} = P_1 = (2,4)$
- $\mu_2^{(0)} = P_3 = (8,3)$

Core Formula

Assignment ধাপে প্রতিটি পয়েন্টকে সবচেয়ে কম Squared Euclidean Distance-এর Centroid-এ বসানো হয়:

$$\|x_i - \mu_j\|^2 = (x_{i1} - \mu_{j1})^2 + (x_{i2} - \mu_{j2})^2$$

আর প্রতিটি Iteration শেষে Centroid নতুন করে গণনা করা হয়:

$$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

Distance Calculation এবং Iteration 1

পয়েন্ট	$\mu_1 = (2, 4)$ থেকে $\ \cdot\ ^2$	$\mu_2 = (8, 3)$ থেকে $\ \cdot\ ^2$	Assignment
$P_1(2,4)$	$(2 - 2)^2 + (4 - 4)^2 = 0$	$(2 - 8)^2 + (4 - 3)^2 = 36 + 1 = 37$	Cluster 1
$P_2(3,5)$	$(3 - 2)^2 + (5 - 4)^2 = 1 + 1 = 2$	$(3 - 8)^2 + (5 - 3)^2 = 25 + 4 = 29$	Cluster 1

$P_3(8,3)$	$(8-2)^2 + (3-4)^2 = 36 + 1 = 37$	$(8-8)^2 + (3-3)^2 = 0$	Cluster 2
$P_4(9,2)$	$(9-2)^2 + (2-4)^2 = 49 + 4 = 53$	$(9-8)^2 + (2-3)^2 = 1 + 1 = 2$	Cluster 2
$P_5(1,6)$	$(1-2)^2 + (6-4)^2 = 1 + 4 = 5$	$(1-8)^2 + (6-3)^2 = 49 + 9 = 58$	Cluster 1

Iteration 1 শেষে Cluster গ্রুপ: $S_1 = \{P_1, P_2, P_5\}, S_2 = \{P_3, P_4\}$

Iteration 1-এর SSE:

$$J^{(1)} = (0 + 2 + 5) + (0 + 2) = 7 + 2 = 9$$

Centroid Update:

$$\mu_1^{(1)} = \left(\frac{2+3+1}{3}, \frac{4+5+6}{3} \right) = \left(\frac{6}{3}, \frac{15}{3} \right) = (2, 5)$$

$$\mu_2^{(1)} = \left(\frac{8+9}{2}, \frac{3+2}{2} \right) = \left(\frac{17}{2}, \frac{5}{2} \right) = (8.5, 2.5)$$

Iteration 2 (Round 2)

নতুন Centroid $\mu_1 = (2, 5)$ এবং $\mu_2 = (8.5, 2.5)$ ব্যবহার করে পুনরায় দূরত্ব গণনা করা হলো:

পয়েন্ট	$\mu_1 = (2, 5)$ থেকে $\ \cdot \ ^2$	$\mu_2 = (8.5, 2.5)$ থেকে $\ \cdot \ ^2$	Assignment
$P_1(2,4)$	$(0)^2 + (-1)^2 = 1$	$(-6.5)^2 + (1.5)^2 = 42.25 + 2.25 = 44.5$	Cluster 1
$P_2(3,5)$	$(1)^2 + (0)^2 = 1$	$(-5.5)^2 + (2.5)^2 = 30.25 + 6.25 = 36.5$	Cluster 1
$P_3(8,3)$	$(6)^2 + (-2)^2 = 36 + 4 = 40$	$(-0.5)^2 + (0.5)^2 = 0.25 + 0.25 = 0.5$	Cluster 2
$P_4(9,2)$	$(7)^2 + (-3)^2 = 49 + 9 = 58$	$(0.5)^2 + (-0.5)^2 = 0.25 + 0.25 = 0.5$	Cluster 2
$P_5(1,6)$	$(-1)^2 + (1)^2 = 1 + 1 = 2$	$(-7.5)^2 + (3.5)^2 = 56.25 + 12.25 = 68.5$	Cluster 1

Iteration 2 শেষে Cluster গ্রুপ: $S_1 = \{P_1, P_2, P_5\}, S_2 = \{P_3, P_4\}$ — আগের Iteration-এর মতোই অপরিবর্তিত।

Iteration 2-এর SSE:

$$J^{(2)} = (1 + 1 + 2) + (0.5 + 0.5) = 4 + 1 = 5$$

Centroid Update:

$$\mu_1^{(2)} = \left(\frac{2+3+1}{3}, \frac{4+5+6}{3} \right) = (2,5) \quad (\text{অপরিবর্তিত})$$

$$\mu_2^{(2)} = \left(\frac{8+9}{2}, \frac{3+2}{2} \right) = (8.5, 2.5) \quad (\text{অপরিবর্তিত})$$

Stopping Condition

যেহেতু $\mu_1^{(2)} = \mu_1^{(1)}$ এবং $\mu_2^{(2)} = \mu_2^{(1)}$ — অর্থাৎ Centroid আর সরছে না এবং Cluster Assignment-ও অপরিবর্তিত থেকেছে — তাই অ্যালগরিদম **Convergence** অর্জন করেছে এবং ট্রেনিং এখানেই শেষ হয়ে যাবে। (লক্ষণীয়, SSE ৯ থেকে কমে ৫-এ নেমে এসে স্থির হয়েছে, যা প্রমাণ করে প্রতিটি Iteration কস্টকে কখনো বাড়ায় না, শুধু কমায় বা অপরিবর্তিত রাখে।)

Training Progress Table

Iteration	Centroid 1 (μ_1)	Centroid 2 (μ_2)	SSE
Initialization	(2,4)	(8,3)	—
1	(2,5)	(8.5,2.5)	9.0
2 (Converged)	(2,5)	(8.5,2.5)	5.0

Final Cluster Assignment

Cluster	অন্তর্ভুক্ত পয়েন্ট	চূড়ান্ত Centroid	Cluster SSE
Cluster 1	$P_1(2,4), P_2(3,5), P_5(1,6)$	(2,5)	$1 + 1 + 2 = 4$
Cluster 2	$P_3(8,3), P_4(9,2)$	(8.5,2.5)	$0.5 + 0.5 = 1$

মোট SSE = 4 + 1 = 5 (Training Progress টেবিলের সাথে হুবহু মিলছে)।

Prediction (নতুন Sample-এর জন্য)

ধরা যাক একটি নতুন Sample $P_{new} = (7,3)$ এসেছে। মডেলটি Training-এর সময়কার একই সূত্র ব্যবহার করে নতুন পয়েন্টের সাথে প্রতিটি চূড়ান্ত Centroid-এর দূরত্ব গণনা করবে:

$$\|P_{new} - \mu_1\|^2 = (7 - 2)^2 + (3 - 5)^2 = 25 + 4 = 29$$

$$\|P_{new} - \mu_2\|^2 = (7 - 8.5)^2 + (3 - 2.5)^2 = 2.25 + 0.25 = 2.5$$

যেহেতু $2.5 < 29$, তাই $P_{new} \rightarrow \text{Cluster 2}$ ।

Interpretation

চূড়ান্ত ফলাফল থেকে দেখা যাচ্ছে ডেটাসেটটি স্বাভাবিকভাবেই দুটি ভৌগোলিকভাবে পৃথক অঞ্চলে বিভক্ত হয়েছে — উপরে-বামে থাকা P_1, P_2, P_5 একটি Cluster এবং নিচে-ডানে থাকা P_3, P_4 আরেকটি Cluster গঠন করেছে। নতুন পয়েন্ট (7,3)-ও জ্যামিতিকভাবে দ্বিতীয় গ্রুপের কাছাকাছি হওয়ায় স্বাভাবিকভাবেই Cluster 2-তে classified হয়েছে, যা মডেলের সিদ্ধান্তের স্বজ্ঞাত যথার্থতা (Intuitive Correctness) প্রমাণ করে।

Chapter 15 — Agglomerative Clustering

1. Introduction

Agglomerative Clustering

Agglomerative Clustering হলো একটি Unsupervised Machine Learning Algorithm এবং Hierarchical Clustering-এর সবচেয়ে জনপ্রিয় ও ব্যাপকভাবে ব্যবহৃত ধরন, যা 'Bottom-Up' বা 'নিচ থেকে ওপরে' পদ্ধতিতে কাজ করে। Supervised Learning-এর মতো এখানে কোনো লেবেল বা টার্গেট ভ্যারিয়েবল আগে থেকে দেওয়া থাকে না — অ্যালগরিদম সম্পূর্ণ স্বাধীনভাবে ডেটার মধ্যে থাকা স্বাভাবিক গ্রুপ বা Cluster খুঁজে বের করে, শুধুমাত্র পয়েন্টগুলোর মধ্যকার দূরত্ব বা সাদৃশ্য বিশ্লেষণ করে।

2. Why This Algorithm Was Developed

K-Means-এর একটি বড় সীমাবদ্ধতা হলো, এটি ব্যবহারকারীকে শুরুতেই Cluster-এর সংখ্যা (K) নির্দিষ্ট করে দিতে বাধ্য করে, এবং একবার Cluster তৈরি হয়ে গেলে ডেটার অন্তর্নিহিত Hierarchical বা স্তরবিন্যাস সম্পর্ক সম্পর্কে কোনো তথ্য দেয় না। বাস্তব জীবনের অনেক ডেটাতে (যেমন জীববিজ্ঞানে প্রজাতির শ্রেণিবিন্যাস, বা প্রতিষ্ঠানের সাংগঠনিক কাঠামো) প্রাকৃতিকভাবেই একটি ধাপে ধাপে Hierarchy বিদ্যমান থাকে। Agglomerative Clustering ঠিক এই প্রয়োজন থেকেই তৈরি হয়েছে — এটি Cluster সংখ্যা আগে থেকে না জেনেই ধাপে ধাপে ডেটাকে একত্রিত করে একটি সম্পূর্ণ Hierarchy তৈরি করে, যা থেকে গবেষক পরে প্রয়োজন অনুযায়ী যেকোনো স্তরে Cluster বেছে নিতে পারেন।

3. Real-life Motivation

জীববিজ্ঞানীরা যখন বিভিন্ন প্রজাতির DNA বা জিন এক্সপ্রেশন ডেটা বিশ্লেষণ করে তাদের বিবর্তনীয় সম্পর্ক (Phylogenetic Tree) তৈরি করেন, তখন তাদের আগে থেকে জানা থাকে না ঠিক কয়টি প্রধান শাখায় প্রজাতিগুলো ভাগ হবে। তাদের প্রয়োজন এমন একটি পদ্ধতি যা প্রতিটি স্তরে দেখাতে পারে কোন প্রজাতি কার সবচেয়ে কাছাকাছি আত্মীয়। Agglomerative Clustering এই ধরনের সমস্যার জন্য আদর্শ — এটি একটি সম্পূর্ণ Dendrogram তৈরি করে দেয় যা প্রতিটি স্তরে সম্পর্কগুলো স্পষ্টভাবে দেখায়। একইভাবে, প্রতিষ্ঠানের সাংগঠনিক কাঠামো বিশ্লেষণ, সামাজিক নেটওয়ার্কে Community-এর স্তরবিন্যাস বিশ্লেষণ, বা গ্রাহক-বিভাজনে ধাপে ধাপে সূক্ষ্ম থেকে বৃহৎ পর্যায়ে বিশ্লেষণের প্রয়োজন হলেও Agglomerative Clustering ব্যবহৃত হয়।

4. Intuition

উদাহরণ (Intuition)

কল্পনা করুন, একটি ক্লাসরুমে ৬ জন ছাত্রছাত্রী রয়েছে এবং আপনি তাদের বসার আসন থেকে দূরত্বের ভিত্তিতে বন্ধু-দল ভাগ করতে চান। শুরুতে প্রতিজন ছাত্রকে একটি করে আলাদা দল হিসেবে ধরা হয় — অর্থাৎ ৬ জন থাকলে শুরুতে ৬টি দল। এরপর যে দুইজন ছাত্র একে অপরের সবচেয়ে কাছাকাছি বসে, তাদের একত্রিত করে একটি নতুন দল বানানো হয়, ফলে এখন দলের সংখ্যা কমে ৫টি হয়ে যায়। এই প্রক্রিয়া বারবার চলতে থাকে — প্রতিবার সবচেয়ে কাছাকাছি অবস্থিত দুটি দলকে একত্রিত করা হয় — যতক্ষণ না শেষ পর্যন্ত সবাই মিলে একটি বড় দলে পরিণত হয়। এই সম্পূর্ণ একত্রীকরণ প্রক্রিয়াটি ধাপে ধাপে লিখে রাখলে একটি গাছের মতো কাঠামো তৈরি হয়, যাকে বলা হয় Dendrogram। এই Dendrogram-এর যেকোনো একটি উচ্চতায় গাছটি 'কেটে' দিলেই আমরা কাঙ্ক্ষিত সংখ্যক Cluster পেয়ে যাই — এটিই Agglomerative (অর্থাৎ 'একত্রীকরণ') Clustering-এর মূল ধারণা।

5. Working Principle

মূল গাঠনিক প্রক্রিয়া (How it Works) – ধাপে ধাপে

ধাপ ১ – প্রাথমিক অবস্থা

প্রতিটি ডেটা পয়েন্টকে একটি করে সম্পূর্ণ স্বতন্ত্র Cluster হিসেবে ধরা হয়। n সংখ্যক Sample থাকলে শুরুতে ঠিক n সংখ্যক Cluster থাকে।

ধাপ ২ – দূরত্ব গণনা

প্রতিটি সম্ভাব্য Cluster জোড়ার মধ্যে দূরত্ব (Distance) গণনা করা হয় (সাধারণত Euclidean Distance ব্যবহার করা হয়) এবং এই দূরত্বগুলো একটি Distance Matrix-এ সংরক্ষণ করা হয়।

ধাপ ৩ – সবচেয়ে কাছের জোড়া একত্রিত করা

Distance Matrix থেকে সবচেয়ে কম দূরত্বে থাকা দুটি Cluster খুঁজে বের করে তাদের একত্রিত করে একটি নতুন, বড় Cluster তৈরি করা হয়।

ধাপ ৪ – Distance Matrix আপডেট ও পুনরাবৃত্তি

নতুন তৈরি হওয়া Cluster-এর সাথে বাকি সব Cluster-এর দূরত্ব পুনরায় গণনা করা হয় (Linkage পদ্ধতি অনুযায়ী), এবং আবার সবচেয়ে কাছের জোড়া একত্রিত করা হয়। এই প্রক্রিয়া বারবার চলতে থাকে, প্রতিবার Cluster-এর সংখ্যা একটি করে কমতে থাকে, যতক্ষণ না সবগুলো ডেটা একটি একক Cluster-এ পরিণত হয়।

ধাপ ৫ – ফলাফল: Dendrogram

পুরো একত্রীকরণ প্রক্রিয়াটি একটি গাছের মতো চিত্র বা Dendrogram-এ প্রদর্শিত হয়, যেখানে X-অক্ষে প্রতিটি ডেটা পয়েন্ট এবং Y-অক্ষে একত্রীকরণের সময়কার দূরত্ব দেখানো হয়। একটি নির্দিষ্ট উচ্চতায় (Distance Threshold) এই গাছটি অনুভূমিকভাবে কেটে দিলে, ঠিক যতগুলো উল্লম্ব রেখা কাটা হবে, ততগুলো Cluster পাওয়া যায়।

6. Mathematical Backend

Linkage Criteria – কীভাবে দুই Cluster-এর দূরত্ব মাপা হয়?

যখন একটি Cluster-এ একাধিক পয়েন্ট থাকে (যেমন উপরের {A,B} Cluster), তখন প্রশ্ন ওঠে — এই Cluster-এর সাথে আরেকটি Cluster-এর দূরত্ব ঠিক কীভাবে মাপা হবে? এর জন্য একাধিক পদ্ধতি রয়েছে, যাকে বলা হয় Linkage।

Linkage	সংজ্ঞা	বৈশিষ্ট্য
Single	দুই Cluster-এর সবচেয়ে কাছের দুই পয়েন্টের দূরত্ব	লম্বা, শিকল-আকৃতির (Chain-like) Cluster তৈরি করে; Noise বা Outlier-এ সংবেদনশীল

Complete	দুই Cluster-এর সবচেয়ে দূরের দুই পয়েন্টের দূরত্ব	কমপ্যাক্ট ও প্রায় সমান আকারের Cluster তৈরি করে
Average	দুই Cluster-এর মধ্যে সব সম্ভাব্য জোড়া পয়েন্টের দূরত্বের গড়	Single ও Complete-এর মাঝামাঝি একটি ভারসাম্যপূর্ণ ফলাফল দেয়
Ward	একত্রিত করলে Cluster-এর ভেতরের Variance কতটা বাড়বে তার পরিমাপ (Minimum Variance পদ্ধতি)	Scikit-Learn-এর ডিফল্ট; সাধারণত সবচেয়ে সুসম ও গোলাকার Cluster দেয়

7. Formula

গাণিতিক ভিত্তি – Ward's Method

Ward Linkage-এ দুটি Cluster A এবং B একত্রিত করলে Cluster-এর ভেতরের Variance বা 'বিশৃঙ্খলা' ঠিক কতটুকু বৃদ্ধি পাবে, তা নিচের সূত্রে গণনা করা হয়। এখানে n_A , n_B হলো প্রতিটি Cluster-এর সদস্য সংখ্যা এবং c_A , c_B হলো তাদের Centroid (কেন্দ্রবিন্দু):

Formula:

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|c_A - c_B\|^2$$

প্রতিটি ধাপে অ্যালগরিদম এমন একটি জোড়া বেছে নেয়, যাদের একত্রিত করলে এই Δ মান (অর্থাৎ Cluster-ভেতরের Variance বৃদ্ধি) সবচেয়ে কম হয়। এই কারণেই Ward Linkage K-Means-এর মতোই তুলনামূলক গোলাকার ও ভারসাম্যপূর্ণ আকারের Cluster তৈরি করে, এবং এই বৈশিষ্ট্যের কারণেই এটি Scikit-Learn-এ ডিফল্ট Linkage হিসেবে নির্ধারিত হয়েছে।

8. Formula Derivation

Ward's Method-এর Δ সূত্রটি মূলত Error Sum of Squares (ESS)-এর বৃদ্ধি থেকে উদ্ভূত। কোনো একটি Cluster C -এর ESS সংজ্ঞায়িত হয় তার সদস্যদের থেকে তার নিজের Centroid c পর্যন্ত দূরত্বের বর্গের সমষ্টি হিসেবে:

$$ESS(C) = \sum_{x \in C} \|x - c\|^2$$

যখন দুটি পৃথক Cluster A এবং B (নিজস্ব ESS সহ) একত্রিত হয়ে একটি নতুন Cluster $A \cup B$ (নতুন Centroid c_{AB} সহ) তৈরি করে, তখন নতুন Cluster-এর ESS পুরনো দুটি Cluster-এর ESS-এর যোগফলের চেয়ে বেশি হয় (কারণ ডেটা এখন একটি ভিন্ন কেন্দ্রবিন্দু থেকে পরিমাপ করা হচ্ছে)। এই বৃদ্ধির পরিমাণকেই বলা হয় $\Delta(A, B)$:

$$\Delta(A, B) = ESS(A \cup B) - ESS(A) - ESS(B)$$

বীজগাণিতিকভাবে এই পার্থক্যটি সরলীকরণ করলে (Parallel Axis Theorem ব্যবহার করে) সরাসরি Centroid-ভিত্তিক সূত্রে পৌঁছানো যায়:

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|c_A - c_B\|^2$$

অর্থাৎ, Ward Linkage কার্যত প্রতিটি ধাপে এমন জোড়া বেছে নেয় যাদের একত্রীকরণে সামগ্রিক Within-Cluster Variance সবচেয়ে কম বৃদ্ধি পায় — এটি সরাসরি K-Means-এর Objective Function-এর সাথে গাণিতিকভাবে সামঞ্জস্যপূর্ণ।

9. Meaning of Every Symbol

প্রতীক	অর্থ
A, B	দুটি পৃথক Cluster
n_A, n_B	যথাক্রমে Cluster A এবং B-এর সদস্য সংখ্যা
c_A, c_B	যথাক্রমে Cluster A এবং B-এর Centroid
$\Delta(A, B)$	A ও B একত্রিত করলে Within-Cluster Variance-এর বৃদ্ধি (Ward Distance)
$ESS(C)$	Cluster C-এর Error Sum of Squares
c_{AB}	A ও B একত্রিত হওয়ার পর নতুন Cluster-এর Centroid

10. Step-by-Step Formula Explanation

Ward's Method সূত্র প্রয়োগের প্রতিটি ধাপ — Distance Matrix তৈরি, সবচেয়ে কম Δ মানের জোড়া নির্বাচন, Merge, এবং Centroid Update — এই অধ্যায়ের Section 24-এ একটি সম্পূর্ণ নতুন ডেটাসেটের ওপর সম্পূর্ণ ম্যানুয়ালি, কোনো ধাপ বাদ না দিয়ে ধারাবাহিকভাবে দেখানো হয়েছে।

11. Numerical Example

সংখ্যাসূচক উদাহরণ — সম্পূর্ণ ধাপে ধাপে প্রক্রিয়া (1D ডেটা, Single Linkage)

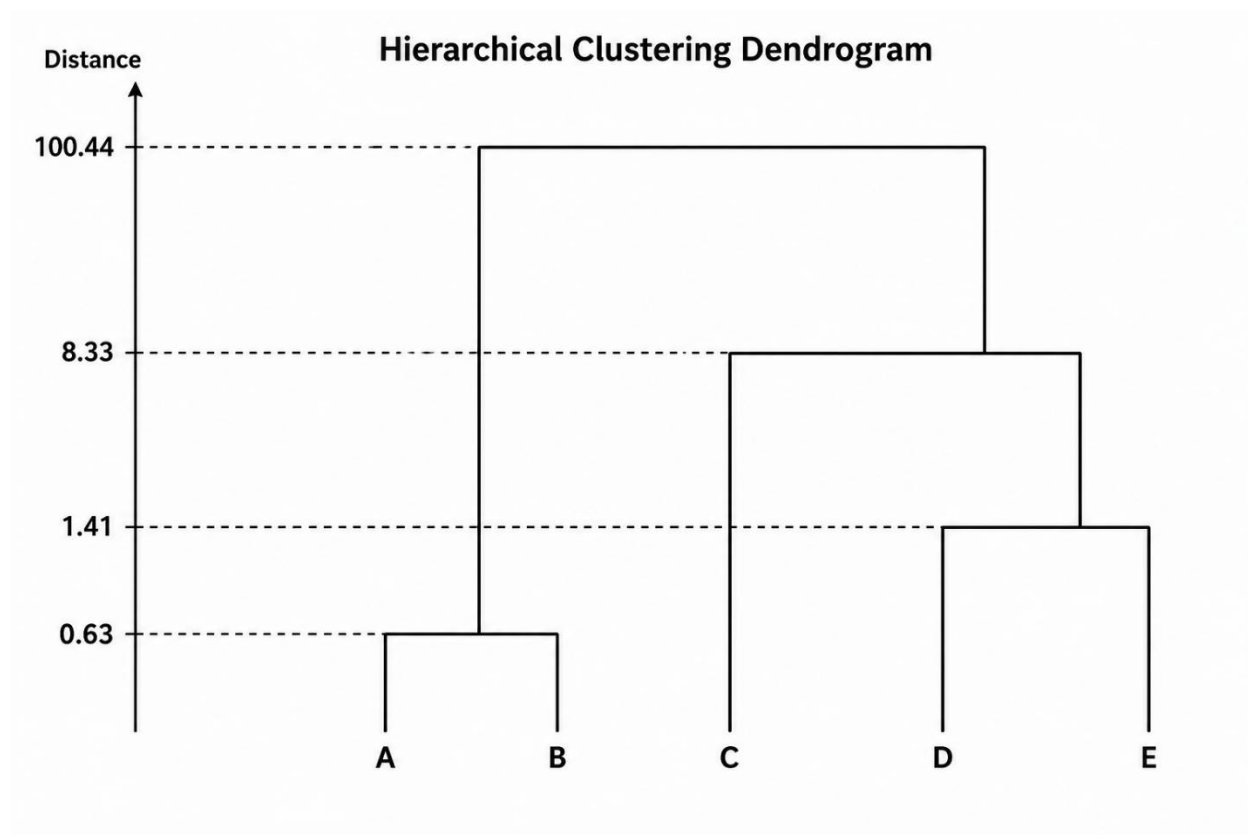
ধরা যাক, আমাদের কাছে একটি অক্ষ (1D) পাঁচটি ডেটা পয়েন্ট আছে: A=2, B=3, C=9, D=10, E=25। আমরা Single Linkage (দুই Cluster-এর সবচেয়ে কাছের দুই পয়েন্টের দূরত্ব) ব্যবহার করে ধাপে ধাপে Agglomerative Clustering করবো।

ধাপ	সবচেয়ে কাছের জোড়া	দূরত্ব	একত্রীকরণের পর অবস্থা
১	A(2) ও B(3)	1	{A,B}, {C}, {D}, {E}
২	C(9) ও D(10)	1	{A,B}, {C,D}, {E}
৩	{A,B} ও {C,D}	6 (B থেকে C-এর দূরত্ব)	{A,B,C,D}, {E}
৪	{A,B,C,D} ও {E}	15 (D থেকে E-এর দূরত্ব)	{A,B,C,D,E}

লক্ষ করুন, প্রতিটি ধাপে দূরত্ব ক্রমশ বাড়ছে (1 → 1 → 6 → 15), যা স্বাভাবিক — কারণ প্রথমে সবচেয়ে কাছাকাছি পয়েন্টগুলো একত্রিত হয়, শেষে সবচেয়ে দূরের গ্রুপগুলো একত্রিত হয়। যদি আমরা Dendrogram-কে Distance = 6-এর ঠিক ওপরে কাটি (অর্থাৎ ৬-এর বেশি দূরত্বে আর কোনো একত্রীকরণ অনুমোদন না করি), তাহলে আমরা পাবো দুইটি Cluster: {A, B, C, D} এবং {E} — যা স্বজ্ঞাতভাবেও সঠিক, কারণ E (25) বাকি চারটি পয়েন্ট থেকে অনেক দূরে অবস্থিত।

12. Visualization

Visualization — ASCII Dendrogram



উপরের Dendrogram স্পষ্টভাবে দেখাচ্ছে A,B প্রথমে একত্রিত হয়েছে সবচেয়ে কম দূরত্বে (0.63), তারপর D,E (1.41), এরপর C যোগ হয়েছে D,E-এর সাথে (8.33), এবং সবশেষে {A,B} ও {C,D,E} একত্রিত হয়ে একটি চূড়ান্ত Cluster তৈরি করেছে (100.44)। যদি আমরা Dendrogram-কে Distance ≈ 10 -এর কাছাকাছি কাটি, তাহলে দুটি স্পষ্ট Cluster পাওয়া যাবে: {A, B} এবং {C, D, E}।

13. Hyperparameters

গুরুত্বপূর্ণ হাইপারপ্যারামিটার – বিস্তারিত ব্যাখ্যা

n_clusters — Default: 2

কাজ কী: চূড়ান্তভাবে কতগুলো Cluster রাখা হবে তা নির্ধারণ করে। কখন Default রাখব: যদি আপনি আগে থেকেই জানেন যে ডেটায় ঠিক কতগুলো প্রাকৃতিক গ্রুপ থাকতে পারে (যেমন কাস্টমার সেগমেন্টেশনে ৩টি সেগমেন্ট চান), তাহলে সরাসরি সেই সংখ্যা বসিয়ে দিন। কখন পরিবর্তন করব: যদি সঠিক সংখ্যা না জানা থাকে, তাহলে n_clusters=None রেখে তার বদলে distance_threshold ব্যবহার করা ভালো — এতে Dendrogram থেকে স্বয়ংক্রিয়ভাবে উপযুক্ত সংখ্যক Cluster নির্ধারিত হয়।

linkage — Default: 'ward'

কাজ কী: দুই Cluster-এর মধ্যে দূরত্ব মাপার পদ্ধতি নির্ধারণ করে ('ward', 'complete', 'average', 'single')। কখন Default রাখব: বেশিরভাগ সাধারণ সমস্যায় 'ward' সবচেয়ে সুষম ও ব্যবহারযোগ্য ফলাফল দেয়। কখন পরিবর্তন করব: যদি আপনার ডেটায় Cluster-গুলোর আকৃতি অস্বাভাবিক বা প্রসারিত (Non-convex) হওয়ার সম্ভাবনা থাকে (যেমন চাঁদের আকৃতির ডেটা), তবে 'single' Linkage ব্যবহার করে দেখা যেতে পারে, যদিও এটি Outlier-এর প্রতি অনেক বেশি সংবেদনশীল।

distance_threshold — Default: None

কাজ কী: নির্দিষ্ট করা হলে, এই দূরত্বের বেশি হলে আর কোনো একত্রীকরণ ঘটবে না — ফলে n_clusters স্বয়ংক্রিয়ভাবে নির্ধারিত হয়ে যায়। কখন ব্যবহার করব: যখন Cluster-এর সঠিক সংখ্যা আগে থেকে জানা নেই, তখন এটি n_clusters=None-এর সাথে একত্রে ব্যবহার করে Dendrogram বিশ্লেষণের মাধ্যমে স্বাভাবিক Cluster সংখ্যা খুঁজে বের করা যায়।

Parameter	Default	সংক্ষিপ্ত কাজ
metric	'euclidean'	দূরত্ব মাপার মূল Distance Metric (মনে রাখতে হবে, 'ward' শুধুমাত্র 'euclidean'-এর সাথেই কাজ করে)।
compute_full_tree	'auto'	সম্পূর্ণ Dendrogram তৈরি করা হবে কি না তা নিয়ন্ত্রণ করে, n_clusters ছোট হলে এটি গণনা সময় বাঁচাতে পারে।
compute_distances	False	True করলে প্রতিটি একত্রীকরণের দূরত্ব সংরক্ষণ করা হয় (distances_ অ্যাট্রিবিউটে), যা Dendrogram Plot করতে প্রয়োজন হয়।

গুরুত্বপূর্ণ অ্যাট্রিবিউট

- labels_ : প্রতিটি ডেটা পয়েন্ট চূড়ান্তভাবে কোন Cluster-এ পড়েছে তার একটি অ্যারে।
- n_clusters_ : প্রকৃতপক্ষে যতগুলো Cluster তৈরি হয়েছে তার সংখ্যা।
- children_ : কোন কোন Cluster ধাপে ধাপে একত্রিত হয়েছিল তার সম্পূর্ণ ইতিহাস — এই তথ্য দিয়েই Dendrogram আঁকা হয়।
- distances_ : প্রতিটি একত্রীকরণের সময়কার দূরত্ব বা Linkage মান (compute_distances=True থাকলে)।

14. Scikit-learn Import

```
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
```

15. Python Example

From-Scratch বাস্তবায়ন (Ward Linkage, NumPy)

```
import numpy as np
import itertools

def ward_delta(nA, nB, cA, cB):
    """Ward Linkage-এর Variance বৃদ্ধি (Delta) গণনা করে"""
    return (nA * nB) / (nA + nB) * np.sum((cA - cB) ** 2)

def agglomerative_ward(X, labels):
    clusters = {
        name: {"members": [name], "size": 1, "centroid": X[i].astype(float)}
        for i, name in enumerate(labels)
    }
    history = []
    while len(clusters) > 1:
        keys = list(clusters.keys())
        best = None
        for a, b in itertools.combinations(keys, 2):
            d = ward_delta(
                clusters[a]["size"], clusters[b]["size"],
                clusters[a]["centroid"], clusters[b]["centroid"]
            )
            if best is None or d < best[0]:
                best = (d, a, b)
        d, a, b = best
        nA, nB = clusters[a]["size"], clusters[b]["size"]
        new_centroid = (nA * clusters[a]["centroid"] + nB * clusters[b]["centroid"]) / (nA + nB)
        new_name = a + b
        clusters[new_name] = {
            "members": clusters[a]["members"] + clusters[b]["members"],
            "size": nA + nB,
            "centroid": new_centroid,
        }
        history.append((a, b, d))
        del clusters[a]
        del clusters[b]
    return history

X = np.array([[1, 1], [1.5, 2], [5, 8], [8, 8], [9, 9]])
labels = ["A", "B", "C", "D", "E"]
merge_history = agglomerative_ward(X, labels)
```



```
for a, b, d in merge_history:
    print(f"Merge {a} & {b} at Ward distance {round(d, 4)}")
```

Scikit-learn ব্যবহার করে বাস্তবায়ন

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import numpy as np
```

```
X = np.array([[1, 1], [1.5, 2], [5, 8], [8, 8], [9, 9]])
```

```
model = AgglomerativeClustering(
    n_clusters=2,
    linkage="ward",
    compute_distances=True
)
model.fit(X)
```

```
print("Cluster Labels:", model.labels_)
print("Merge History (children_):", model.children_)
print("Merge Distances:", model.distances_)
```

```
# Dendrogram আঁকার জন্য SciPy Linkage matrix
Z = linkage(X, method="ward")
print("Linkage Matrix:\n", Z)
```

16. Prediction / Clustering Process

Agglomerative Clustering একটি সম্পূর্ণ Transductive পদ্ধতি — অর্থাৎ, K-Means-এর মতো এতে কোনো পৃথক predict(X) মেথড নেই যা দিয়ে সম্পূর্ণ নতুন, অদেখা কোনো Data Point-কে সরাসরি Cluster-এ ফেলা যায়। পুরো ডেটাসেটের ওপর একবারে fit_predict(X) চালিয়েই সব Cluster Label একসাথে পাওয়া যায়। নতুন কোনো Sample আসলে সাধারণত তার সবচেয়ে কাছের Cluster Centroid (যা ট্রেনিং-এর সময়কার চূড়ান্ত Merge থেকে গণনা করা যায়) খুঁজে বের করে ম্যানুয়ালি Assign করতে হয় — ঠিক যেমনটি এই অধ্যায়ের Section 24-এ দেখানো হয়েছে।

17. Advantages

- Cluster-এর সংখ্যা আগে থেকে জানার প্রয়োজন নেই — Dendrogram তৈরি হওয়ার পর যেকোনো উচ্চতায় কেটে ইচ্ছেমতো সংখ্যক Cluster বেছে নেওয়া যায়।
- সম্পূর্ণ Deterministic — K-Means-এর মতো Random Initialization না থাকায় একই ডেটায় প্রতিবার হুবহু একই ফলাফল পাওয়া যায়।

- Dendrogram-এর মাধ্যমে ডেটার সম্পূর্ণ Hierarchical কাঠামো এবং প্রতিটি ধাপে Cluster-গুলোর সম্পর্ক দৃশ্যমান হয়, যা শুধু চূড়ান্ত Label দেখানো অ্যালগরিদমের চেয়ে অনেক বেশি তথ্য দেয়।
- বিভিন্ন Linkage Criteria (Single, Complete, Average, Ward) ব্যবহার করে বিভিন্ন আকৃতি ও ঘনত্বের Cluster শনাক্ত করার নমনীয়তা রয়েছে।

18. Disadvantages

- **উচ্চ কম্পিউটেশনাল জটিলতা:** Time Complexity সাধারণত $O(n^2 \log n)$ থেকে $O(n^3)$ পর্যন্ত হতে পারে এবং Space Complexity $O(n^2)$ (সম্পূর্ণ Distance Matrix সংরক্ষণের জন্য), ফলে বড় ডেটাসেটে (যেমন লক্ষাধিক Sample) এটি ব্যবহারিকভাবে অচল হয়ে পড়ে।
- **Irreversible Merging:** একবার দুটি Cluster একত্রিত হয়ে গেলে তা আর পরবর্তীতে বিভক্ত করা যায় না (Greedy পদ্ধতি), ফলে শুরুর দিকের একটি ভুল সিদ্ধান্ত পুরো Hierarchy-তে প্রভাব ফেলে।
- **Outlier Sensitivity (Single Linkage-এ):** Single Linkage ব্যবহার করলে দুটি ভিন্ন Cluster একটিমাত্র Noise পয়েন্টের মাধ্যমে ভুলভাবে সংযুক্ত হয়ে যেতে পারে, যাকে বলা হয় Chaining Effect।
- **Distance Threshold নির্বাচন কঠিন:** কোন উচ্চতায় Dendrogram কাটা উচিত তা প্রায়ই বিষয়ভিত্তিক (Subjective) সিদ্ধান্ত, এবং ভুল Threshold অর্থহীন Cluster তৈরি করতে পারে।

19. Applications

কোন কোন সেক্টরে ব্যবহার হয়?

- জীববিজ্ঞান ও বায়োইনফরমেটিক্স: জিন এক্সপ্রেশন ডেটা বিশ্লেষণ করে জীবের বিবর্তনীয় সম্পর্ক (Phylogenetic Tree) নির্ণয় করা — এখানে Dendrogram-এর প্রাকৃতিক শ্রেণিবিন্যাস কাঠামো অত্যন্ত মূল্যবান।
- মার্কেটিং: গ্রাহকদের ক্রয় আচরণের ভিত্তিতে ধাপে ধাপে Segment তৈরি করা, যেখানে ব্যবসায়িক প্রয়োজন অনুযায়ী পরে সিদ্ধান্ত নেওয়া যায় কয়টি Segment ব্যবহার করা হবে।
- নথি ও পাঠ্য বিশ্লেষণ (Document Clustering): সংবাদ নিবন্ধ বা গবেষণাপত্রকে বিষয়ভিত্তিক Hierarchical গ্রুপে ভাগ করা।
- ইমেজ প্রসেসিং: একই রকম রঙ বা বৈশিষ্ট্যের ছবিগুলোকে ধাপে ধাপে গ্রুপবদ্ধ করা।
- Gene Expression বিশ্লেষণ করে Phylogenetic Tree (বিবর্তনীয় সম্পর্ক) তৈরি করা।
- গ্রাহক আচরণের ভিত্তিতে ধাপে ধাপে Market Segmentation করা।
- সংবাদ প্রতিবন্ধ বা গবেষণাপত্রের Topic-ভিত্তিক Hierarchical গ্রুপিং।

- ইমেজের রঙ বা টেক্সচার-ভিত্তিক Hierarchical Segmentation।
- Social Network Analysis-এ Community Detection।

20. Comparison with Previous Algorithm

K-Means বনাম Agglomerative Clustering

বিষয়	K-Means	Agglomerative Clustering
পদ্ধতি	Centroid-ভিত্তিক, Iterative (বারবার Centroid সরিয়ে অপ্টিমাইজ করা)	Hierarchical, Bottom-Up একত্রীকরণ
Cluster সংখ্যা আগে থেকে জানা লাগে?	হ্যাঁ, k নির্দিষ্ট করে দিতেই হয়	না, Dendrogram তৈরির পর প্রয়োজন অনুযায়ী সিদ্ধান্ত নেওয়া যায়
Cluster-এর আকার	সাধারণত গোলাকার (Convex)	Linkage অনুযায়ী যেকোনো আকৃতি হতে পারে (বিশেষত Single Linkage-এ)
বড় ডেটাসেটে গতি	দ্রুত, সহজেই Scalable	ধীর ($O(n^2)$ থেকে $O(n^3)$ Time Complexity)
ফলাফলের ব্যাখ্যা	শুধু চূড়ান্ত Cluster লেবেল পাওয়া যায়	Dendrogram-এর মাধ্যমে সম্পূর্ণ শ্রেণিবিন্যাস কাঠামো দেখা যায়
Randomness	Initial Centroid র্যান্ডম, ফলে প্রতিবার সামান্য ভিন্ন ফলাফল হতে পারে	সম্পূর্ণ Deterministic — একই ডেটায় সবসময় একই ফলাফল

21. Common Mistakes

- **Feature Scaling ছাড়াই মডেল ফিট করা:** Agglomerative Clustering সম্পূর্ণভাবে দূরত্ব-নির্ভর, তাই Scaling ছাড়া বড় স্কেলের Feature পুরো ফলাফল ডমিনেন্ট করে ফেলে।
- **Ward Linkage-এর সাথে ভুল Distance Metric ব্যবহার করা:** Ward Linkage গাণিতিকভাবে শুধুমাত্র Euclidean Distance-এর সাথেই সঠিকভাবে কাজ করে; অন্য Metric (যেমন Manhattan বা Cosine) ব্যবহার করলে Scikit-learn Error দেখাবে বা ফলাফল অর্থহীন হয়ে যাবে।
- **বড় ডেটাসেটে সরাসরি প্রয়োগ করা:** n অনেক বড় হলে (যেমন $n > 10,000$) পুরো $n \times n$ Distance Matrix মেমোরিতে রাখা প্রায় অসম্ভব হয়ে যায়; এক্ষেত্রে প্রথমে Sampling বা Mini-Batch পদ্ধতি ব্যবহার করা উচিত।
- **Dendrogram না দেখেই $n_{clusters}$ নির্ধারণ করা:** Dendrogram Visualize না করে সরাসরি একটি সংখ্যা বসিয়ে দিলে স্বাভাবিক গ্রুপ কাঠামো উপেক্ষিত হওয়ার ঝুঁকি থাকে।

22. Interview Questions

প্রশ্ন: Dendrogram থেকে সঠিক Cluster সংখ্যা কীভাবে নির্ধারণ করব?

উত্তর: Dendrogram-এ সবচেয়ে লম্বা উল্লম্ব রেখা (যেটি কোনো অনুভূমিক রেখা দ্বারা বাধাপ্রাপ্ত হয়নি) খুঁজে বের করে সেই জায়গায় গাছটি কাটা হয়। একটি লম্বা উল্লম্ব রেখার অর্থ হলো, দুটি Cluster একত্রিত করতে অনেক বেশি দূরত্ব অতিক্রম করতে হয়েছে — অর্থাৎ তারা স্বাভাবিকভাবেই একে অপরের থেকে অনেকটা আলাদা। এই সবচেয়ে লম্বা 'ফাঁকা' জায়গায় গাছ কাটলেই সাধারণত সবচেয়ে অর্থবহ (Meaningful) সংখ্যক Cluster পাওয়া যায়।

প্রশ্ন ২: Agglomerative Clustering-এর Time Complexity এত বেশি কেন, এবং কীভাবে তা কমানো যায়?

- **উত্তর:** প্রতিটি ধাপে অ্যালগরিদমটিকে সম্পূর্ণ Distance Matrix থেকে সবচেয়ে কম দূরত্বের জোড়া খুঁজে বের করতে হয় এবং তারপর Matrix Update করতে হয়, যা মোট $n - 1$ বার ঘটে। Naive বাস্তবায়নে এটি $O(n^3)$ পর্যন্ত সময় নিতে পারে। Priority Queue বা Nearest-Neighbor Chain Algorithm ব্যবহার করে এটি $O(n^2 \log n)$ -এ নামিয়ে আনা যায়, তবুও বড় ডেটাসেটে এটি K-Means বা DBSCAN-এর তুলনায় অনেক ধীর থেকে যায়।

প্রশ্ন ৩: Ward Linkage কেন Scikit-learn-এর Default, এবং এটি K-Means-এর সাথে কীভাবে সম্পর্কিত?

- **উত্তর:** Ward Linkage প্রতিটি ধাপে এমন জোড়া বেছে নেয় যাদের একত্রীকরণে Cluster-ভেতরের Variance (Within-Cluster Sum of Squares) সর্বনিম্ন বৃদ্ধি পায় — এটি অনেকটা K-Means-এর Objective Function-এর মতোই একই দর্শন অনুসরণ করে। ফলে Ward Linkage সাধারণত K-Means-এর মতোই তুলনামূলক গোলাকার, ভারসাম্যপূর্ণ ও কম-Noise Cluster তৈরি করে, যা বেশিরভাগ বাস্তব সমস্যায় সবচেয়ে ব্যবহারযোগ্য ফলাফল দেয়।

23. Summary

Quick Revision

- Type: Unsupervised, Hierarchical (Bottom-Up) Clustering Algorithm।
- প্রক্রিয়া: প্রতিটি পয়েন্ট নিজে একটি Cluster থেকে শুরু করে ধাপে ধাপে সবচেয়ে কাছের জোড়া একত্রিত করে একটি বড় গাছ (Dendrogram) তৈরি করা।
- Linkage: Ward (Default), Complete, Average, Single — প্রতিটি ভিন্ন আকৃতির Cluster তৈরি করে।
- সুবিধা: Cluster সংখ্যা আগে থেকে জানার প্রয়োজন নেই, Dendrogram-এর মাধ্যমে সম্পূর্ণ কাঠামো দেখা যায়, সম্পূর্ণ Deterministic।
- সীমাবদ্ধতা: বড় ডেটাসেটে ধীরগতি ($O(n^2)$ থেকে $O(n^3)$ Complexity), একবার একত্রিত হলে তা আর বিভক্ত করা যায় না (Greedy ও Irreversible)।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

এই সেকশনে Ward Linkage ব্যবহার করে একটি সম্পূর্ণ নতুন 2D ডেটাসেটের ওপর Agglomerative Clustering-এর প্রতিটি ধাপ, প্রতিটি Distance Matrix Update-সহ, হাতে-গণনা করে দেখানো হলো।

Dataset

$D = 2$ ডাইমেনশনের $N = 5$ টি Data Point:

- $A = (1,1)$
- $B = (1.5,2)$
- $C = (5,8)$
- $D = (8,8)$
- $E = (9,9)$

Initialization

শুরুতে প্রতিটি পয়েন্ট নিজে একটি স্বতন্ত্র Cluster, অর্থাৎ শুরুতে ৫টি Cluster রয়েছে: $\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$ ।

Core Formula — Ward's Method

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|c_A - c_B\|^2$$

যেখানে n_A, n_B হলো প্রতিটি Cluster-এর সদস্য সংখ্যা এবং c_A, c_B হলো তাদের Centroid। প্রতিটি ধাপে সবচেয়ে কম Δ মানের জোড়াকে একত্রিত করা হয়।

Iteration 1 — প্রাথমিক Distance Matrix

সবগুলো পয়েন্ট এখনো Singleton ($n = 1$), তাই $\Delta(A, B) = \frac{1 \times 1}{1+1} \|c_A - c_B\|^2 = 0.5 \|c_A - c_B\|^2$ ।

	A	B	C	D	E
A	0	$0.5(0.5^2 + 1^2)$ = 0.625	$0.5(4^2 + 7^2)$ = 32.5	$0.5(7^2 + 7^2)$ = 49.0	$0.5(8^2 + 8^2)$ = 64.0
B	0.625	0	$0.5(3.5^2 + 6^2)$ = 24.125	$0.5(6.5^2 + 6^2)$ = 39.125	$0.5(7.5^2 + 7^2)$ = 52.625
C	32.5	24.125	0	$0.5(3^2 + 0^2)$ = 4.5	$0.5(4^2 + 1^2)$ = 8.5
D	49.0	39.125	4.5	0	$0.5(1^2 + 1^2)$ = 1.0
E	64.0	52.625	8.5	1.0	0

সবচেয়ে কম মান: $\Delta(A, B) = 0.625$ । তাই A এবং B একত্রিত হবে।

নতুন Cluster AB-এর Centroid:

$$c_{AB} = \left(\frac{1 \times 1 + 1 \times 1.5}{2}, \frac{1 \times 1 + 1 \times 2}{2} \right) = (1.25, 1.5)$$

Iteration 2 — Distance Matrix Update

এখন Cluster-গুলো হলো: $\{AB\}$ (size=2), $\{C\}$, $\{D\}$, $\{E\}$ (প্রতিটি size=1)। AB-এর সাথে বাকিদের নতুন Δ পুনরায় Centroid ব্যবহার করে গণনা করা হলো:

$$\Delta(AB, C) = \frac{2 \times 1}{3} \|(1.25, 1.5) - (5, 8)\|^2 = \frac{2}{3} (3.75^2 + 6.5^2) = \frac{2}{3} (14.0625 + 42.25) = 37.5417$$

	C	D	E	AB
C	0	4.5	8.5	37.5417
D	4.5	0	1.0	58.5417
E	8.5	1.0	0	77.5417
AB	37.5417	58.5417	77.5417	0

সবচেয়ে কম মান: $\Delta(D, E) = 1.0$ । তাই D এবং E একত্রিত হবে।

নতুন Cluster DE-এর Centroid:

$$c_{DE} = \left(\frac{8 + 9}{2}, \frac{8 + 9}{2} \right) = (8.5, 8.5)$$

Iteration 3 — Distance Matrix Update

এখন Cluster-গুলো: $\{C\}$ (size=1), $\{AB\}$ (size=2), $\{DE\}$ (size=2)।

$$\Delta(C, DE) = \frac{1 \times 2}{3} \|(5, 8) - (8.5, 8.5)\|^2 = \frac{2}{3} (3.5^2 + 0.5^2) = \frac{2}{3} (12.25 + 0.25) = 8.3333$$

$$\Delta(AB, DE) = \frac{2 \times 2}{4} \|(1.25, 1.5) - (8.5, 8.5)\|^2 = 1 \times (7.25^2 + 7^2) = 52.5625 + 49 = 101.5625$$

	C	AB	DE
C	0	37.5417	8.3333
AB	37.5417	0	101.5625
DE	8.3333	101.5625	0

সবচেয়ে কম মান: $\Delta(C, DE) = 8.3333$ । তাই C, {D,E}-এর সাথে একত্রিত হবে।

নতুন Cluster CDE-এর Centroid:

$$c_{CDE} = \left(\frac{1 \times 5 + 2 \times 8.5}{3}, \frac{1 \times 8 + 2 \times 8.5}{3} \right) = \left(\frac{22}{3}, \frac{25}{3} \right) \approx (7.333, 8.333)$$

Iteration 4 – চূড়ান্ত Merge

এখন মাত্র দুটি Cluster বাকি আছে: $\{AB\}$ (size=2) এবং $\{CDE\}$ (size=3)।

$$\begin{aligned} \Delta(AB, CDE) &= \frac{2 \times 3}{5} \|(1.25, 1.5) - (7.333, 8.333)\|^2 = 1.2 \times (6.083^2 + 6.833^2) \\ &= 1.2 \times (37.0 + 46.69) \approx 100.44 \end{aligned}$$

শুধুমাত্র একটি জোড়া বাকি থাকায় এটিই একত্রিত হবে, এবং সব ডেটা এখন একটি একক Cluster $\{A, B, C, D, E\}$ -এ পরিণত হয়েছে। এখানেই অ্যালগরিদম থেমে যায় কারণ **Stopping Condition** পূরণ হয়েছে — মাত্র একটি Cluster অবশিষ্ট আছে।

Merge Progress Table

Iteration	Merged Clusters	Ward Distance (Δ)	নতুন Cluster Size
1	A, B	0.625	2
2	D, E	1.000	2
3	C, {D,E}	8.333	3
4	{A,B}, {C,D,E}	100.442	5

Final Cluster Assignment ($n_clusters = 2$ ধরলে)

Dendrogram-কে Iteration 3 এবং Iteration 4-এর মাঝামাঝি উচ্চতায় (Distance ≈ 10 থেকে 100-এর মধ্যে যেকোনো জায়গায়) কাটলে ঠিক দুটি Cluster পাওয়া যায়:

Cluster	অন্তর্ভুক্ত পয়েন্ট
Cluster 1	A(1,1), B(1.5,2)
Cluster 2	C(5,8), D(8,8), E(9,9)

Interpretation

লক্ষণীয় যে প্রতিটি ধাপে Merge Distance ক্রমাগত বৃদ্ধি পেয়েছে (0.625 $\rightarrow$ 1.0 $\rightarrow$ 8.333 $\rightarrow$ 100.442), যা Ward Linkage-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য — সবচেয়ে কাছাকাছি ও সবচেয়ে সাদৃশ্যপূর্ণ পয়েন্টগুলো আগে একত্রিত হয়, আর সবচেয়ে ভিন্ন গ্রুপগুলো (এখানে নিচে-বামের $\{A, B\}$ এবং ওপরে-ডানের $\{C, D, E\}$) সবার শেষে একত্রিত হয়। Iteration 3 থেকে Iteration 4-এ Merge Distance-এর হঠাৎ বিশাল লাফ (8.333 $\rightarrow$ 100.442) নির্দেশ করে যে এই দুটি গ্রুপ প্রকৃতপক্ষে সম্পূর্ণ পৃথক প্রাকৃতিক Cluster, যা Dendrogram থেকে সঠিক $n_clusters$ নির্বাচনের একটি বাস্তব উদাহরণ।

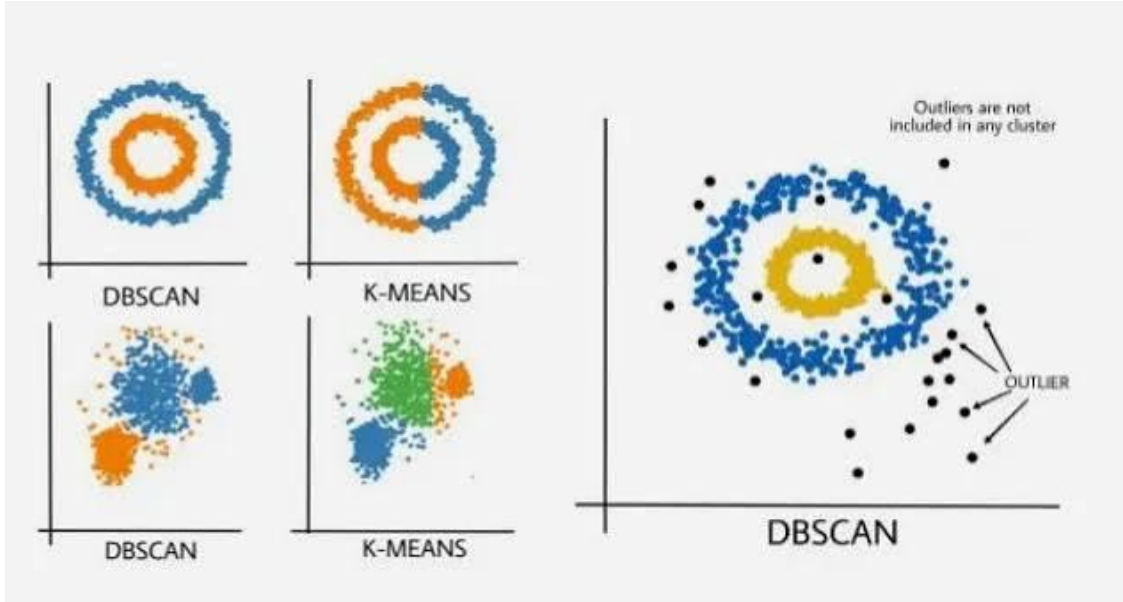
Chapter 16 — DBSCAN Clustering

1. Introduction

DBSCAN-এর পূর্ণরূপ হলো Density-Based Spatial Clustering of Applications with Noise।

একটি Density-Based বা ঘনত্ব-ভিত্তিক আনসুপারভাইজড মেশিনলার্নিং অ্যালগরিদম। K-Means-এর মতো এটি বৃত্তাকার ক্লাস্টার তৈরি করে না বা আগে থেকে ক্লাস্টারের সংখ্যা (K) জানতে চায় না। বরং, ডেটার ঘনত্ব (Density) মেপে এটি স্বয়ংক্রিয়ভাবে ক্লাস্টার তৈরি করে।

- **Density-Based (DB):** এটি ডেটার উচ্চ-ঘনত্ব বিশিষ্ট এলাকাগুলোকে ক্লাস্টার হিসেবে চিহ্নিত করে।
- **Spatial (S):** এটি বহুমাত্রিক স্পেসে ডেটা পয়েন্টগুলোর পারস্পরিক দূরত্বের ওপর ভিত্তি করে কাজ করে।
- **Clustering (C):** সমধর্মী ডেটাগুলোকে একসাথে গ্রুপ বা দলভুক্ত করে।



Applications with Noise (AN): বাস্তব জীবনের অ্যাপ্লিকেশনে ডেটার মধ্যে যে নয়েজ (Noise) বা অপ্রয়োজনীয় Outlier থাকে, এটি সেগুলোকে ক্লাস্টারে জোর করে না ঢুকিয়ে আলাদাভাবে সনাক্ত করতে পারে।

DBSCAN মূলত ২টি প্যারামিটারের সাহায্যে ডেটার ঘনত্ব পরিমাপ করে:

১. **Epsilon (ϵ):** একটি নির্দিষ্ট ডেটা পয়েন্টের চারপাশের খোঁজাখুঁজির সর্বোচ্চ ব্যাসার্ধ বা Radius।
২. **MinPts (Minimum Points):** একটি ক্লাস্টার শুরু করার জন্য ওই ব্যাসার্ধের ভেতর ন্যূনতম কয়টি ডেটা পয়েন্ট থাকতে হবে তার সংখ্যা।

2. Why This Algorithm Was Developed

Why We Need DBSCAN Instead of K-Means

বাস্তব জীবনের জটিল ডেটাসেটে K-Means Clustering-এর মূল সীমাবদ্ধতাগুলো দূর করার জন্যই DBSCAN-এর প্রয়োজন হয়। নিচে প্রধান কারণগুলো আলোচনা করা হলো:

১. জটিল এবং আঁকাবাঁকা আকৃতির ক্লাস্টার হ্যান্ডলিং (Arbitrary Shaped Clusters)

- **K-Means-এর সমস্যা:** K-Means দূরত্বের গাণিতিক গড় (Mean) ব্যবহার করে বলে এটি সবসময় ধরে নেয় ক্লাস্টারগুলো বৃত্তাকার বা গোলকাকার (Spherical Clusters)। ডেটা যদি অর্ধচন্দ্রাকৃতি, কনসেন্ট্রিক বৃত্ত (একটির ভেতর আরেকটি রিং) বা আঁকাবাঁকা লাইনের মতো হয়, তবে K-Means সেখানে ভুল ক্লাস্টারিং করে।
- **DBSCAN-এর সুবিধা:** যেহেতু DBSCAN দূরত্বের চেয়ে ডেটার ঘনত্বের (Density) ওপর ভিত্তি করে কাজ করে, তাই ডেটার আকৃতি যত জটিল বা আঁকাবাঁকা হোক না কেন, এটি খুব সহজেই আসল প্যাটার্ন সনাক্ত করতে পারে।

২. আউটলায়ার এবং নয়েজ থেকে মুক্তি (Robustness to Noise & Outliers)

- **K-Means-এর সমস্যা:** K-Means-এর ডিসিশন বাউন্ডারি অত্যন্ত সংবেদনশীল। এটি ডেটাসেটে থাকা প্রতিটি সিঙ্গেল আউটলায়ার বা নয়েজ পয়েন্টকে জোর করে কোনো না কোনো ক্লাস্টারে ঢুকিয়ে নেয়। এর ফলে ক্লাস্টারের সেন্ট্রয়েড বা কেন্দ্রবিন্দুটি আসল অবস্থান থেকে বিচ্যুত (Drift) হয়ে যায়।
- **DBSCAN-এর সুবিধা:** DBSCAN-এর নামের ভেতরেই 'Noise' শব্দটি আছে। এটি কম ঘনত্বের এলাকার বিচ্ছিন্ন পয়েন্টগুলোকে সরাসরি Noise বা আউটলায়ার হিসেবে ফ্ল্যাগ (Flag) করে রেখে দেয় এবং ক্লাস্টারের ভেতরে জায়গা দেয় না। ফলে আসল ক্লাস্টারগুলো একদম নিখুঁত থাকে।

৩. K-এর মান আগে থেকে জানার ঝামেলা নেই (No Need to Specify 'K')

- **K-Means-এর সমস্যা:** K-Means রান করার একদম শুরুতে ইউজারকে $n_clusters$ বা K -এর মান নির্দিষ্ট করে দিতে হয়। বাস্তব জীবনের অজানা বড় ডেটাসেটে কয়টি গ্রুপ আছে তা আগে থেকে অনুমান করা অত্যন্ত কঠিন এবং ভুল K সিলেক্ট করলে পুরো আউটপুট নষ্ট হয়ে যায়।
- **DBSCAN-এর সুবিধা:** DBSCAN-এ আগে থেকে কয়টি ক্লাস্টার তৈরি হবে তা বলে দিতে হয় না। এটি ডেটার ডেনসিটি বা ঘনত্ব বিশ্লেষণ করে স্বয়ংক্রিয়ভাবে নিজেই খুঁজে বের করে যে সেখানে ঠিক কয়টি ক্লাস্টার হওয়া উচিত।

৪. সেন্ট্রয়েড ইনিশিয়ালাইজেশনের ওপর নির্ভরশীল নয় (No Initial Centroid Dependency)

- **K-Means-এর সমস্যা:** K-Means-এর পারফরম্যান্স শুরুতে র্যান্ডমভাবে সিলেক্ট করা সেন্ট্রয়েডের ওপর নির্ভর করে। ইনিশিয়াল সেন্ট্রয়েড সিলেকশন খারাপ হলে মডেলটি লোকাল মিনিমামে (Local Minima) আটকে যায় এবং প্রতিবার রান করলে ভিন্ন ভিন্ন ফলাফল দিতে পারে।
- **DBSCAN-এর সুবিধা:** DBSCAN-এ কোনো সেন্ট্রয়েড বা কেন্দ্রবিন্দুর ধারণা নেই। এটি ডেটা পয়েন্টগুলোর ভেতরের ডেনসিটি লিংক ধরে অগ্রসর হয়, তাই এটি সবসময় স্থিতিশীল (Stable) এবং ডিটারমিনিস্টিক ফলাফল দেয়।

3. Real-life Motivation

কল্পনা করা যাক একটি শহরের ভূমিকম্প-পরবর্তী স্যাটেলাইট ডেটায় ক্ষতিগ্রস্ত ভবনগুলোর অবস্থান চিহ্নিত করতে হবে। ক্ষতিগ্রস্ত এলাকাগুলো কখনোই নিখুঁত বৃত্তাকার হয় না — এগুলো রাস্তার ধার ঘেঁষে আঁকাবাঁকা প্যাটার্নে ছড়িয়ে থাকতে পারে, এবং কিছু বিচ্ছিন্ন ভবনও (Outlier) থাকতে পারে যেগুলো কোনো গ্রুপের অংশ নয়। K-Means এখানে ব্যর্থ হবে কারণ এটি জোর করে প্রতিটি বিন্দুকে একটি বৃত্তাকার Cluster-এ ফেলে দেবে। ঠিক এই বাস্তব প্রয়োজন থেকেই DBSCAN-এর উদ্ভব — এটি ঘনত্ব অনুসরণ করে যেকোনো আকৃতির প্রকৃত Cluster খুঁজে বের করে এবং বিচ্ছিন্ন বিন্দুগুলোকে সততার সাথে Noise হিসেবে আলাদা রাখে। একইভাবে ক্রেডিট কার্ড Fraud Detection-এ স্বাভাবিক লেনদেনের ঘন Cluster থেকে দূরে থাকা অস্বাভাবিক লেনদেন সনাক্ত করতেও DBSCAN ব্যাপকভাবে ব্যবহৃত হয়।

4. Intuition

DBSCAN-এর স্বজ্ঞা বোঝার জন্য একটি জনাকীর্ণ শহরের রাস্তায় মানুষের ভিড়ের কথা ভাবা যাক। যেখানে মানুষ ঘন হয়ে দাঁড়িয়ে আছে (একটি নির্দিষ্ট ব্যাসার্ধের ভেতর পর্যাপ্ত সংখ্যক মানুষ), সেটি একটি “ভিড়” বা Cluster হিসেবে গণ্য হবে। যে মানুষ ভিড়ের একদম কেন্দ্রে আছে সে “Core” ব্যক্তি, যে ভিড়ের একদম কিনারায় দাঁড়িয়ে আছে কিন্তু ভিড়ের সংস্পর্শে আছে সে “Border” ব্যক্তি, আর যে সম্পূর্ণ একা রাস্তার অন্য প্রান্তে দাঁড়িয়ে আছে, কোনো ভিড়ের সংস্পর্শেই নেই, সে “Noise” বা বিচ্ছিন্ন ব্যক্তি। DBSCAN ঠিক এই যুক্তি অনুসরণ করেই একটি Core বিন্দু থেকে শুরু করে তার প্রতিবেশী, প্রতিবেশীর প্রতিবেশী — এভাবে ঘনত্বের শিকল ধরে ধরে পুরো Cluster আবিষ্কার করে।

5. Working Principle

ডিবিএসক্যান অ্যালগরিদমের অভ্যন্তরীণ কার্যপ্রক্রিয়া (Full Backend Workflow)

সুপারভাইজড লার্নিং বা সেন্ট্রয়েড-ভিত্তিক ক্লাস্টারিংয়ের মতো DBSCAN কোনো লুপ বা গাণিতিক সমীকরণের মাধ্যমে কেন্দ্রবিন্দু খোঁজার চেষ্টা করে না। এটি সম্পূর্ণ ডেটাসেটের ঘনত্ব এবং বিন্দুগুলোর মধ্যকার জ্যামিতিক লিঙ্কিং বিশ্লেষণ করে ধাপে ধাপে অগ্রসর হয়।

ব্যাকএন্ডে পুরো প্রসেসটি নিচের ধাপগুলোর মাধ্যমে সম্পন্ন হয়:

ধাপ ১: উপাত্ত বিন্দু নির্বাচন এবং লেবেলহীন ঘোষণা (Initialization)

অ্যালগরিদমের শুরুতেই ডেটাসেটের প্রতিটি বিন্দুকে 'Unvisited' বা লেবেলহীন হিসেবে চিহ্নিত করা হয়। এরপর সম্পূর্ণ উপাত্ত ক্ষেত্র থেকে র্যান্ডমলি (এলোমেলোভাবে) যেকোনো একটি প্রাথমিক বিন্দু P বেছে নেওয়া হয়।

ধাপ ২: এলাকার ঘনত্ব পরিমাপ (Neighborhood Query)

নির্বাচিত বিন্দু P -কে কেন্দ্র করে আগে থেকে নির্ধারিত ব্যাসার্ধ ϵ (Epsilon) অনুযায়ী একটি জ্যামিতিক বৃত্ত বা এলাকা কল্পনা করা হয়। এই সীমানার ভেতর মোট কতটি প্রতিবেশী বিন্দু (Neighborhood Points) অবস্থান করছে তা গণনা করা হয়।

ধাপ ৩: বিন্দুর শ্রেণীবিভাগ এবং ক্লাস্টার সূচনা (Core Point Check)

- যদি বৃত্তের ভেতর নিজের সহ বিন্দুর সংখ্যা $MinPts$ -এর চেয়ে কম হয়, তবে সাময়িকভাবে P -কে Noise বা আউটলায়ার হিসেবে লেবেল করা হয় এবং অ্যালগরিদমটি পুনরায় ধাপ ১-এ ফিরে গিয়ে নতুন একটি আনভিজিটেড বিন্দু বেছে নেয়।
- আর যদি বিন্দুর সংখ্যা $MinPts$ বা তার বেশি হয়, তবে P -কে একটি Core Point ঘোষণা করা হয় এবং এই বিন্দুটিকে কেন্দ্র করে একটি নতুন ক্লাস্টার (Cluster) গঠন করার প্রক্রিয়া শুরু হয়।

ধাপ ৪: চেইন রিয়্যাকশন ও ক্লাস্টার সম্প্রসারণ (Density-Reachable Expansion)

যখন একটি নতুন ক্লাস্টার তৈরি হয়, তখন P -এর ϵ ব্যাসার্ধের ভেতরে থাকা সমস্ত প্রতিবেশী বিন্দুকে একটি অস্থায়ী 'Queue' বা কার্য তালিকায় যুক্ত করা হয়। এরপর এই তালিকা ধরে চেইন রিয়্যাকশন শুরু হয়:

- তালিকা থেকে প্রতিটি বিন্দুকে একে একে তুলে আনা হয় এবং তার চারপাশে আবার ϵ ব্যাসার্ধের এলাকা পরীক্ষা করা হয়।
- যদি সেই নতুন বিন্দুটিও একটি Core Point হিসেবে প্রমাণিত হয়, তবে তার নিজস্ব প্রতিবেশীদেরও এই ক্লাস্টারের কার্য তালিকায় (Queue) যুক্ত করা হয়। এভাবে ঘনত্বের সেতু তৈরি করে ক্লাস্টারটি চারদিকে ছড়াতে থাকে।
- যদি কোনো বিন্দু Core Point না হয়ে অন্য কোনো Core Point-এর সীমানায় থাকে, তবে তাকে Border Point হিসেবে ক্লাস্টারে অন্তর্ভুক্ত করা হয়, কিন্তু তার চারপাশে নতুন করে আর কোনো প্রতিবেশীর খোঁজ করা হয় না (চেইন রিয়্যাকশন এখানে থেমে যায়)।

ধাপ ৫: ক্লাস্টার সমাপ্তি (Convergence of a Single Cluster)

যখন বর্তমান কার্য তালিকার (Queue) সমস্ত বিন্দুর খোঁজাখুঁজি শেষ হয়ে যায় এবং ঘনত্ব-সংযুক্ত (Density-Connected) হওয়ার মতো নতুন কোনো বিন্দু আর অবশিষ্ট থাকে না, তখন অ্যালগরিদমটি বর্তমান ক্লাস্টারটির বৃদ্ধি বন্ধ করে দেয় এবং সেটিকে একটি সম্পূর্ণ ক্লাস্টার হিসেবে মেমোরিতে লক করে ফেলে।

ধাপ ৬: পুনরাবৃত্তি এবং চূড়ান্ত নয়েজ নির্ধারণ (Iteration & Final Output)

এরপর অ্যালগরিদমটি ডেটাসেটের অবশিষ্ট 'Unvisited' বিন্দুগুলোর দিকে নজর দেয় এবং **ধাপ ১ থেকে ৫** পর্যন্ত পুরো প্রক্রিয়াটি বারবার পুনরাবৃত্তি করতে থাকে।

- এই প্রক্রিয়ায় যদি কোনো বিন্দু পূর্বে 'Noise' হিসেবে লেবেল হয়ে থাকে, কিন্তু পরবর্তীতে সম্প্রসারণের সময় সেটি কোনো ক্লাস্টারের Border Point হিসেবে ধরা পড়ে, তবে তার আগের ভুল লেবেলটি মুছে তাকে ওই ক্লাস্টারের অংশ বানিয়ে দেওয়া হয়।
- সম্পূর্ণ ডেটাসেটের প্রতিটি বিন্দু ভিজিট করা শেষ হলে, যে বিন্দুগুলো কোনো ক্লাস্টারের Core বা Border Point হতে পারেনি, সেগুলোকে চূড়ান্তভাবে **Noise / Outlier (Label: -1)** ঘোষণা করে মেমোরি থেকে আলাদা করে দেওয়া হয় এবং ট্রেনিং প্রক্রিয়া সমাপ্ত হয়।

6. Mathematical Backend

Core Parameters of DBSCAN & Handling Sparse Density

DBSCAN অ্যালগরিদমের পুরো মেকানিজম এবং ক্লাস্টার গঠনের নিখুঁততা সম্পূর্ণভাবে দুটি হাইপারপ্যারামিটারের ওপর নির্ভর করে। নিচে এদের গাণিতিক ও ব্যবহারিক রূপ আলোচনা করা হলো:

১ . Epsilon (ϵ / Eps)

- **প্যারামিটারটি কী কাজ করে:** এটি হলো একটি নির্দিষ্ট ডেটা পয়েন্টের চারপাশে প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা Radius। সহজ ভাষায়, কোনো পয়েন্টের চারপাশের কতটুকু দূরত্বকে তার "Neighborhood" বা এলাকা ধরা হবে, তা ϵ নির্ধারণ করে।
- **প্রভাব (Effect of Tuning):**
 - ϵ **খুব ছোট হলে (Small ϵ):** খোঁজাখুঁজির এলাকা ছোট হয়ে যায়। ফলে ডেটা পয়েন্টগুলো একে অপরের খুব কাছে থাকা সত্ত্বেও প্রয়োজনীয় ঘনত্ব না পেয়ে বিচ্ছিন্ন হয়ে যায়। এতে অনেক আসল ক্লাস্টার তৈরি হতে পারে না এবং ডেটার একটি বড় অংশ Noise হিসেবে বাদ পড়ে যায় (Underfitting/High Bias)।
 - ϵ **খুব বড় হলে (Large ϵ):** এলাকা অনেক বড় হয়ে যায়। এর ফলে দুটি আলাদা ও ভিন্ন ভিন্ন ক্লাস্টারও একটি বড় ব্যাসার্ধের কারণে জোড়া লেগে একটি একক ক্লাস্টার হয়ে যেতে পারে (Overfitting/High Variance)।

২ . Minimum Points (MinPts)

- **প্যারামিটারটি কী কাজ করে:** এটি নির্ধারণ করে যে, ϵ ব্যাসার্ধের ভেতরের এলাকায় (Neighborhood) নিজেসহ নূন্যতম কতটি ডেটা পয়েন্ট উপস্থিত থাকলে তবেই একটি নতুন ক্লাস্টার বা ঘন অঞ্চল (Dense Region) গঠন করা শুরু হবে।
- **প্রভাব (Effect of Tuning):**
 - **MinPts খুব ছোট হলে (যেমন: $\text{MinPts} \leq 2$):** মডেলটি অত্যন্ত নমনীয় হয়ে যায়। এর ফলে সামান্য ৩-৪টি নয়েজ বা আউটলায়ার পয়েন্টও কাছাকাছি থাকলে তারা নিজেদের একটি স্বতন্ত্র ক্লাস্টার বানিয়ে ফেলে।
 - **MinPts খুব বড় হলে:** মডেলটি কঠোর বা Strict হয়ে যায়। তখন শুধুমাত্র চরম ঘন (Extremely Dense) অঞ্চল ছাড়া অন্য কোথাও ক্লাস্টার তৈরি হতে পারে না, এবং মাঝারি ঘনত্বের এলাকাগুলোও নয়েজ হিসেবে বাদ পড়ে যায়।
- **Rule of Thumb:** সাধারণ ডেটাসেটের ক্ষেত্রে $\text{MinPts} = 2 \times \text{Dimension}$ সূত্রটি ব্যবহার করা হয়। ডেটাতে নয়েজ বেশি থাকলে MinPts-এর মান আরও বাড়িয়ে দেওয়া উচিত।

Data Point Classifications in DBSCAN

DBSCAN অ্যালগরিদমে `model.fit()` হওয়ার পর, এটি ইনপুট করা ϵ (Epsilon) এবং *MinPts* (Minimum Points)-এর ওপর ভিত্তি করে ডেটাসেটের প্রতিটি ডেটা পয়েন্টকে ৩টি সুনির্দিষ্ট ভাগে ক্লাসিফাই করে।

১ . Core Points (মূল বিন্দু)

- **সংজ্ঞা (Definition):** কোনো একটি ডেটা পয়েন্টের চারপাশে যদি ϵ ব্যাসার্ধের (Epsilon Radius) একটি বৃত্ত আঁকা হয় এবং সেই বৃত্তের ভেতর নিজেসহ নূন্যতম $MinPts$ সংখ্যক ডেটা পয়েন্ট থাকে, তবে তাকে **Core Point** বলা হয়।

- **গাণিতিক শর্ত (Mathematical Condition):**

$$\text{Neighborhood Points}(P) \geq MinPts$$

- এই পয়েন্টগুলো ক্লাস্টারের একদম কেন্দ্র বা ভেতরের ঘন অঞ্চল (Dense Interior) গঠন করে। একটি ক্লাস্টারে একাধিক বা শত শত Core Point থাকতে পারে এবং তারা একে অপরের সাথে ঘনত্বের শিকল দিয়ে যুক্ত থাকে।

২ . Border Points (সীমান্ত বিন্দু)

- **সংজ্ঞা (Definition):** একটি ডেটা পয়েন্টের নিজস্ব ϵ ব্যাসার্ধের বৃত্তের ভেতর যদি $MinPts$ -এর চেয়ে কম সংখ্যক পয়েন্ট থাকে, কিন্তু পয়েন্টটি যদি অন্য কোনো একটি Core Point-এর এলাকার ভেতরে অবস্থান করে, তবে তাকে **Border Point** বলা হয়।

- **গাণিতিক শর্ত (Mathematical Condition):**

$$\text{Neighborhood Points}(P) < MinPts \quad \text{AND} \quad P \in \text{Neighborhood of a Core Point}$$

- এরা নিজেরা কোনো ক্লাস্টার শুরু করার মতো যথেষ্ট ঘন নয়, কিন্তু কোনো একটি শক্তিশালী Core Point-এর প্রতিবেশী হওয়ায় এরা ক্লাস্টারের অংশ হিসেবে গণ্য হয়। এই পয়েন্টগুলো মূলত একটি ক্লাস্টারের একদম বাইরের সীমানা বা ডালপালা (Outer Edges) তৈরি করে।

৩ . Noise Points / Outliers (নয়েজ বা বিচ্ছিন্ন বিন্দু)

- **সংজ্ঞা (Definition):** যে ডেটা পয়েন্টটি নিজে কোনো Core Point-ও নয়, আবার অন্য কোনো Core Point-এর সীমানার বা এলাকার ভেতরেও পড়ে না (অর্থাৎ সে কোনো Core Point-এর প্রতিবেশী নয়), তাকে **Noise Point** বলা হয়।

- **গাণিতিক শর্ত (Mathematical Condition):**

$$\text{Neighborhood Points}(P) < MinPts \quad \text{AND} \quad P \notin \text{Neighborhood of any Core Point}$$

- **নোটে রাখার মতো বৈশিষ্ট্য (Key Concept):** এই পয়েন্টগুলো ডেটাসেটের অত্যন্ত কম ঘনত্বের (Sparse Density) বা ছড়ানো-ছিটানো এলাকায় একা অবস্থান করে। DBSCAN-এর সবচেয়ে চমৎকার ব্যাপার হলো, এটি এই Noise Point গুলোকে কোনো ক্লাস্টারে জোর করে জায়গা না দিয়ে সরাসরি -1 লেবেল দিয়ে আউটলায়ার হিসেবে আলাদা করে দেয়।

ধরা যাক, আমরা প্যারামিটার সেট করেছি: $\epsilon = 2 \text{ cm}$ এবং $MinPts = 4$ । এবার নিচের ৩টি পয়েন্টের আচরণ লক্ষ্য করি:

- **Point A:** তার ২ সেমি ব্যাসার্ধের বৃত্তের ভেতর মোট ৫টি পয়েন্ট আছে। যেহেতু $5 \geq 4$, তাই Point A একটি Core Point।

- **Point B:** তার ২ সেমি ব্যাসার্ধের বৃত্তের ভেতর মাত্র ৩টি পয়েন্ট আছে ($3 < 4$, তাই সে Core হতে পারল না)। কিন্তু, সে উপরের **Point A (Core Point)**-এর বৃত্তের ভেতরে বসা আছে। সুতরাং, **Point B একটি Border Point**।
- **Point C:** তার ২ সেমি বৃত্তের ভেতর মাত্র ১টি পয়েন্ট আছে (সে নিজে)। এবং আশেপাশে ২ সেমির মধ্যে কোনো Core Point-ই নেই। সুতরাং, **Point C একটি Noise Point**।

ঘনত্ব-সংযুক্ত বিন্দু এবং ক্লাস্টার গঠনের মেকানিজম (Density-Connected Points & Cluster Formation)

মেশিন লার্নিং শাস্ত্রে DBSCAN অ্যালগরিদমের ব্যাকগ্রাউন্ড ওয়ার্কফ্লো এবং জ্যামিতিক ডিসিশন বাউন্ডারি বোঝার জন্য সবচেয়ে গুরুত্বপূর্ণ মেকানিজম হলো বিন্দুগুলোর পারস্পরিক ঘনত্ব-গম্যতা (Density-Reachability) এবং ঘনত্ব-সংযুক্ততা (Density-Connectedness)। বাস্তব জীবনের জটিল উপাত্ত ক্ষেত্রে দুটি অত্যন্ত দূরবর্তী বিন্দু সরাসরি একে অপরের সীমানার মধ্যে না থাকা সত্ত্বেও কীভাবে একই ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত হয়, তা মূলত একটি অবিচ্ছিন্ন গাণিতিক শিকল বা চেইনের মাধ্যমে নির্ধারিত হয়।

নিচে এই মেকানিজমের গাঠনিক উপাদান এবং গাণিতিক নীতি বিস্তারিতভাবে আলোচনা করা হলো:

১. সরাসরি ঘনত্ব-গম্যতা (Directly Density-Reachable)

কোনো একটি উপাত্ত বিন্দু P যদি অন্য একটি মূল বিন্দু বা Core Point Q -এর ϵ ব্যাসার্ধের (Epsilon Neighborhood) ভেতরে সরাসরি অবস্থান করে, তবে গাণিতিক ভাষায় বলা হয় যে, P বিন্দুটি Q থেকে সরাসরি ঘনত্ব-গম্য বা Directly Density-Reachable।

- **নীতি:** এই শর্তটি পূরণের জন্য একটি বিন্দুকে অবশ্যই Core Point হতে হবে, যার ওপর অন্য বিন্দুটি নির্ভরশীল। তবে নির্ভরশীল বিন্দুটি নিজে Core Point বা Border Point যেকোনো একটি হলেই চলে।
- **কাঠামোগত উদাহরণ:** ধরা যাক, B একটি Core Point এবং A তার সীমানার ভেতরে থাকা একটি সীমান্ত বিন্দু বা Border Point। এই জ্যামিতিক বিন্যাসে A বিন্দুটি B থেকে সরাসরি ঘনত্ব-গম্য (Directly Density-Reachable)।

২. ধারাবাহিক ঘনত্ব-গম্যতা (Density-Reachable)

যদি কোনো একটি বিন্দু P থেকে অন্য একটি দূরবর্তী বিন্দু Q -তে পৌঁছানোর জন্য মাঝখানে একঝাঁক মূল বিন্দুর একটি ধারাবাহিক শিকল (Chain of Core Points) বিদ্যমান থাকে, যেখানে প্রতি জোড়া পরপর বিন্দু একে অপরের থেকে Directly Density-Reachable, তবে তাকে ধারাবাহিক ঘনত্ব-গম্য বা Density-Reachable বলা হয়।

- **নীতি:** এটি মূলত একটি ট্রানজিটিভ প্রোপারটি (Transitive Property)। যদি X থেকে Y -তে যাওয়া যায় এবং Y থেকে Z -এ যাওয়া যায়, তবে পরোক্ষভাবে X থেকে Z -এ পৌঁছানো সম্ভব।

- **কাঠামোগত উদাহরণ:** ধরা যাক, B, C , এবং D হলো তিনটি স্বতন্ত্র মূল বিন্দু বা Core Points, যারা পরস্পরের ব্যাসার্ধের সাথে ওভারল্যাপ করে একটি চেইন গঠন করেছে ($B \leftrightarrow C \leftrightarrow D$)। যেহেতু B থেকে C এবং C থেকে D -তে ক্রমান্বয়ে পৌঁছানো যাচ্ছে, তাই দূরবর্তী হওয়া সত্ত্বেও B এবং D একে অপরের সাপেক্ষে ঘনত্ব-গম্য (Density-Reachable)।

৩. ঘনত্ব-সংযুক্ততা (Density-Connected)

এটিই ক্লাস্টার গঠনের চূড়ান্ত গাণিতিক শর্ত। যদি উপাত্ত ক্ষেত্রের দুটি বিন্দু A এবং F একে অপরের সাথে সরাসরি যুক্ত না থাকে, এমনকি তারা যদি একে অপরের থেকে মাইলের পর মাইল দূরে অবস্থান করে, তবুও তারা যদি অন্য কোনো একটি সাধারণ মূল বিন্দু চেইনের (Common Core Point Chain) সাথে যুক্ত বা Density-Reachable থাকে, তবে গাণিতিকভাবে A এবং F -কে ঘনত্ব-সংযুক্ত বা Density-Connected বলা হয়।

DBSCAN-এর স্বর্ণসূত্র (The Golden Rule of DBSCAN):

উপাত্ত সেটের যে বিন্দুগুলো একে অপরের সাথে জ্যামিতিকভাবে ঘনত্ব-সংযুক্ত (Density-Connected) অবস্থায় থাকে, অ্যালগরিদমটি তাদের সবাইকে একটি একক ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত করে।

The Workflow Chain

DBSCAN-এর ব্যাকএন্ডে পুরো প্রসেসটি নিচের চেইনের মাধ্যমে ধাপে ধাপে সম্পন্ন হয়:

$$A \text{ (Border)} \leftrightarrow B \text{ (Core)} \leftrightarrow C \text{ (Core)} \leftrightarrow D \text{ (Core)} \leftrightarrow F \text{ (Border)}$$

১. শুরুতে B নামক Core Point-এর খোঁজাখুঁজি প্রক্রিয়ায় তার Border Point হিসেবে A -কে সনাক্ত করা হয়।
২. এরপর B -এর ঘনত্ব বা লিঙ্কিং ধরে অ্যালগরিদমটি সামনে অগ্রসর হয়ে পরবর্তী Core Point C -কে খুঁজে পায়।
৩. C -এর ব্যাসার্ধ ধরে চেইনটি আরও বিস্তৃত হয়ে D নামক আরেকটি Core Point-এর সাথে যুক্ত হয়।
৪. চূড়ান্ত ধাপে D তার নিজস্ব সীমানার শেষ প্রান্তে থাকা Border Point F -কে ক্লাস্টারে টেনে নেয়।

চূড়ান্ত সিদ্ধান্ত:

এই জ্যামিতিক বিন্যাসে A এবং D সরাসরি কানেক্টেড নয়, কিংবা A এবং F -এর মধ্যে কোনো সরাসরি যোগাযোগ নেই। কিন্তু ব্যাকএন্ডে $B \rightarrow C \rightarrow D$ নামক কোর বিন্দুগুলোর একটি অবিচ্ছিন্ন ডেনসিটি সেতু থাকার কারণে পুরো শিকলটি একসাথে লক হয়ে যায়। এই ঘনত্ব-সংযুক্ততার (Density-Connectedness) কারণেই A, B, C, D , এবং F বিন্দুগুলো কোনো বিভাজন ছাড়াই একটি একক এবং সুনির্দিষ্ট ক্লাস্টারের অংশ হিসেবে আত্মপ্রকাশ করে।

7. Formula

১. প্রতিবেশ বা এলাকার দূরত্ব পরিমাপের সূত্র (Euclidean Distance Formula)

ধাপ ২ (Neighborhood Query)-এ একটি বিন্দুকে কেন্দ্র করে চারপাশে যে Epsilon (ϵ) ব্যাসার্ধের এলাকা বা বৃত্ত কল্পনা করা হয়, তার জন্য ব্যাকএন্ডে **ইউক্লিডিয়ান দূরত্ব (Euclidean Distance)**-এর জ্যামিতিক সূত্র ব্যবহার করা হয়।

ধরা যাক, আমাদের নির্বাচিত প্রাথমিক বিন্দুটি হলো $P(x_1, y_1)$ এবং ডেটাসেটের অন্য যেকোনো একটি বিন্দু হলো $Q(x_2, y_2)$ । তাহলে তাদের মধ্যকার দূরত্ব (d) নির্ণয়ের সূত্র:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

DBSCAN-এর জ্যামিতিক শর্ত: যদি $d(P, Q) \leq \epsilon$ হয়, তবেই কেবল Q বিন্দুটি P -এর প্রতিবেশ বা Neighborhood ($N_\epsilon(P)$)-এর অংশ হিসেবে গণ্য হবে।

২. এলাকার ঘনত্ব এবং বিন্দুর শ্রেণীবিভাগ পরিমাপের সূত্র (Density Set-Inclusion Formula)

ধাপ ৩ (Core Point Check) এবং **ধাপ ৪ (Cluster Expansion)**-এ কোনো বিন্দু Core Point নাকি Noise হবে, তা নির্ধারণের জন্য সেট থিওরি বা **সেট-ইনক্লুশন (Set-Inclusion)** এবং **কার্ডিনালিটি (Count/Cardinality)**-এর গাণিতিক সূত্র ব্যবহার করা হয়।

ধরা যাক, P বিন্দুর ϵ ব্যাসার্ধের ভেতরের প্রতিবেশীদের সেটটি হলো $N_\epsilon(P)$ । এই সেটের গাণিতিক রূপ:

$$N_\epsilon(P) = \{Q \in X \mid d(P, Q) \leq \epsilon\}$$

এখন, এই সেটের মোট বিন্দুর সংখ্যা বা কার্ডিনালিটিকে $|N_\epsilon(P)|$ দ্বারা প্রকাশ করা হয়।

8. Formula Derivation

বিন্দুর শ্রেণীবিভাগের গাণিতিক শর্তসমূহ:

- **Core Point (মূল বিন্দু) হওয়ার শর্ত:**

$$|N_\epsilon(P)| \geq \text{min_samples}$$

(অর্থাৎ, এলাকার মোট বিন্দুর সংখ্যা যদি min_samples বা তার বেশি হয়)

- **Noise Point (নয়েজ বা আউটলায়ার) হওয়ার শর্ত:**

$$|N_\epsilon(P)| < \text{min_samples} \quad \text{AND} \quad P \notin N_\epsilon(C)$$

(এখানে C হলো ডেটাসেটের যেকোনো একটি Core Point। অর্থাৎ, নিজস্ব এলাকায় বিন্দু কম থাকবে এবং সে অন্য কোনো কোর বিন্দুর প্রতিবেশে বা এলাকাতেও থাকবে না)

- **Border Point (সীমান্ত বিন্দু) হওয়ার শর্ত:**

$$|N_\epsilon(P)| < \text{min_samples} \quad \text{AND} \quad P \in N_\epsilon(C)$$

(অর্থাৎ, নিজস্ব এলাকায় বিন্দু কম থাকবে, কিন্তু সে অন্য একটি কোর বিন্দু C -এর এলাকার ভেতরে অবস্থান করবে)

ডিবিএসক্যান অ্যালগরিদমের সম্পূর্ণ গাণিতিক প্রয়োগ (Hand-Calculated Numerical Example)

ডিবিএসক্যান (DBSCAN) অ্যালগরিদমের অভ্যন্তরীণ কার্যপ্রক্রিয়া এবং এর দুটি গাণিতিক সূত্র (Euclidean Distance ও Density Set-Inclusion) কীভাবে ব্যাকএন্ডে কাজ করে, তা একটি ছোট হাতে-গণনাযোগ্য ডেটাসেটের মাধ্যমে শুরু থেকে শেষ পর্যন্ত ধাপে ধাপে দেখানো হলো।

9. Meaning of Every Symbol

প্রতীক	অর্থ
ϵ (Eps)	প্রতিবেশী খোঁজার সর্বোচ্চ ব্যাসার্ধ
$MinPts$	Core Point হওয়ার জন্য প্রয়োজনীয় ন্যূনতম প্রতিবেশী সংখ্যা
$N_\epsilon(p)$	বিন্দু p -এর ϵ -ব্যাসার্ধের ভেতরের সব প্রতিবেশীর সেট
$d(p, q)$	দুটি বিন্দু p এবং q -এর মধ্যে Euclidean Distance
Core Point	যে বিন্দুর $ N_\epsilon(p) \geq MinPts$
Border Point	Core নয়, কিন্তু কোনো Core Point-এর Neighborhood-এর ভেতরে অবস্থিত
Noise Point	Core নয় এবং কোনো Core Point-এর Neighborhood-এও নেই

10. Step-by-Step Formula Explanation

১. ডেটাসেট ও হাইপারপ্যারামিটার নির্ধারণ (Dataset & Parameters)

ধরা যাক, একটি দ্বিমাত্রিক স্থানে (2D Space) আমাদের কাছে $N = 4$ টি ডেটা পয়েন্ট রয়েছে:

- $A_1 = (1,1)$
- $A_2 = (2,1)$
- $A_3 = (2,2)$
- $A_4 = (6,5)$

ডোমেইন নলেজের ভিত্তিতে প্যারামিটার নির্বাচন:

- Epsilon (ϵ) = 2 (প্রতিবেশ খোঁজার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ)
- min_samples = 3 (Core Point হওয়ার সর্বনিম্ন ঘনত্ব থ্রেশহোল্ড)

২. প্রতিবেশীদের দূরত্ব গণনা ও শ্রেণীবিভাগ (Distance Calculation & Classification)

অ্যালগরিদমটি প্রতিটি বিন্দুর জন্য ইউক্লিডিয়ান দূরত্ব সূত্র ব্যবহার করে তার প্রতিবেশ সেট $N_\epsilon(p)$ এবং বিন্দুর সংখ্যা $|N_\epsilon(p)|$ গণনা করবে।

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

ক. বিন্দু $A_1(1, 1)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_1, A_1) = \sqrt{(1-1)^2 + (1-1)^2} = 0 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_1)$
- $d(A_1, A_2) = \sqrt{(2-1)^2 + (1-1)^2} = \sqrt{1+0} = 1 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_1)$
- $d(A_1, A_3) = \sqrt{(2-1)^2 + (2-1)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_1)$
- $d(A_1, A_4) = \sqrt{(6-1)^2 + (5-1)^2} = \sqrt{25+16} = \sqrt{41} \approx 6.40 > 2 \Rightarrow A_4 \notin N_\epsilon(A_1)$
- **ফলাফল:** A_1 এর প্রতিবেশ সেট, $N_\epsilon(A_1) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_1)| = 3$ | যেহেতু $3 \geq \min_samples(3)$, সেহেতু গাণিতিক শর্ত অনুযায়ী:

$A_1 \rightarrow$ Core Point

খ. বিন্দু $A_2(2, 1)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_2, A_1) = \sqrt{(1-2)^2 + (1-1)^2} = 1 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_2)$
- $d(A_2, A_2) = \sqrt{(2-2)^2 + (1-1)^2} = 0 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_2)$
- $d(A_2, A_3) = \sqrt{(2-2)^2 + (2-1)^2} = 1 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_2)$
- $d(A_2, A_4) = \sqrt{(6-2)^2 + (5-1)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66 > 2 \Rightarrow A_4 \notin N_\epsilon(A_2)$
- **ফলাফল:** A_2 এর প্রতিবেশ সেট, $N_\epsilon(A_2) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_2)| = 3$ | যেহেতু $3 \geq 3$, সেহেতু:

$A_2 \rightarrow$ Core Point

গ. বিন্দু $A_3(2, 2)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_3, A_1) = \sqrt{2} \approx 1.41 \leq 2 \Rightarrow A_1 \in N_\epsilon(A_3)$
- $d(A_3, A_2) = 1 \leq 2 \Rightarrow A_2 \in N_\epsilon(A_3)$
- $d(A_3, A_3) = 0 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_3)$
- $d(A_3, A_4) = \sqrt{(6-2)^2 + (5-2)^2} = \sqrt{16+9} = \sqrt{25} = 5 > 2 \Rightarrow A_4 \notin N_\epsilon(A_3)$
- **ফলাফল:** A_3 এর প্রতিবেশ সেট, $N_\epsilon(A_3) = \{A_1, A_2, A_3\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_3)| = 3$ | যেহেতু $3 \geq 3$, সেহেতু:

$A_3 \rightarrow$ Core Point

ঘ. বিন্দু $A_4(6, 5)$ এর জন্য প্রতিবেশ অনুসন্ধান:

- $d(A_4, A_1) \approx 6.40 > 2 \Rightarrow A_1 \notin N_\epsilon(A_4)$
- $d(A_4, A_2) \approx 5.66 > 2 \Rightarrow A_2 \notin N_\epsilon(A_4)$
- $d(A_4, A_3) = 5 > 2 \Rightarrow A_3 \notin N_\epsilon(A_4)$
- $d(A_4, A_4) = 0 \leq 2 \Rightarrow A_4 \in N_\epsilon(A_4)$
- **ফলাফল:** A_4 এর প্রতিবেশ সেট, $N_\epsilon(A_4) = \{A_4\}$
- **ঘনত্ব চেক:** $|N_\epsilon(A_4)| = 1$ । যেহেতু $1 < 3$, সেহেতু সে Core Point হতে পারল না। আবার, A_4 অন্য কোনো Core Point (A_1, A_2, A_3)-এর প্রতিবেশ সেটের ভেতরেও অবস্থান করে না। সুতরাং, গাণিতিক শর্ত অনুযায়ী:

$A_4 \rightarrow$ Noise Point

11. Numerical Example

৩. ব্যাকএন্ড ওয়ার্কফ্লো এবং ক্লাস্টার গঠন (Cluster Formation Process)

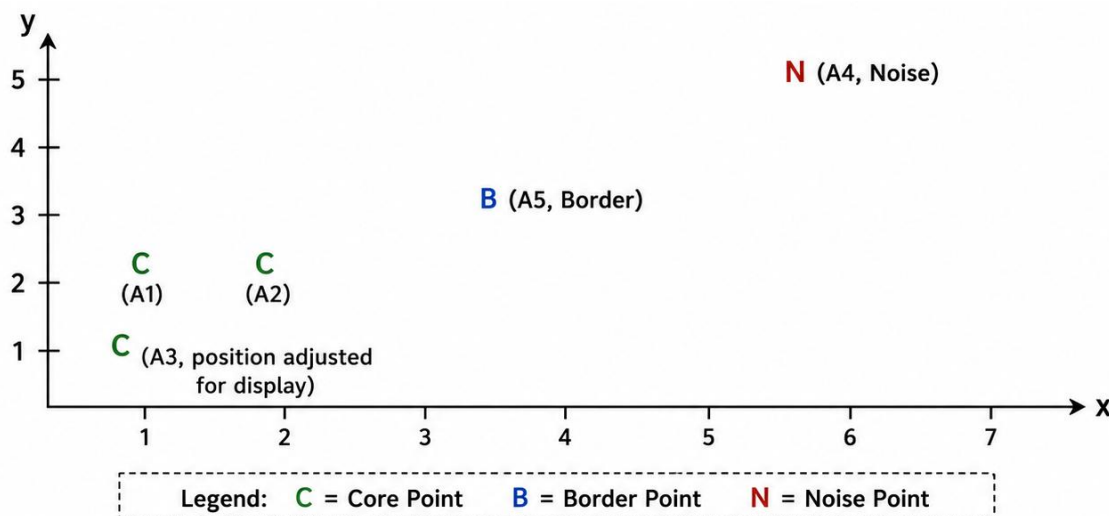
এবার ডিবিএসক্যানের অভ্যন্তরীণ লজিক চেইনের মাধ্যমে চূড়ান্ত আউটপুট তৈরি হবে:

1. **ইটারেশন ১:** অ্যালগরিদমটি প্রথমে র্যান্ডমলি A_1 বিন্দুটি ভিজিট করে এবং দেখে এটি একটি Core Point। তাৎক্ষণিকভাবে একটি নতুন ক্লাস্টার **Cluster 0** তৈরি হয়।
2. **সম্প্রসারণ (Expansion):** A_1 এর প্রতিবেশীদের (A_2, A_3) একটি অস্থায়ী Queue-তে যুক্ত করা হয়।
3. **ডেনসিটি চেইন লিফ্টিং:** * Queue থেকে A_2 বের করা হয়। যেহেতু A_2 নিজেও একটি Core Point, তাই তার প্রতিবেশীদের দিয়ে ক্লাস্টার সম্প্রসারণের চেষ্টা করা হয় (কিন্তু নতুন কোনো বিন্দু পাওয়া যায় না)। A_2 সফলভাবে **Cluster 0**-এ অন্তর্ভুক্ত হয়।
 - এরপর Queue থেকে A_3 বের করা হয়। A_3 নিজেও Core Point প্রমাণিত হওয়ায় সেও **Cluster 0**-এর অংশ হয়ে যায়।
4. **ক্লাস্টার সমাপ্তি:** Queue খালি হয়ে যায় এবং A_1, A_2, A_3 বিন্দুত্রয় জ্যামিতিকভাবে **Density-Connected** হওয়ায় প্রথম ক্লাস্টারের কাজ সম্পন্ন হয়।
5. **ইটারেশন ২:** অবশিষ্ট আনভিজিটেড বিন্দু A_4 ভিজিট করা হয়। যেহেতু এটি কোনো কোর বা বর্ডার বিন্দু নয়, তাই এটিকে চূড়ান্তভাবে **Noise** লেবেল দেওয়া হয়।

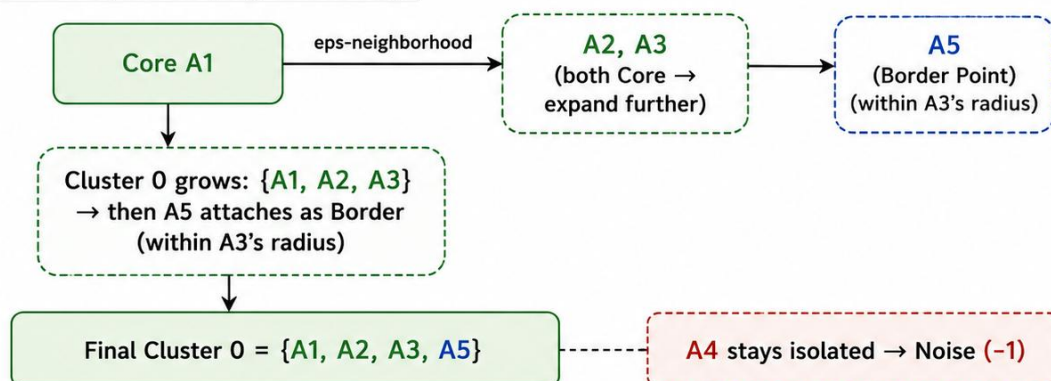
বিন্দু (Point)	আইডি	স্থানাঙ্ক (Coordinates)	বিন্দুর (Type)	ধরণ	চূড়ান্ত ক্লাস্টার (labels_)	লেবেল
A_1		(1,1)	Core Point		0 (First Cluster)	
A_2		(2,1)	Core Point		0 (First Cluster)	
A_3		(2,2)	Core Point		0 (First Cluster)	
A_4		(6,5)	Noise Point		-1 (Noise / Outlier)	

12. Visualization

Visualization — ASCII Diagram



Cluster Expansion (Region Growing):



13. Hyperparameters

sklearn.cluster.DBSCAN ক্লাসে মডেলের ক্লাস্টারিং পারফরম্যান্স, গতি এবং মেমোরি অপ্টিমাইজেশন নিয়ন্ত্রণ করার জন্য বেশ কয়েকটি গুরুত্বপূর্ণ হাইপারপ্যারামিটার রয়েছে। নিচে সবচেয়ে গুরুত্বপূর্ণ (Most Important) থেকে কম গুরুত্বপূর্ণ (Less Important) ক্রমানুসারে প্রতিটি প্যারামিটার বইয়ের ফরম্যাটে বিস্তারিত আলোচনা করা হলো:

১. eps (Epsilon)

- **Default:** 0.5
- **প্যারামিটারটি কী কাজ করে:** এটি হলো দুটি উপাত্ত বিন্দুর মধ্যকার প্রতিবেশ সনাক্ত করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা দূরত্ব। কোনো একটি বিন্দু থেকে অন্য বিন্দুর দূরত্ব যদি এই eps-এর চেয়ে কম বা সমান হয়, তবেই তারা একে অপরের প্রতিবেশী (Neighbor)

হিসেবে গণ্য হবে। এটি ডিবিএসক্যান অ্যালগরিদমের সবচেয়ে সংবেদনশীল এবং গুরুত্বপূর্ণ প্যারামিটার, যা ক্লাস্টারের সংখ্যা এবং আকৃতি নির্ধারণ করে। এর মান সাধারণত যত ছোট নেওয়া হয়, ক্লাস্টারের সংখ্যা তত বৃদ্ধি পায়।

- **অন্যান্য অপশন ও ব্যবহার:** এখানে যেকোনো ধনাত্মক দশমিক সংখ্যা বা ফ্লোট (Float) মান ইনপুট দেওয়া যায়।
 - ডেটা পয়েন্টগুলো যদি খুব ঘন বা গাদাগাদি অবস্থায় থাকে, তবে সূক্ষ্ম ক্লাস্টার খোঁজার জন্য eps-এর মান কমিয়ে দেওয়া উচিত।
 - আর ডেটা যদি ছড়ানো-ছিটানো (Sparse) হয়, তবে বড় ডেনসিটি চেইন তৈরি করতে এর মান বাড়িয়ে দেওয়া প্রয়োজন। সঠিক মান খুঁজে পেতে অনেক সময় K-Distance Graph বা Elbow Method-এর সাহায্য নেওয়া হয়।

২. min_samples

- **Default:** 5
- **প্যারামিটারটি কী কাজ করে:** এটি নির্ধারণ করে যে একটি নির্দিষ্ট বিন্দুর ϵ ব্যাসার্ধের ভেতর নিজেসহ নূন্যতম কতটি উপাত্ত বিন্দু (অথবা টোটাল ওয়েট) উপস্থিত থাকলে সেই বিন্দুটিকে একটি **Core Point** হিসেবে ঘোষণা করা হবে। এর মান যত বেশি বা হাই সেট করা হবে, ডিবিএসক্যান তত বেশি ঘন (Denser) ক্লাস্টার খুঁজে বের করবে। আর এর মান কম হলে তুলনামূলক ছড়ানো বা কম ঘনত্বের (Sparse) ক্লাস্টারও সনাক্ত করা সম্ভব হয়।
- **অন্যান্য অপশন ও ব্যবহার:** এটি যেকোনো ধনাত্মক পূর্ণসংখ্যা (Integer) গ্রহণ করে।
 - ডেটাসেটে যদি নয়েজ বা আউটলাইয়ারের পরিমাণ অনেক বেশি থাকে, তবে নয়েজ পয়েন্টগুলো যেন ভুল করে নিজেদের ক্লাস্টার বানাতে না পারে, তা আটকাতে min_samples-এর মান বাড়িয়ে দেওয়া (যেমন: 10 বা তার বেশি) উচিত।
 - সাধারণ নিয়ম অনুযায়ী (Rule of Thumb), এর মান নূন্যতম ফিচারের সংখ্যার চেয়ে বেশি বা দ্বিগুণ ($\text{MinPts} \geq \text{Features} + 1$) রাখা স্ট্যান্ডার্ড।

৩. metric

- **Default:** 'euclidean'
- **প্যারামিটারটি কী কাজ করে:** ফিচার অ্যারের উপাত্ত বিন্দুগুলোর মধ্যকার পারস্পরিক জ্যামিতিক দূরত্ব পরিমাপ করার জন্য ব্যাকগ্রাউন্ডে কোন গাণিতিক সূত্র ব্যবহার করা হবে, তা এই প্যারামিটারটি ঠিক করে।
- **অন্যান্য অপশন ও ব্যবহার:** এটি স্ট্রিং (String) হিসেবে অনেকগুলো অপশন সাপোর্ট করে (যেমন: 'manhattan', 'cosine', 'minkowski' ইত্যাদি) অথবা কোনো কাস্টম ফাংশনও গ্রহণ করতে পারে।

- 'euclidean': সাধারণ এবং কন্টিনিউয়াস নিউমেরিক্যাল ডেটার জন্য এটি ডিফল্ট এবং সেরা চয়েস।
- 'manhattan': হাই-ডাইমেনশনাল ডেটা কিংবা গ্রিড-ভিত্তিক পথের দূরত্ব মাপার জন্য এটি বেশ কার্যকর।
- 'precomputed': আপনার কাছে যদি আগে থেকেই একটি স্কয়ার ডিস্টেন্স ম্যাট্রিক্স (Distance Matrix) বা স্পার্স গ্রাফ তৈরি করা থাকে, তখন এক্সপ্রেসন হিসেবে ইনপুট ম্যাট্রিক্স পাস করার জন্য এই অপশনটি ব্যবহার করতে হবে।

8. p

- **Default:** None (যা গাণিতিকভাবে $p=2$ বা Euclidean distance-এর সমতুল্য)
- **প্যারামিটারটি কী কাজ করে:** এটি শুধুমাত্র তখনই কার্যকর হয় যখন `metric='minkowski'` ব্যবহার করা হয়। এটি মিনকোভস্কি মেট্রিকের পাওয়ার প্যারামিটার (Power Parameter) হিসেবে দূরত্বের ঘাত নির্ধারণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এখানে যেকোনো পূর্ণসংখ্যা বা দশমিক সংখ্যা ইনপুট দেওয়া যায়।
 - $p=1$: এটি দূরত্বের হিসাবকে সরাসরি ম্যানহাটন দূরত্ব (Manhattan Distance) রূপান্তর করে ফেলে।
 - $p=2$: এটি ইউক্লিডিয়ান দূরত্ব (Euclidean Distance) হিসেবে কাজ করে।
 - যদি ফিচারের বৈশিষ্ট্য অনুযায়ী কাস্টম কোনো ঘাতের দূরত্ব পরিমাপ করতে হয়, কেবল তখনই এই মান পরিবর্তন করার প্রয়োজন পড়ে।

৫. algorithm

- **Default:** 'auto'
- **প্যারামিটারটি কী কাজ করে:** প্রতিটি বিন্দুর জন্য তার নিকটবর্তী প্রতিবেশী বা Nearest Neighbors খুঁজে বের করার সময় ব্যাকএন্ডে কোন সার্চ অ্যালগরিদম আর্কিটেকচার ব্যবহার করা হবে, তা এটি নির্ধারণ করে। এটি ক্লাস্টারিংয়ের এক্যুরেসিতে কোনো পরিবর্তন আনে না, শুধুমাত্র প্রসেসিং স্পিড নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এর অপশনগুলো হলো: 'ball_tree', 'kd_tree', এবং 'brute'।
 - 'auto': সায়েন্টিফিক লাইব্রেরিটি `.fit()` মেথডে পাস করা ডেটার সাইজ এবং ডাইমেনশন দেখে নিজেই সবচেয়ে দ্রুততম ও সেরা অ্যালগরিদমটি স্বয়ংক্রিয়ভাবে বেছে নেয়।

- 'kd_tree': কম ডাইমেনশন বা কম ফিচার বিশিষ্ট বড় ডেটাসেটে দ্রুত কোয়েরি করার জন্য উপযোগী।
- 'ball_tree': হাই-ডাইমেনশনাল ডেটাসেটে যেখানে কেডি-ট্রি দুর্বল পারফর্ম করে, সেখানে এটি ভালো গতি দেয়।
- 'brute': একদম ছোট ডেটাসেটের জন্য ব্রুট-ফোর্স (সব বিন্দুর সাথে সবার দূরত্ব সরাসরি হিসাব করা) পদ্ধতিটি ব্যবহার করা যেতে পারে।

৬. n_jobs

- **Default:** None (যার অর্থ কম্পিউটারের মাত্র ১টি প্রসেসর কোর প্রসেসিংয়ে অংশ নেবে)
- **প্যারামিটারটি কী কাজ করে:** Nearest Neighbors কোয়েরি এবং দূরত্ব গণনার বড় ম্যাথমেটিক্যাল ক্যালকুলেশনগুলো সম্পন্ন করার সময় সিপিইউ (CPU)-এর কতগুলো কোর সমান্তরালভাবে বা প্যারালালি (Parallel Jobs) কাজ করবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন ও ব্যবহার:** এখানে নির্দিষ্ট প্রসেসর সংখ্যা (পূর্ণসংখ্যা) দেওয়া যায়।
 - বড় ডেটাসেটে কম্পিউটেশন স্পিড বহুগুণ বৃদ্ধি করার জন্য এই প্যারামিটারের মান -1 ব্যবহার করা উচিত। n_jobs=-1 দিলে সিস্টেমের সমস্ত অ্যাভেইলেবল সিপিইউ কোর একসাথে প্যারালাল প্রসেসিং শুরু করে, যা মডেল ট্রেনিংয়ের সময় বহুগুণ বাঁচিয়ে দেয়।

৭. leaf_size

- **Default:** 30
- **প্যারামিটারটি কী কাজ করে:** এটি শুধুমাত্র তখনই প্রভাব ফেলে যখন আগের algorithm প্যারামিটারে 'ball_tree' অথবা 'kd_tree' সক্রিয় থাকে। এটি ইন্টারনাল ট্রির লিফ নোডের সাইজ নির্ধারণ করে দেয়।
- **অন্যান্য অপশন ও ব্যবহার:** যেকোনো পূর্ণসংখ্যা গ্রহণ করে।
 - এই মানটি পরিবর্তনের মাধ্যমে ট্রি কনস্ট্রাকশন স্পিড, কোয়েরি স্পিড এবং মেমোরি স্টোরেজ কস্ট অপ্টিমাইজ করা যায়। সাধারণ মেশিন লার্নিং ওয়ার্কফ্লোতে এর ডিফল্ট মান ৩০-ই সবচেয়ে সুষম পারফরম্যান্স দেয়, তাই অ্যাডভান্সড অপ্টিমাইজেশন ছাড়া এটি পরিবর্তন করার প্রয়োজন হয় না।

৮. metric_params

- **Default:** None
- **প্যারামিটারটি কী কাজ করে:** দূরত্বের জন্য ব্যবহৃত কাস্টম বা জটিল কোনো ডিস্টেন্স ফাংশনে যদি অতিরিক্ত কোনো কি-ওয়ার্ড আর্গুমেন্ট (Keyword Arguments) বা

প্যারামিটার পাস করার প্রয়োজন হয়, তবে সেটিকে ডিকশনারি (Dictionary) আকারে পাঠানোর জন্য এটি ব্যবহৃত হয়।

- **অন্যান্য অপশন ও ব্যবহার:** এটি একটি পাইথন ডিকশনারি {} গ্রহণ করে। সাধারণ স্ট্যান্ডার্ড মেট্রিক ব্যবহারের সময় এটি সম্পূর্ণ অব্যবহৃত থাকে; শুধুমাত্র অ্যাডভান্সড কাস্টম ডিস্টেন্স ম্যাপিংয়ের সময় এর ব্যবহার দেখা যায়।

(Attributes & Methods)

sklearn.cluster.DBSCAN মডেলটি .fit() করার পর, অ্যালগরিদমটি তার অভ্যন্তরীণ লার্নিং এবং ক্লাস্টার বিন্যাসের সমস্ত তথ্য বেশ কিছু গুরুত্বপূর্ণ অ্যাট্রিবিউটের (Attributes) মধ্যে সংরক্ষণ করে। নিচে গুরুত্বপূর্ণ অ্যাট্রিবিউটসমূহ এবং মডেলের কোর মেথডগুলোর (Methods) কার্যকারিতা বইয়ের ফরম্যাটে বিস্তারিত আলোচনা করা হলো:

মডেল অ্যাট্রিবিউটসমূহ (DBSCAN Attributes)

মডেল ফিট হওয়ার পর ব্যাকএন্ডের লার্নিং ডেটা অ্যানালাইসিস করার জন্য এই অ্যাট্রিবিউটগুলো ব্যবহার করা হয়। এদের প্রতিটির শেষে একটি আন্ডারস্কোর (_) থাকে।

labels_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি হলো সবচেয়ে গুরুত্বপূর্ণ অ্যাট্রিবিউট, যা ইনপুট করা মূল ডেটাসেটের প্রতিটি উপাত্ত বিন্দুর জন্য নির্ধারিত ক্লাস্টার আইডির একটি অ্যারে (Array) প্রদান করে। ডিবিএসক্যান যে বিন্দুগুলোকে সফলভাবে ক্লাস্টারে গ্রুপ করতে পারে, সেগুলোকে অ-ঋণাত্মক পূর্ণসংখ্যা বা নন-নেগেটিভ ইন্টিজার (যেমন: 0, 1, 2 ইত্যাদি) লেবেল দেয়। আর যে বিন্দুগুলো কম ঘনত্বের কারণে কোনো ক্লাস্টারের অংশ হতে পারে না, সেগুলোকে সরাসরি -1 লেবেল দিয়ে নয়েজ বা আউটলায়ার (Noise/Outliers) হিসেবে চিহ্নিত করে।

core_sample_indices_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি ডেটাসেটে খুঁজে পাওয়া সমস্ত **Core Points**-এর সূচক বা ইণ্ডেক্স নম্বরগুলোর (Indices) একটি অ্যারে ধারণ করে। এর মাধ্যমে জানা যায় মূল ডেটাসেটের ঠিক কত নম্বর লাইনের সারিগুলো (Rows) কোর স্যাম্পল হিসেবে প্রমাণিত হয়েছে।

components_

- **Default:** Created after fit()
- **অ্যাট্রিবিউটটি কী কাজ করে:** এটি ট্রেনিংয়ের সময় খুঁজে পাওয়া প্রতিটি কোর স্যাম্পলের (Core Samples) মূল ফিচারের মানগুলোর একটি বাস্তব কপি মেমোরিতে ধরে রাখে। এর শেপ বা আকৃতি হয় (n_core_samples, n_features)।

`n_features_in_`

- **Default:** Created after `fit()`
- **অ্যাট্রিবিউটটি কী কাজ করে:** মডেলটি ফিট করার সময় ইনপুট ম্যাট্রিক্সে (X) মোট কতটি ফিচার বা কলাম দেখা হয়েছিল, তার সুনির্দিষ্ট সংখ্যা এটি সংরক্ষণ করে।

`feature_names_in_`

- **Default:** Created after `fit()` (Only for DataFrame input)
- **অ্যাট্রিবিউটটি কী কাজ করে:** ট্রেনিংয়ের সময় ইনপুট ডেটা হিসেবে যদি কলামের নামযুক্ত প্যান্ডাস ডেটাফ্রেম (Pandas DataFrame) ব্যবহার করা হয়, তবে সব কলামের অফিশিয়াল নামের একটি অ্যারে এটি সংরক্ষণ করে।

14. Scikit-learn Import

```
import numpy as np
from sklearn.cluster import DBSCAN
```

15. Python Example

From-Scratch বাস্তবায়ন (NumPy ব্যবহার করে)

```
import numpy as np

def region_query(X, point_idx, eps):
    """point_idx বিন্দুর eps-ব্যাসার্ধের ভেতরে থাকা সব প্রতিবেশী পয়েন্টের ইনডেক্স ফেরত দেয়"""
    distances = np.linalg.norm(X - X[point_idx], axis=1)
    return np.where(distances <= eps)[0]

def dbscan_from_scratch(X, eps, min_samples):
    n = len(X)
    labels = np.full(n, -2) # -2 = এখনো ভিজিট করা হয়নি
    cluster_id = 0

    for i in range(n):
        if labels[i] != -2:
            continue # ইতিমধ্যে প্রসেস করা হয়েছে

        neighbors = region_query(X, i, eps)

        if len(neighbors) < min_samples:
            labels[i] = -1 # প্রাথমিকভাবে Noise (পরে Border হতে পারে)
            continue

        # i একটি Core Point - নতুন Cluster শুরু করা হলো
        labels[i] = cluster_id
        seeds = list(neighbors)
```

```

seeds.remove(i)

j = 0
while j < len(seeds):
    q = seeds[j]
    if labels[q] == -1:
        labels[q] = cluster_id # Noise ছিল, এখন Border হিসেবে যুক্ত
        হলো
    if labels[q] == -2:
        labels[q] = cluster_id
        q_neighbors = region_query(X, q, eps)
        if len(q_neighbors) >= min_samples:
            # q নিজেও Core Point - Cluster আরও সম্প্রসারিত হবে
            seeds.extend([x for x in q_neighbors if x not in seeds])
        j += 1

cluster_id += 1

return labels

```

```

X = np.array([[1, 1], [2, 1], [2, 2], [6, 5], [3, 3]])
labels = dbscan_from_scratch(X, eps=2, min_samples=3)
print("Cluster Labels:", labels) # -1 = Noise

```

Scikit-learn ব্যবহার করে বাস্তবায়ন

```

from sklearn.cluster import DBSCAN
import numpy as np

```

```

X = np.array([[1, 1], [2, 1], [2, 2], [6, 5], [3, 3]])

```

```

model = DBSCAN(eps=2, min_samples=3)
model.fit(X)

```

```

print("Cluster Labels:", model.labels_) # -1 = Noise
print("Core Sample Indices:", model.core_sample_indices_)
print("Core Samples:", X[model.core_sample_indices_])

```

16. Prediction / Clustering Process

কোর মেথডসমূহ (DBSCAN Methods)

ডিবিএসক্যান অবজেক্টের ওপর অপারেশন চালানো এবং পাইপলাইনে ডেটা প্রসেস করার জন্য নিচের মেথডগুলো ব্যবহৃত হয়।

fit(X, y=None, sample_weight=None)

- **মেথডটি কী কাজ করে:** এটি ইনপুট ফিচার অ্যারে বা ডিস্টেন্স ম্যাট্রিক্স ব্যবহার করে ডিবিএসক্যানের মূল ক্লাস্টারিং প্রক্রিয়া সম্পন্ন করে। এটি ডেটার ঘনত্ব পরিমাপ করে কোর, বর্ডার ও নয়েজ পয়েন্ট আলাদা করে ব্যাকএন্ড মেকানিজম সম্পন্ন করে।
- **অন্যান্য প্যারামিটার লজিক:** * `y`: এটি এখানে সম্পূর্ণ ইগনোর করা হয় (Ignored)। শুধুমাত্র Scikit-Learn-এর অফিশিয়াল এপিআই ডিজাইন এবং কনভেনশন বজায় রাখার জন্য প্যারামিটারটি উপস্থিত থাকে।
 - `sample_weight`: এটি প্রতিটি ডেটা স্যাম্পলের জন্য কাস্টম ওয়েট বা গুরুত্ব নির্ধারণ করতে ব্যবহৃত হয়। কোনো স্যাম্পলের ওয়েট যদি `min_samples`-এর চেয়ে বেশি হয়, তবে সে একাই একটি কোর স্যাম্পল হতে পারে। অন্যদিকে, কোনো স্যাম্পলের ওয়েট খণাত্মক হলে তা তার প্রতিবেশীদের কোর স্যাম্পল হতে বাধা দিতে পারে। এর ডিফল্ট মান থাকে 1।

fit_predict(X, y=None, sample_weight=None)

- **মেথডটি কী কাজ করে:** এটি অত্যন্ত জনপ্রিয় একটি শর্টকাট মেথড। এটি এক লাইনের কোডেই মডেলকে ডেটার ওপর ফিট করে এবং একই সাথে প্রতিটি বিন্দুর জন্য নির্ধারিত ক্লাস্টার লেবেলের অ্যারেটি সরাসরি রিটার্ন (Return) করে।
- **ব্যবহারের প্রসঙ্গ:** কোডিংয়ের সময় আলাদা করে `.fit(X)` এবং পরবর্তীতে `.labels_` কল না করে সরাসরি লেবেল অ্যারেটি মেমোরিতে নিয়ে আসার জন্য এটি ব্যবহার করা হয়। এটি মূলত `fit(X).labels_`-এর সমতুল্য।

get_params(deep=True)

- **মেথডটি কী কাজ করে:** এস্টিমেটরের বর্তমান কনফিগারেশনে কোন কোন হাইপারপ্যারামিটার (যেমন: `eps`, `min_samples` ইত্যাদি) কী মানে সেট করা আছে, তার একটি সুনির্দিষ্ট ম্যাপিং ডিকশনারি রিটার্ন করে। এটি গ্রিড সার্চ বা মডেল ট্র্যাকিংয়ের সময় বেশ উপযোগী।

set_params(params)

- **মেথডটি কী কাজ করে:** এটি একটি কাস্টম মেথড যার সাহায্যে অবজেক্ট তৈরি

17. Advantages

- **যেকোনো আকৃতির Cluster শনাক্তকরণ:** গোলাকার আকৃতির কোনো পূর্বধারণা ছাড়াই DBSCAN জটিল, আঁকাবাঁকা আকৃতির Cluster সঠিকভাবে খুঁজে বের করতে পারে।
- **স্বয়ংক্রিয় Noise সনাক্তকরণ:** কোনো বিন্দুকে জোর করে Cluster-এ না ঢুকিয়ে DBSCAN স্বাভাবিকভাবেই Outlier-কে -1 Label দিয়ে আলাদা করে।
- **K আগে থেকে জানার প্রয়োজন নেই:** Cluster-এর সংখ্যা ডেটা থেকেই স্বয়ংক্রিয়ভাবে নির্ধারিত হয়।
- **Centroid-নির্ভরতা নেই:** ফলে Outlier-এর কারণে কোনো কেন্দ্রবিন্দু বিকৃত হওয়ার ঝুঁকি নেই।

18. Disadvantages

- **প্যারামিটার সংবেদনশীলতা:** ϵ এবং $MinPts$ -এর সঠিক মান নির্বাচন করা কঠিন, এবং ভুল মান সম্পূর্ণ ফলাফল নষ্ট করে দিতে পারে।
- **Variable Density ডেটাসেটে দুর্বলতা:** DBSCAN একটি Global ϵ ব্যবহার করে বলে একই ডেটাসেটে বিভিন্ন ঘনত্বের অঞ্চল থাকলে এটি সঠিকভাবে সবগুলো Cluster ধরতে ব্যর্থ হতে পারে (এক্ষেত্রে HDBSCAN ব্যবহার করা উচিত)।
- **উচ্চ-মাত্রিক ডেটায় Curse of Dimensionality:** Dimension বেশি হলে Euclidean Distance-এর অর্থপূর্ণতা কমে যায়, ফলে Neighborhood নির্ধারণ অনির্ভরযোগ্য হয়ে পড়ে।
- **বড় ডেটাসেটে মেমোরি খরচ:** Spatial Index (KD-Tree/Ball-Tree) ছাড়া প্রতিটি বিন্দুর জন্য প্রতিবেশী খোঁজা ব্যয়বহুল হয়ে ওঠে।

19. Applications

- ভৌগোলিক/Spatial ডেটায় Hotspot বা ঘনবসতিপূর্ণ অঞ্চল চিহ্নিতকরণ।
- ক্রেডিট কার্ড লেনদেনে Fraud ও Anomaly Detection।
- স্যাটেলাইট ইমেজ থেকে ভূমির ব্যবহার (Land-use) বিশ্লেষণ।
- Network Traffic-এ অস্বাভাবিক প্যাটার্ন সনাক্তকরণ।
- Astronomy-তে তারার Cluster বা Galaxy Cluster শনাক্তকরণ।

20. Comparison with Previous Algorithm

K-Means বনাম DBSCAN

বিষয়	K-Means	DBSCAN
Training Process	Iterative Centroid Optimization (Expectation-Maximization ধাঁচের)	Density-ভিত্তিক Region Growing, Core Point থেকে Cluster সম্প্রসারণ
Distance Measure	প্রতিটি পয়েন্ট থেকে Centroid পর্যন্ত Euclidean Distance	পয়েন্ট-থেকে-পয়েন্ট Euclidean Distance এবং ϵ -Neighborhood ঘনত্ব
Cluster-এর আকৃতি	শুধুমাত্র উত্তল/গোলাকার (Convex/Spherical)	সম্পূর্ণ যেকোনো আকৃতি (Arbitrary shape), Non-convex ডেটায়ও কার্যকর
Noise Handling	নেই — প্রতিটি পয়েন্ট বাধ্যতামূলকভাবে নিকটতম Cluster-এ পড়ে, Outlier Centroid-কে বিকৃত করে	স্বয়ংক্রিয়ভাবে Noise/Outlier শনাক্ত করে আলাদা রাখে (Label = -1)

Scalability	অত্যন্ত দ্রুত ও সহজে Scalable, বৃহৎ ডেটাসেটে জনপ্রিয়	মাঝারি — Spatial Index (KD-Tree/Ball-Tree) ছাড়া বড় ডেটাসেটে ধীর হতে পারে
Computational Cost	গড়ে $O(K \cdot n \cdot T)$	গড়ে $O(n \log n)$ (Index সহ), সবচেয়ে খারাপ ক্ষেত্রে $O(n^2)$
Best Use Cases	Cluster সংখ্যা আগে থেকে জানা এবং প্রায় গোলাকার, সমান-ঘনত্বের গ্রুপ (যেমন Customer Segmentation)	অনিয়মিত আকৃতির Cluster, প্রচুর Noise-সহ ভৌগোলিক/Spatial বা Anomaly Detection সমস্যা

বিষয়	Agglomerative Clustering	DBSCAN
Training Process	Bottom-Up Hierarchical, প্রতি ধাপে নিকটতম জোড়া একত্রীকরণ	Density-ভিত্তিক Region Growing, Core Point থেকে Cluster সম্প্রসারণ
Distance Measure	Linkage Criteria (Ward, Single, Complete, Average)	Euclidean Distance-এর ভিত্তিতে ϵ -Neighborhood
Cluster-এর আকৃতি	Linkage-নির্ভর; Ward সাধারণত গোলাকার, Single যেকোনো আকৃতি নিতে পারে	সম্পূর্ণ যেকোনো আকৃতি (Arbitrary shape), Non-convex ডেটায়ও কার্যকর
Noise Handling	নেই — প্রতিটি পয়েন্ট বাধ্যতামূলকভাবে কোনো না কোনো Cluster-এ পড়ে	স্বয়ংক্রিয়ভাবে Noise/Outlier শনাক্ত করে (Label = -1)
Scalability	কম — বড় ডেটাসেটে অত্যন্ত ধীর	মাঝারি — Spatial Index (KD-Tree) ব্যবহার করলে ভালো Scale করে
Computational Cost	$O(n^2 \log n)$ থেকে $O(n^3)$, Space $O(n^2)$	গড়ে $O(n \log n)$ (Index সহ), সবচেয়ে খারাপ ক্ষেত্রে $O(n^2)$
Best Use Cases	Hierarchical সম্পর্ক প্রয়োজন এমন ডেটা (Genomics, Taxonomy), ছোট-মাঝারি ডেটাসেট	অনিয়মিত আকৃতির Cluster ও প্রচুর Noise-সহ ভৌগোলিক/Spatial ডেটা

21. Common Mistakes

- **ϵ যাচাই ছাড়াই আন্দাজে বসানো:** k-distance graph (k-NN Distance Plot) বিশ্লেষণ না করে সরাসরি ϵ বসিয়ে দিলে ভুল ফলাফল আসতে পারে।
- **Feature Scaling উপেক্ষা করা:** DBSCAN সম্পূর্ণ দূরত্ব-নির্ভর, তাই Scaling ছাড়া বড় স্কেলের Feature দূরত্ব গণনাকে ভুলভাবে প্রভাবিত করে।
- **Noise Label-কে সাধারণ Cluster ভেবে ভুল করা:** -1 Label-কে ভুলবশত আরেকটি বৈধ Cluster হিসেবে বিশ্লেষণ করলে ফলাফলের ভুল ব্যাখ্যা হতে পারে।

- **Variable Density ডেটায় সাধারণ DBSCAN প্রয়োগ করা:** এমন ডেটাসেটে HDBSCAN বা OPTICS ব্যবহার না করে সাধারণ DBSCAN প্রয়োগ করলে গুরুত্বপূর্ণ Cluster হারিয়ে যেতে পারে।

22. Interview Questions

প্রশ্ন ১: কে-মিন্স (K-Means)-এর তুলনায় জটিল ও অরৈখিক (Non-linear) আকৃতির উপাত্ত শ্রেণীবিভাগে ডিবিএসক্যান (DBSCAN) কেন অনন্য কার্যকারিতা প্রদর্শন করে?

- **উত্তর:** K-Means অ্যালগরিদমটি সেন্ট্রয়েড-ভিত্তিক হওয়ার কারণে এটি ধরে নেয় ক্লাস্টারগুলো সবসময় বৃত্তাকার বা গোলকাকার (Spherical). ফলে আঁকাবাঁকা বা চাঁদের মতো (make_moons) জটিল ডেটার বিন্যাস থাকলে K-Means ক্লাসগুলোকে মাঝখান থেকে সোজা লাইনে কেটে ফেলে এবং ভুল ক্লাস্টারিং করে.
- বিপরীতে, DBSCAN কোনো নির্দিষ্ট কেন্দ্রবিন্দু খোঁজার চেষ্টা করে না; এটি সম্পূর্ণভাবে ডেটার উচ্চ-ঘনত্ব বিশিষ্ট এলাকা অনুসরণ করে অগ্রসর হয়. চাঁদের মতো বাঁকা লাইনের প্রতিটি বিন্দু পরস্পরের খুব কাছাকাছি থাকায় তারা একটি অবিচ্ছিন্ন ডেনসিটি শিকল বা সেতু তৈরি করে. এই ঘনত্ব-সংযুক্ততার (Density-Connectedness) কারণে জ্যামিতিক আকৃতি যত জটিল বা আঁকাবাঁকাই হোক না কেন, DBSCAN নিখুঁতভাবে আসল ক্লাস্টার সনাক্ত করতে পারে.

প্রশ্ন ২: ডিবিএসক্যান (DBSCAN) অ্যালগরিদমে Epsilon (ϵ) প্যারামিটারটি খুব ছোট বা খুব বড় সেট করলে মডেলের আউটপুটে কী প্রভাব পড়ে?

- **উত্তর:** eps হলো একটি নির্দিষ্ট ডেটা পয়েন্ট থেকে তার প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা খোঁজাখুঁজির এলাকা.
 - ϵ **অত্যধিক ছোট হলে (Too Small):** খোঁজাখুঁজির ব্যাসার্ধ ছোট হওয়ার কারণে ডেটা পয়েন্টগুলো একে অপরের খুব কাছে থাকা সত্ত্বেও প্রয়োজনীয় ঘনত্ব (MinPts) পূর্ণ করতে পারে না. এর ফলে অনেক আসল ক্লাস্টার তৈরি হতে ব্যর্থ হয় এবং ডেটার একটি বিশাল অংশ **Noise (Label: -1)** হিসেবে বাদ পড়ে যায় (Underfitting/High Bias).
 - ϵ **অত্যধিক বড় হলে (Too Large):** এলাকা অনেক বড় হয়ে যাওয়ার কারণে সম্পূর্ণ আলাদা এবং ভিন্ন ভিন্ন দুটি ক্লাস্টারও একটি কমন ব্যাসার্ধের ভেতর চলে আসে. এর ফলে একাধিক ক্লাস্টার জোড়া লেগে একটি একক বৃহৎ ক্লাস্টারে পরিণত হয় (Overfitting/High Variance).

প্রশ্ন ৩: ডিবিএসক্যান (DBSCAN)-এ একটি সীমান্ত বিন্দু (Border Point) এবং নয়েজ বিন্দু (Noise Point)-এর মধ্যকার গাণিতিক ও কাঠামোগত পার্থক্য কী?

- **উত্তর:** 1. **Border Point (সীমান্ত বিন্দু):** এই বিন্দুর নিজস্ব ϵ ব্যাসার্ধের বৃত্তের ভেতর প্রয়োজনীয় MinPts-এর চেয়ে কম সংখ্যক পয়েন্ট থাকে (তাই সে নিজে Core Point হতে পারে না). কিন্তু, পয়েন্টটি যদি অন্য কোনো একটি Core Point-এর এলাকার সীমানার

ভেতরে অবস্থান করে, তবে তাকে **Border Point** বলে। এরা ক্লাস্টারের একদম বাইরের সীমানা বা ডালপালা তৈরি করে।

2. Noise Point (নয়েজ বিন্দু): এই বিন্দুর নিজস্ব ব্যাসার্ধের ভেতর $MinPts$ -এর চেয়ে কম পয়েন্ট থাকে এবং এটি আশেপাশের কোনো Core Point-এর এলাকার ভেতরেও পড়ে না (অর্থাৎ সে সম্পূর্ণ বিচ্ছিন্ন)। DBSCAN এদেরকে কোনো ক্লাস্টারে জায়গা না দিয়ে সরাসরি -1 লেবেল দিয়ে আউটলায়ার হিসেবে মেমোরি থেকে আলাদা করে দেয়।

প্রশ্ন ৪: ডিবিএসক্যানে ঘনত্ব-সংযুক্ত বিন্দু (Density-Connected Points) বলতে কী বোঝায় এবং এটি ক্লাস্টার গঠনে কীভাবে ভূমিকা রাখে?

- **উত্তর:** যদি উপাত্ত ক্ষেত্রের দুটি বিন্দু A এবং F একে অপরের সাথে সরাসরি কানেক্টেড না থাকে, এমনকি তারা যদি পরস্পর থেকে অনেক দূরে অবস্থান করে, তবুও তারা দুজনেই যদি অন্য কোনো একটি সাধারণ মূল বিন্দু চেইনের (Common Core Point Chain) সাথে যুক্ত বা Density-Reachable থাকে, তবে গাণিতিকভাবে A এবং F -কে **Density-Connected** বলা হয়।
- **ভূমিকা:** ডিবিএসক্যানের মূল নিয়মই হলো—যে পয়েন্টগুলো একে অপরের সাথে ঘনত্ব-সংযুক্ত অবস্থায় থাকে, অ্যালগরিদমটি তাদের সবাইকে একটি একক ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত করে। চেইন রিয়াকশনের মতো এক কোর বিন্দু অন্য কোর বিন্দুর হাত ধরে পুরো সেতুটি লক করে ফেলে, যার ফলে দূরবর্তী বিন্দুগুলোও একই গ্রুপের অংশ হিসেবে আত্মপ্রকাশ করে।

প্রশ্ন ৫: ভিন্ন ভিন্ন বা পরিবর্তনশীল ঘনত্ব (Variable/Multi-Density) যুক্ত ডেটাসেটে ডিবিএসক্যান (DBSCAN) কেন দুর্বল পারফর্ম করে এবং এর সমাধান কী?

- **উত্তর:** DBSCAN-এর একটি বড় সীমাবদ্ধতা হলো, এটি সম্পূর্ণ ডেটাসেটের ওপর একটি **Global ϵ** এবং **Global $MinPts$** ব্যবহার করে কাজ করে। অর্থাৎ, পুরো উপাত্ত ক্ষেত্রের সব জায়গায় ঘনত্বের থ্রেশহোল্ড সমান থাকে।
- **দুর্বলতার কারণ:** ডেটাসেটে যদি Variable Density (কোথাও খুব ঘন এবং কোথাও খুব ছড়ানো ডেটা) থাকে, তবে ছড়ানো বা কম ঘনত্বের (Sparse Density) এলাকাগুলোকে ধরার জন্য ϵ -এর মান বড় করলে ঘন এলাকার একাধিক আলাদা ক্লাস্টার একসাথে ব্লেণ্ড বা মার্জ (Merge) হয়ে যায়। আবার ঘন এলাকাকে ধরার জন্য ϵ ছোট করলে, ছড়ানো এলাকার ডেটাগুলো কোনো ক্লাস্টার না পেয়ে সম্পূর্ণ **Noise** হিসেবে বাদ পড়ে যায়।
- **সমাধান:** ডোমেইন নলেজের ভিত্তিতে ডেটাতে এমন Variable Density দেখা গেলে সাধারণ DBSCAN ব্যবহার না করে এর উন্নত সংস্করণ **HDBSCAN (Hierarchical DBSCAN)** অথবা **OPTICS** অ্যালগরিদম ব্যবহার করা উচিত, যা স্বয়ংক্রিয়ভাবে এলাকার ঘনত্ব অনুযায়ী পরিবর্তনশীল ব্যাসার্ধ হিসাব করতে পারে।

23. Summary

ডিবিএসক্যান ক্লাস্টারিং সারসংক্ষেপ

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) হলো একটি শক্তিশালী আনসুপারভাইজড মেশিন লার্নিং অ্যালগরিদম, যা উপাত্তের জ্যামিতিক ঘনত্ব (Density) বিশ্লেষণ করে স্বয়ংক্রিয়ভাবে ক্লাস্টার তৈরি করে।

১. মূল প্যারামিটারসমূহ (Core Parameters)

- **Epsilon (ϵ):** প্রতিবেশীদের খুঁজে বের করার সর্বোচ্চ জ্যামিতিক ব্যাসার্ধ বা দূরত্ব।
- **MinSamples (MinPts):** ক্লাস্টার গঠনের জন্য ওই ব্যাসার্ধের ভেতর নিজেসহ নূন্যতম প্রয়োজনীয় বিন্দুর সংখ্যা।

২. বিন্দুর শ্রেণীবিভাগ (Data Point Classifications)

DBSCAN প্যারামিটারের ওপর ভিত্তি করে পুরো ডেটাসেটকে ৩টি সুনির্দিষ্ট বিন্দুতে ভাগ করে:

- **Core Points:** ব্যাসার্ধের ভেতর *MinPts* বা তার বেশি প্রতিবেশী থাকা ঘন বিন্দু।
- **Border Points:** নিজে Core Point নয়, কিন্তু কোনো Core Point-এর এলাকার ভেতরে থাকা সীমান্ত বিন্দু।
- **Noise Points:** ব্যাসার্ধের ভেতর প্রয়োজনীয় ঘনত্ব না পাওয়া এবং সম্পূর্ণ বিচ্ছিন্ন আউটলায়ার বিন্দু (Label: -1)।

৩. ক্লাস্টার গঠনের নীতি (Density-Connected Rules)

- **Directly Density-Reachable:** একটি বিন্দু অন্য একটি Core Point-এর সীমানায় সরাসরি অবস্থান করলে।
- **Density-Reachable:** কোর বিন্দুগুলোর একটি অবিচ্ছিন্ন ধারাবাহিক শিকল বা চেইনের মাধ্যমে যুক্ত থাকলে।
- **Density-Connected:** দুটি দূরবর্তী বিন্দু সরাসরি যুক্ত না থাকলেও যদি একই কমন কোর বিন্দু চেইনের সাথে যুক্ত থাকে। **গাণিতিক নিয়ম অনুযায়ী, ঘনত্ব-সংযুক্ত বিন্দুগুলো সবসময় একই ক্লাস্টারের (Same Cluster) অন্তর্ভুক্ত হয়।**

৪. K-Means-এর পরিবর্তে কেন DBSCAN শ্রেষ্ঠ?

- **Arbitrary Shapes:** K-Means যেখানে শুধু বৃত্তাকার ক্লাস্টার তৈরি করতে পারে, DBSCAN সেখানে চাঁদের মতো (make_moons) যেকোনো জটিল বা আঁকাবাঁকা আকৃতির ক্লাস্টার নিখুঁতভাবে সনাক্ত করতে পারে।
- **No Pre-defined K:** K-Means-এর মতো ডিবিএসক্যানে রান করার আগে থেকে ক্লাস্টারের সংখ্যা (K) বলে দিতে হয় না।

- **Robust to Outliers:** এটি আউটলায়ারগুলোকে জোর করে ক্লাস্টারে না ঢুকিয়ে সরাসরি **Noise** হিসেবে আলাদা করে দেয়, ফলে আসল ক্লাস্টারের জ্যামিতিক সীমানা সুরক্ষিত থাকে।

Hierarchical DBSCAN (HDBSCAN)

24. Complete Mathematical Implementation (Full Manual Walkthrough)

আগের সেকশনে A_1, A_2, A_3, A_4 নিয়ে সম্পূর্ণ Core ও Noise Point নির্ধারণ ইতিমধ্যে ধাপে ধাপে দেখানো হয়েছে। এখন সেই একই ডেটাসেটে একটি নতুন পঞ্চম পয়েন্ট A_5 যোগ করে **Border Point** কীভাবে নির্ধারিত হয় তা সম্পূর্ণভাবে ম্যানুয়ালি দেখানো হলো, যাতে Core, Border এবং Noise — এই তিন ধরনের পয়েন্টই একটি একক উদাহরণে সম্পূর্ণরূপে প্রদর্শিত হয়।

Dataset (সম্প্রসারিত)

একই $\epsilon = 2$ এবং $min_samples = 3$ রেখে একটি নতুন পয়েন্ট যোগ করা হলো:

- $A_1 = (1,1), A_2 = (2,1), A_3 = (2,2), A_4 = (6,5)$ (আগের মতোই)
- $A_5 = (3,3)$ (নতুন পয়েন্ট)

Distance Calculation — A_5 -এর জন্য

$$d(A_5, A_1) = \sqrt{(3-1)^2 + (3-1)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.828 > 2$$

$$d(A_5, A_2) = \sqrt{(3-2)^2 + (3-1)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.236 > 2$$

$$d(A_5, A_3) = \sqrt{(3-2)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.414 \leq 2 \Rightarrow A_3 \in N_\epsilon(A_5)$$

$$d(A_5, A_4) = \sqrt{(3-6)^2 + (3-5)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.606 > 2$$

ফলাফল: $N_\epsilon(A_5) = \{A_5, A_3\}$, অর্থাৎ $|N_\epsilon(A_5)| = 2$ ।

Core Point নির্ধারণ: যেহেতু $2 < min_samples (3)$, তাই A_5 নিজে **Core Point** হতে পারল না।

Border Point নির্ধারণ: যদিও A_5 নিজে Core নয়, কিন্তু A_3 -এর Neighborhood পুনরায় পরীক্ষা করলে দেখা যায় A_5 তার ভেতরে অবস্থিত ($d(A_3, A_5) = 1.414 \leq \epsilon$), এবং A_3 ইতিমধ্যে একটি প্রমাণিত Core Point। যেহেতু —

$$|N_\epsilon(A_5)| < min_samples \quad \text{AND} \quad A_5 \in N_\epsilon(A_3), \quad A_3 \rightarrow \text{Core}$$

— তাই গাণিতিক শর্ত অনুযায়ী:

$$A_5 \rightarrow \text{Border Point}$$

Cluster Assignment (আপডেট করা)

যেহেতু A_5 , Core Point A_3 -এর প্রতিবেশী, তাই Cluster Expansion প্রক্রিয়ায় (যা A_1 থেকে শুরু হয়ে A_2, A_3 হয়ে সম্প্রসারিত হয়েছিল) A_3 প্রসেস হওয়ার সময় A_5 স্বয়ংক্রিয়ভাবে **Cluster 0**-এ যুক্ত হয়ে যাবে, যদিও এটি নিজে Cluster আর সম্প্রসারিত করবে না (কারণ এটি নিজে Core নয়)।

Progress Table (সম্পূর্ণ, ৫টি পয়েন্ট)

Point	Coordinates	Neighborhood $N_\epsilon(P)$	Neighbor Count	Point Type	Assigned Cluster
A_1	(1,1)	$\{A_1, A_2, A_3\}$	3	Core	0
A_2	(2,1)	$\{A_1, A_2, A_3\}$	3	Core	0
A_3	(2,2)	$\{A_1, A_2, A_3, A_5\}$	4	Core	0
A_4	(6,5)	$\{A_4\}$	1	Noise	-1
A_5	(3,3)	$\{A_3, A_5\}$	2	Border	0

Interpretation

এই সম্প্রসারিত উদাহরণটি DBSCAN-এর তিনটি বিন্দু-শ্রেণিকেই (Core, Border, Noise) একটি একক ডেটাসেটে সম্পূর্ণভাবে প্রদর্শন করে। লক্ষণীয় যে A_5 যদিও নিজে ঘনত্বের শর্ত পূরণ করতে পারেনি (মাত্র ২টি প্রতিবেশী), তবুও এটি সম্পূর্ণ Noise হিসেবে বাদ যায়নি — কারণ এটি একটি প্রমাণিত Core Point (A_3)-এর সংস্পর্শে (Density-Reachable) রয়েছে। এটিই DBSCAN-এর মূল শক্তি: এটি শুধু প্রতিটি পয়েন্টকে আলাদাভাবে বিচার করে না, বরং পুরো Density Chain বিশ্লেষণ করে সিদ্ধান্ত নেয়। অন্যদিকে A_4 কোনো Core Point-এর Neighborhood-এর মধ্যেই পড়েনি বলে এটি চূড়ান্তভাবে Noise (-1) হিসেবেই থেকে যায়।